



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филиал) № 8 «Компьютерные науки и прикладная математика»

Образовательный центр института № 8

Группа М8О-209СВ-24 Направление подготовки 02.04.02 «Фундаментальная информатика и информационные технологии»

Профиль Искусственный интеллект и суперкомпьютерные технологии

Квалификация: инженер-исследователь

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

На тему: Оптимизация холодного старта и масштабирования моделей машинного обучения в бессерверном инференсе для обеспечения высокой доступности

Автор ВКР: _____ Блинов Максим Алексеевич (_____)

Научный руководитель: _____ Булакина Мария Борисовна (_____)

Консультант: _____ — (_____)

Консультант: _____ — (_____)

Рецензент: _____ Ахмед Самир Халид (_____)

К защите допустить

Заведующий образовательным

центром института № 8

_____ июня 2026 года

_____ Крылов Сергей Сергеевич (_____)

Москва 2026

РЕФЕРАТ

Выпускная квалификационная работа состоит из 114 страниц, 12 рисунков, 7 таблиц, 42 использованных источников.

ИИ, ОБЛАЧНАЯ ПЛАТФОРМА, БОЛЬШАЯ ЯЗЫКОВАЯ МОДЕЛЬ, МАШИННОЕ ОБУЧЕНИЕ, БЕССЕРВЕРНЫЕ ВЫЧИСЛЕНИЯ, ХОЛОДНЫЙ СТАРТ, ИНФЕРЕНС, КЭШИРОВАНИЕ, *KUBERNETES*, *KUBEBUILD*, *CONTAINER STORAGE INTERFACE*, *P2P*, *FUSE*, МИКРОСЕРВИСЫ, *GOLANG*, *HIGHLOAD*

Объект исследования — инфраструктура бессерверного инференса моделей машинного обучения, развернутая в среде *Kubernetes* с применением механизма масштабирования до нуля.

Предмет исследования — методы кэширования, доставки и виртуализации доступа к файлам весов *ML*-моделей при холодном старте инференс-подов в *Kubernetes*-кластере.

Цель работы — Сокращение задержки холодного старта, уменьшение нагрузки на внешние реестры моделей и повышение доступности системы при горизонтальном масштабировании путем разработки архитектуры распределенного кэширования файлов моделей машинного обучения для бессерверного инференса в *Kubernetes*. Для достижения поставленной цели проведен анализ причин холодного старта и влияния роста размеров *ML*-моделей на инфраструктуру инференса; исследованы существующие распределенные решения для кэширования моделей — *Alluxio*, *JuiceFS*, *Dragonfly*, *CubeFS*, *Rook/Ceph*; реализована специализированная система распределенного *chunk*-кэша, включающая агент *storage-agent*, работающий как *DaemonSet* на каждой *worker*-ноде, *FUSE*-клиент *storage-mounter*, предоставляющий *ML-runtime* стандартный файловый интерфейс, и *Redis*-кластер в роли реестра метаданных чанков и активных агентов; сформирована методика оценки эффективности предложенной архитектуры.

Ключевые методы — трехуровневое кэширование по схеме «локальный диск — соседняя нода — внешний репозиторий»; *FUSE*-монтирование виртуальной файловой системы; *P2P*-доставка чанков между агентами по *TCP*; локальная связь через *Unix Domain Socket*; *Redis heartbeat* как реестр метаданных; атомарная запись чанков во избежание гонки и частично загруженных данных.

Ожидаемые результаты — снижение количества повторных загрузок весов из внешнего репозитория, уменьшение нагрузки на *HuggingFace Hub*, ускорение холодного старта при наличии локального или соседнего кэша, устранение *NFS* как единственной точки отказа.

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	9
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	11
ВВЕДЕНИЕ	13
1 ПРОБЛЕМА ХОЛОДНОГО СТАРТА <i>ML</i> -МОДЕЛЕЙ В БЕССЕРВЕРНОМ ИНФЕРЕНСЕ	19
1.1 Эволюция размеров моделей машинного обучения	19
1.1.1 От компактных сетей к масштабным языковым моделям	19
1.1.2 Экспоненциальный рост весов языковых моделей	20
1.1.3 Законы масштабирования как движущая сила роста	22
1.2 Механизм холодного старта в <i>Kubernetes</i>	23
1.2.1 Декомпозиция задержки холодного старта	23
1.2.2 Фаза планирования: T_{schedule}	23
1.2.3 Фазы загрузки и распаковки образа: $T_{\text{pull}} + T_{\text{unpack}}$	23
1.2.4 Фаза <i>init</i> -контейнера: T_{init}	24
1.2.5 Фаза загрузки весов из репозитория: T_{download}	24
1.2.6 Ограничения схемы с <i>emptyDir</i>	25
1.2.7 Фаза загрузки в <i>GPU</i> -память: T_{load}	25
1.2.8 Итоговый характер задержки	26
1.3 Влияние холодного старта на <i>SLA</i> , пользовательский опыт и стоимость	26
1.3.1 <i>SLA</i> и метрики задержки	26
1.3.2 Закон Литтла: накопление ожидающих запросов	27
1.3.3 Вероятность холодного старта при стратегии <i>scale-to-zero</i>	28
1.3.4 Дилемма <i>scale-to-zero</i> и <i>over-provisioning</i>	29
1.3.5 Влияние на пользовательский опыт	29
1.4 Требования к системе оптимизации холодного старта	30
1.4.1 Функциональные требования	30
1.4.2 Нефункциональные требования	31
1.5 Выводы по разделу	32
2 АНАЛИЗ СУЩЕСТВУЮЩИХ РЕШЕНИЙ ДЛЯ ХРАНЕНИЯ И КЭШИРОВАНИЯ МОДЕЛЕЙ	34
2.1 Критерии сравнения решений	34
2.2 <i>Alluxio</i>	35

2.2.1	Архитектура	36
2.2.2	Преимущества	37
2.2.3	Недостатки	37
2.2.4	Вывод по <i>Alluxio</i>	38
2.3	<i>JuiceFS</i>	38
2.3.1	Архитектура	38
2.3.2	Ограничения для <i>serverless cold start</i>	39
2.3.3	Вывод по <i>JuiceFS</i>	40
2.4	<i>Dragonfly</i> , <i>CubeFS</i> и <i>Rook/Ceph</i>	40
2.4.1	<i>Dragonfly</i>	40
2.4.2	<i>CubeFS</i>	41
2.4.3	<i>Rook/Ceph</i>	42
2.5	Сравнительная таблица	44
2.6	Обоснование разработки специализированного решения	45
2.7	Выводы по разделу	46
3	ПРОМЕЖУТОЧНОЕ РЕШЕНИЕ НА БАЗЕ <i>NFS</i> И КООРДИНАЦИИ ЗАГРУЗОК	48
3.1	Исходная схема: <i>storage-initializer</i> и <i>emptyDir</i>	48
3.1.1	Паттерн <i>init</i> -контейнера и жизненный цикл тома <i>emptyDir</i>	48
3.1.2	Фундаментальный недостаток: повторные загрузки	49
3.2	Переход к <i>NFS</i> как общему хранилищу	50
3.2.1	<i>NFS</i> и режим <i>ReadWriteMany</i>	50
3.2.2	Интеграция в <i>Kubernetes</i> : <i>PV</i> и <i>PVC</i>	50
3.2.3	Устранение повторных загрузок	51
3.3	Проблема параллельного скачивания и условие гонки	52
3.3.1	<i>Race condition</i> при первом развёртывании	52
3.3.2	Алгоритм координированной загрузки	53
3.4	Файловые блокировки <i>flock</i> и их особенности поверх <i>NFS</i>	55
3.4.1	<i>Advisory locking</i> : разделяемые и эксклюзивные блокировки	55
3.4.2	<i>NFSv3</i> : блокировки через <i>Network Lock Manager</i>	56
3.4.3	<i>NFSv4</i> : встроенные блокировки и <i>lease</i> -механизм	56
3.4.4	Практическая модель надёжности: <i>flock</i> + <i>rename</i> + маркер	57
3.5	<i>Kubernetes Lease</i> и <i>Leader Election</i>	57
3.5.1	Ресурс <i>coordination.k8s.io/v1/Lease</i>	57
3.5.2	Алгоритм <i>client-go leaderelection</i>	58

3.5.3	Координация <i>per-model</i> через пространство имён <i>Lease</i> . . .	59
3.6	Ограничения <i>NFS</i> -этапа	60
3.6.1	Централизованный сервер как единая точка отказа	60
3.6.2	<i>CAP</i> -теорема и выбор <i>NFS</i>	61
3.7	Выводы по разделу	62
4	РАЗРАБОТКА <i>P2P/FUSE</i> -СИСТЕМЫ КЭШИРОВАНИЯ ML-МОДЕЛЕЙ	63
4.1	Общая идея и требования к архитектуре	63
4.2	Модель данных: репозиторий, модель, ревизия, файл, чанк . . .	66
4.3	<i>Redis</i> как реестр метаданных чанков	67
4.3.1	Обоснование выбора <i>Redis</i>	67
4.3.2	Схема ключей	69
4.3.3	<i>Heartbeat</i> и управление жизненным циклом	70
4.4	Обнаружение узлов: <i>Gossip</i> , <i>mDNS</i> и <i>Redis heartbeat</i>	70
4.4.1	<i>Gossip Protocol</i>	70
4.4.2	<i>mDNS/UDP Broadcast</i>	71
4.4.3	<i>Redis heartbeat</i> : выбранный подход	72
4.5	Компонент <i>storage-agent</i> : <i>DaemonSet</i> -агент на каждой ноде . . .	73
4.5.1	<i>HTTP API</i> и текущая реализация	74
4.5.2	Поддержка <i>singleflight</i> для предотвращения <i>thundering herd</i> . . .	75
4.5.3	Целевое <i>P2P</i> -расширение	75
4.5.4	Многоуровневый кэш <i>storage-agent</i>	76
4.6	Компонент <i>storage-mounter</i> : <i>FUSE</i> -клиент	77
4.6.1	Механизм <i>FUSE</i>	77
4.6.2	Ключевые операции	79
4.6.3	Трансляция <i>read(offset, size)</i> в <i>chunk</i> -запросы	79
4.6.4	Текущая реализация и целевое развитие	80
4.6.5	Кэш и буферизация чтения в <i>storage-mounter</i>	80
4.7	<i>UDS</i> и <i>TCP</i> : разделение локальной и межнодовой коммуникации . . .	82
4.7.1	<i>Unix Domain Socket</i> : внутриузловая связь	82
4.7.2	<i>TCP</i> : межнодовая связь	82
4.7.3	Оценка пропускной способности через <i>BDP</i>	83
4.8	Алгоритм чтения чанка	84
4.8.1	Полная тринадцатиступенчатая последовательность	84
4.8.2	Модель среднего времени доступа (<i>AMAT</i>)	85

4.9	Политика кэширования и вытеснения	86
4.9.1	Распределение популярности моделей: закон Ципфа	86
4.9.2	<i>LRU</i> как политика вытеснения	86
4.9.3	Аппроксимация Чи для оценки <i>hit rate</i>	87
4.10	Параллельная загрузка, выбор пира и <i>prefetching</i>	87
4.10.1	Алгоритм выбора пира	87
4.10.2	Параллельная загрузка и закон Амдала	88
4.10.3	Оптимальный размер чанка	89
4.10.4	Алгоритм <i>prefetching</i>	89
4.11	Надёжность и высокая доступность	90
4.11.1	Формула доступности при репликации	90
4.11.2	<i>TTL</i> как защита от <i>stale metadata</i>	91
4.11.3	<i>Fallback</i> и <i>degraded mode</i>	91
4.11.4	<i>Eventual consistency</i> для метаданных	92
4.12	Выводы по разделу	92
5	ОЦЕНКА ЭФФЕКТИВНОСТИ И НАДЁЖНОСТИ ПРЕДЛОЖЕННОГО ПОДХОДА	94
5.1	Методика экспериментов	94
5.1.1	Тестовый стенд	94
5.1.2	Процедура эксперимента	94
5.1.3	Сравниваемые сценарии	95
5.2	Метрики оценки	95
5.3	Расчётная модель задержки	97
5.3.1	Трёхуровневая модель <i>AMAT</i>	97
5.3.2	Характерные значения компонентов задержки	97
5.3.3	Расчёт <i>AMAT</i> при различных сценариях	98
5.3.4	Время загрузки модели и применение формулы <i>P2P</i>	98
5.3.5	Закон Амдала и предел параллельного ускорения	99
5.3.6	Применение закона Литтла	99
5.4	Ожидаемые результаты сравнения	100
5.5	Анализ сценариев отказов	104
5.5.1	Падение <i>worker</i> -ноды с чанками	104
5.5.2	Устаревшая запись в <i>Redis</i> (<i>stale metadata</i>)	104
5.5.3	Недоступность <i>Redis</i>	104
5.5.4	Недоступность <i>HuggingFace Hub</i>	104

5.5.5	Частичная запись чанка (<i>crash</i> во время записи)	105
5.5.6	Сетевое разбиение кластера	105
5.6	Ограничения предложенного подхода	105
5.7	Выводы по разделу	106
ЗАКЛЮЧЕНИЕ		108
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ		112

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей выпускной квалификационной работе применяют следующие термины с соответствующими определениями:

Инференс — процесс получения предсказаний от обученной модели машинного обучения по новым входным данным без изменения ее параметров

Холодный старт — состояние, при котором инстанс сервиса инференса создается с нуля и перед обработкой первого запроса должен выполнить полную инициализацию, включая загрузку файлов модели

Веса модели — набор числовых параметров нейросети, определяющих ее поведение и загружаемых в оперативную и видеопамять перед инференсом

Чанк — фрагмент файла весов фиксированного размера, используемый как минимальная единица кэширования, репликации и *P2P*-передачи

Бессерверные вычисления — модель развертывания, при которой платформа автоматически управляет выделением и освобождением вычислительных ресурсов в зависимости от нагрузки, включая масштабирование до нуля

Pod — минимальная единица развертывания в *Kubernetes*, объединяющая один или несколько контейнеров с общим сетевым пространством имен и томами хранения

DaemonSet — ресурс *Kubernetes*, обеспечивающий запуск экземпляра служебного пода на каждой подходящей *worker*-ноде кластера

Scale-to-zero — способность платформы снижать число активных реплик сервиса до нуля при отсутствии входящих запросов

Control Plane — плоскость управления, в рамках работы представленная *Redis*-реестром метаданных о чанках и состоянии агентов

Data Plane — плоскость данных, включающая локальные диски *worker*-нод, каналы доставки чанков и компоненты, непосредственно обслуживающие чтение весов модели

Реестр метаданных — служба, хранящая сведения о расположении чанков, их размере, контрольных суммах и доступных узлах-держателях

Файловая система в пространстве пользователя (*FUSE*) — механизм *Linux*, позволяющий реализовать файловую систему как пользовательский процесс с обработкой обращений через *VFS*

Сокет домена *Unix* — механизм межпроцессного взаимодействия внутри одного узла, использующий файловую систему вместо *IP*-стека

Сетевая файловая система — протокол распределенного доступа к удаленным файлам, позволяющий монтировать удаленный каталог как локальный

flock — системный вызов для установки рекомендательных файловых блокировок, используемый для координации параллельных загрузок

Lease — ресурс *Kubernetes Coordination API*, представляющий временной токен владения и применяемый для распределенной координации

Leader Election — алгоритм выбора единственного активного исполнителя привилегированной операции среди конкурирующих процессов

Cache hit — ситуация, при которой запрошенные данные найдены в кэше и возвращаются без обращения к более медленному источнику

Cache miss — ситуация, при которой данные отсутствуют в текущем уровне кэша и требуется переход к следующему источнику

Fallback — механизм переключения на резервный источник данных при отсутствии или недоступности основного пути чтения

Единственная точка отказа — компонент системы, отказ которого приводит к полной недоступности критически важной функциональности

Обнаружение узлов — процесс получения актуального списка агентов, которые могут обслуживать запросы на чтение конкретных чанков модели

Предзагрузка — заблаговременная загрузка следующих чанков на основе наблюдаемого паттерна чтения для сокращения задержки последующих запросов

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей выпускной квалификационной работе применяют следующие сокращения и обозначения:

ML — *Machine Learning* — машинное обучение

LLM — *Large Language Model* — большая языковая модель

SLA — *Service Level Agreement* — соглашение об уровне обслуживания

SLO — *Service Level Objective* — целевой показатель уровня обслуживания

K8s — *Kubernetes* — система оркестрации контейнеров

P2P — *Peer-to-Peer* — одноранговая сеть

FUSE — *Filesystem in Userspace* — файловая система в пространстве пользователя

UDS — *Unix Domain Socket* — сокет домена *Unix*

TCP — *Transmission Control Protocol* — протокол управления передачей

NFS — *Network File System* — сетевая файловая система

PV — *PersistentVolume* — постоянный том *Kubernetes*

PVC — *PersistentVolumeClaim* — заявка на постоянный том *Kubernetes*

RWX — *ReadWriteMany* — режим доступа к тому *Kubernetes* для нескольких нод

API — *Application Programming Interface* — программный интерфейс приложения

RTT — *Round-Trip Time* — время двойного прохода сигнала

BDP — *Bandwidth-Delay Product* — произведение полосы пропускания на задержку

IPC — *Inter-Process Communication* — межпроцессное взаимодействие

VFS — *Virtual File System* — виртуальная файловая система

LRU — *Least Recently Used* — алгоритм вытеснения наименее давно используемых данных

TTL — *Time-To-Live* — время жизни записи в кэше или реестре

HA — *High Availability* — высокая доступность

SPOF — *Single Point of Failure* — единственная точка отказа

CDN — *Content Delivery Network* — сеть доставки содержимого

CNN — *Convolutional Neural Network* — сверточная нейронная сеть

RNN — *Recurrent Neural Network* — рекуррентная нейронная сеть

GPU — *Graphics Processing Unit* — графический процессор

CPU — *Central Processing Unit* — центральный процессор

AMAT — *Average Memory Access Time* — среднее время доступа к данным

mDNS — *Multicast DNS* — многоадресный *DNS*

CNI — *Container Network Interface* — сетевой интерфейс контейнеров

CSI — *Container Storage Interface* — интерфейс хранилища контейнеров

ВВЕДЕНИЕ

1 Актуальность темы

Развитие методов машинного обучения привело к смещению отрасли от относительно небольших классических моделей к крупным языковым моделям (*LLM*, *Large Language Model*), мультимодальным архитектурам и диффузионным генераторам. Число параметров современных моделей выросло с сотен миллионов (*BERT* [1], 2018) до триллионов, что особенно наглядно демонстрирует *DeepSeek V4 Pro* [2]; вместе с этим размер хранимых весов увеличился с сотен мегабайт до терабайтного диапазона. Несмотря на тенденцию к повышению плотности возможностей моделей [3], *state-of-the-art* результаты по-прежнему требуют архитектур колоссального масштаба.

Одновременно индустрия уверенно движется в сторону бессерверного инференса (*serverless inference*) — модели эксплуатации, при которой облачная платформа автоматически масштабирует сервисы в ответ на изменение нагрузки, включая масштабирование до нуля активных реплик при отсутствии входящих запросов. Данная модель реализована в таких платформах, как *Knative* [4] — открытый стандарт на базе *Kubernetes*, поддерживаемый *Google*, *IBM* и *Red Hat*; *OpenFaaS* — система развёртывания функций в *Kubernetes*-кластере; *Amazon Web Services Lambda* — управляемая *serverless*-платформа публичного облака; а также *KServe* [5] — стандартизованная платформа для *ML*-инференса с поддержкой *scale-to-zero*. Эти платформы позволяют значительно экономить ресурсы кластера: ноды не удерживают вычислительные мощности для простаивающих сервисов, а плата взимается только за фактически обработанные запросы.

Однако *serverless*-модель вступает в острое противоречие с природой современных *ML*-моделей. Перед началом обработки запросов контейнер инференса должен загрузить файлы весов из внешнего репозитория или общего хранилища. Для модели весом 15–350 ГБ такая загрузка занимает от нескольких минут до десятков минут даже по высокоскоростному каналу. По оценкам, опубликованным командой *BentoML* [6], при стандартном подходе загрузка весов *LLM*-модели размером 15 ГБ может занимать свыше 10 минут, что неприемлемо для большинства реальных сервисов. Таким образом, холодный старт (*cold start*) — задержка от момента появления запроса до готовности сервиса его обработать — становится не техническим

неудобством, а фактором, определяющим выполнимость *SLA* (*Service Level Agreement*, соглашение об уровне обслуживания) в продуктивных системах.

Актуальность задачи оптимизации холодного старта подтверждается и отраслевыми тенденциями: доля *ML*-нагрузок в публичных облаках стремительно растёт, а требования к задержке инференса ужесточаются. Пользовательские интерфейсы генеративного ИИ — чаты, ассистенты, системы рекомендаций — создают нагрузки с ярко выраженной пиковостью, при которых платформа обязана быстро масштабировать парк реплик. С экономической точки зрения *serverless*-инференс привлекателен: оператор платит только за реально затраченные вычислительные ресурсы, а не за простаивающие *GPU*-инстансы. Однако реализация этой экономии требует, чтобы время масштабирования сервиса было значительно меньше времени ожидания пользователя. Для нагрузок реального времени приемлемое время ответа составляет единицы секунд, тогда как задержка холодного старта при загрузке крупных моделей измеряется минутами. Это противоречие делает задачу оптимизации холодного старта одной из центральных в области *MLOps* и инфраструктуры *ML*-платформ.

2 Проблема и противоречие

Классический подход к загрузке весов в *Kubernetes* использует *init*-контейнер *storage-initializer*, который скачивает файлы из внешнего репозитория (*HuggingFace*, *Ollama*, *ModelScope*) в том *emptyDir*, после чего основной *runtime*-контейнер загружает эти веса в *GPU*-память. При горизонтальном масштабировании каждая новая реплика повторяет загрузку заново, не используя данные, уже скачанные другими репликами или оставшиеся на диске ноды от предыдущих запусков.

Масштаб проблемы нагляден на конкретном примере. Предположим, что *burst*-событие порождает 10 реплик инференса для модели *DeepSeek-LLM 7B* с весами объёмом порядка 15 ГБ. При использовании *emptyDir*-подхода каждая реплика независимо скачивает модель из *HuggingFace Hub*:

$$V_{\text{total}} = N \times S_{\text{model}} = 10 \times 15 \text{ ГБ} = 150 \text{ ГБ}. \quad (1)$$

где V_{total} — суммарный внешний трафик при *burst*-масштабировании, N — число одновременно запускаемых реплик, S_{model} — объём весов одной модели. Это 150 ГБ внешнего трафика за одно *burst*-событие. При типичной

ширине канала 1 Гбит/с (125 МБ/с) каждая реплика тратит $15,000/125 \approx 120$ с ≈ 2 мин на загрузку. Все 10 реплик стартуют с задержкой около 2 минут, конкурируя за один и тот же канал к внешнему репозиторию, что на практике увеличивает реальное время ещё в 2–3 раза. Суммарная нагрузка на внешний канал составляет 150 ГБ — тогда как при наличии кэша достаточно загрузить модель один раз и распределить чанки внутри кластера.

Существующий паллиативный подход — *NFS*-хранилище — устраняет повторные загрузки из внешнего репозитория: первый под скачивает модель, остальные читают из общей директории. Однако *NFS* вводит центральную точку отказа (*SPOF*, *Single Point of Failure*) и *bottleneck* пропускной способности: при одновременном чтении десяти репликами через один *NFS*-сервер сеть и диск сервера становятся узким местом. Деградация или недоступность *NFS*-сервера делает недоступными все сервисы инференса.

Таким образом, существует противоречие: бессерверное масштабирование требует быстрого старта произвольного числа реплик, тогда как существующие механизмы доставки весов либо порождают избыточный внешний трафик (*emptyDir*), либо вводят централизованное узкое место (*NFS*). Требуется механизм, который сохраняет преимущества *serverless*-масштабирования, но устраняет повторную загрузку уже известных кластеру весов без введения новой единственной точки отказа.

3 Цель работы

Целью работы является сокращение задержки холодного старта, уменьшение нагрузки на внешние реестры моделей и повышение доступности системы при горизонтальном масштабировании путем разработки архитектуры распределенного кэширования файлов моделей машинного обучения для бессерверного инференса в *Kubernetes*.

4 Задачи работы

Для достижения поставленной цели необходимо решить следующие задачи:

- а) **Проанализировать причины холодного старта *ML*-моделей в *Kubernetes*** — исследовать зависимость задержки инициализации от размера весов, описать механизм работы *storage-initializer* и

emptyDir, формализовать декомпозицию времени старта и применить закон Литтла для оценки нагрузки на систему.

- б) **Исследовать существующие решения** для распределённого хранения и кэширования моделей — провести сравнительный анализ *Alluxio*, *JuiceFS*, *Dragonfly*, *CubeFS* и *Rook/Ceph* по критериям *POSIX*-совместимости, накладных расходов на метаданные, зависимости от внешней инфраструктуры и применимости к *serverless*-инференсу.
- в) **Разработать и оценить промежуточный подход на базе NFS** с механизмами координации параллельных загрузок — исследовать файловые блокировки (*flock*), ресурс *Lease* и алгоритм *Leader Election*; описать ограничения *NFS* как централизованного решения.
- г) **Исследовать механизмы координации в Kubernetes** — детально проанализировать файловые блокировки (*flock*) над *NFS*, ресурс *Lease* из *Coordination API*, алгоритм *Leader Election* из библиотеки *client-go* и обосновать применимость каждого механизма.
- д) **Спроектировать архитектуру распределённого *chunk*-кэша** с локальным хранилищем на каждой *worker*-ноде — разработать трёхуровневую схему доступа: *local cache hit* → *peer cache hit* → *remote fallback*; определить протоколы взаимодействия компонентов.
- е) **Обосновать выбор *Redis*** в роли реестра метаданных чанков и активных *storage-agent* — исследовать *TTL-based heartbeat* как механизм *discovery*, сравнить с *gossip*-протоколом и *mDNS*, обосновать конфигурации *Redis Sentinel* и *Redis Cluster*.
- ж) **Обосновать применение *FUSE* (*Filesystem in Userspace*)** для предоставления *runtime*-контейнеру стандартного файлового интерфейса — проанализировать механизм *VFS*-перехватов, оценить накладные расходы *context switch* и преимущества прозрачной интеграции с любым *ML-framework*.
- и) **Сравнить *Unix Domain Socket* и *TCP*** и обосновать их роли в локальной и междодовой коммуникации — формализовать разницу в задержке и накладных расходах; обосновать применение *UDS* для канала *storage-mounter* → *storage-agent* и *TCP* для канала между агентами на разных нодах.

- к) **Реализовать компоненты *storage-agent* и *storage-mounter*** — реализовать локальный *chunk*-кэш на дисковом хранилище, *FUSE*-файловую систему с *lazy-loading*, интерфейс к *HuggingFace Hub* и интеграцию между компонентами.
- л) **Сформировать методику оценки эффективности и надёжности** предложенной архитектуры — построить расчётную модель на базе *AMAT*, проанализировать сценарии отказов, выявить ограничения и определить направления дальнейшего развития.

5 Объект и предмет исследования

Объект исследования — системы развёртывания и масштабирования *ML*-инференса в *Kubernetes/serverless*-средах.

Предмет исследования — методы кэширования, доставки и виртуализации доступа к файлам весов *ML*-моделей при холодном старте.

6 Научная новизна и практическая значимость

Научная новизна работы состоит в следующем:

- Предложена специализированная архитектура, объединяющая *FUSE*-доступ, локальный *chunk*-кэш на дисках *worker*-нод, *Redis*-реестр метаданных и *P2P*-доставку чанков для сценария *serverless ML*-инференса в *Kubernetes*.
- Обосновано предпочтение *Redis heartbeat* перед *gossip-discovery* и *mDNS* для малых и средних кластеров с жёстким требованием к задержке на пути *cold start*.
- Формализовано разделение локальной (*UDS*) и межнодовой (*TCP*) коммуникации как инструмент минимизации накладных расходов при гарантии совместимости с *Kubernetes networking*.

Практическая значимость: снижение времени повторного старта модели за счёт попадания в локальный или соседний кэш; уменьшение нагрузки на *HuggingFace Hub* и внешний канал при масштабировании реплик; возможность использовать локальные диски всех *worker*-нод как распределённый кэш без выделенного хранилищного кластера; устранение *NFS* как единственной точки отказа.

7 Структура работы

Работа состоит из введения, пяти разделов, заключения, списка литературы и приложений.

В первом разделе проанализирована эволюция размеров *ML*-моделей, описан механизм холодного старта в *Kubernetes* и его влияние на *SLA*.

Второй раздел содержит сравнительный анализ существующих решений для хранения и кэширования моделей и обоснование необходимости специализированной разработки.

В третьем разделе описан промежуточный *NFS*-этап с алгоритмами координации загрузок, его преимущества и ограничения.

Четвёртый раздел посвящён основной разработке — *P2P/FUSE*-архитектуре с описанием компонентов *storage-agent*, *storage-mounter* и *Redis*-реестра.

Пятый раздел содержит методику оценки, расчётную модель задержки, ожидаемые результаты сравнения и анализ ограничений.

1 ПРОБЛЕМА ХОЛОДНОГО СТАРТА *ML*-МОДЕЛЕЙ В БЕССЕРВЕРНОМ ИНФЕРЕНСЕ

Настоящий раздел посвящён обоснованию актуальности проблемы холодного старта в системах бессерверного инференса моделей машинного обучения. Рассматривается эволюция архитектур и размеров *ML*-моделей, показывающая, что объём весовых файлов вырос от единиц мегабайт до сотен гигабайт. Раскрывается механизм каждого этапа холодного старта в *Kubernetes*, обосновывается, почему классическая схема с *init*-контейнером и томом *emptyDir* неэффективно использует уже загруженные данные. Оцениваются последствия для соблюдения *SLA* и пользовательского опыта с применением закона Литтла и вероятностной модели холодного старта. Формулируются функциональные и нефункциональные требования к системе оптимизации.

1.1 Эволюция размеров моделей машинного обучения

1.1.1 От компактных сетей к масштабным языковым моделям

На раннем этапе развития глубокого обучения доминировали сверточные нейронные сети (*CNN*), ориентированные преимущественно на задачи компьютерного зрения. Прорывная работа Крижевского, Суцкевера и Хинтона представила модель *AlexNet* в 2012 году: сеть насчитывала 60 миллионов параметров, а объём её весовых файлов в 32-битном формате составлял около 233 МБ [7]. При типичной для того времени пропускной способности сети порядка 100 Мбит/с такой объём передавался за несколько секунд и не создавал значимых инфраструктурных проблем. Параллельно развивались рекуррентные нейронные сети (*RNN*) и их разновидности *LSTM* и *GRU* для задач обработки последовательностей, однако их размеры также оставались в пределах сотен мегабайт.

Качественный переход произошел в 2017 году с публикацией архитектуры *Transformer*. Механизм самовнимания (*self-attention*) обеспечил возможность эффективно обрабатывать длинные контекстные зависимости и параллелизовать обучение, что сделало *Transformer* доминирующей архитектурой для задач обработки естественного языка [1]. Вместе с этим значительно выросли и вычислительные требования.

Модель *BERT Base* (2018) с 110 миллионами параметров занимала около

440 МБ в 32-битном формате; версия *BERT Large* содержала 340 миллионов параметров и требовала порядка 1.4 ГБ [1]. Уже на этом этапе загрузка весов из удалённого хранилища начинала ощутимо влиять на задержку инициализации сервиса.

1.1.2 Экспоненциальный рост весов языковых моделей

Выход *GPT-3* в 2020 году ознаменовал качественный скачок: 175 миллиардов параметров потребовали около 350 ГБ дискового пространства для хранения весов в 16-битном формате [8]. Для загрузки такого объёма по каналу 10 Гбит/с требуется около 280 секунд — почти пять минут только на передачу данных. В реальных условиях с конкурентными запросами из других подов это время возрастает существенно.

Тенденция к открытым моделям, усиленная развитием семейства *DeepSeek*, расширила доступ исследователей и производственных команд к крупным моделям. В работе *DeepSeek-LLM* рассматриваются конфигурации 7B и 67B параметров, а расчётный объём основных весов варианта *DeepSeek-LLM 67B* в формате *FP16* составляет около 134 ГБ [9]. Следующее поколение, *DeepSeek V2*, уже переходит в диапазон сотен миллиардов параметров [10].

Значения в таблице получены как произведение числа параметров на два байта, то есть описывают только основные тензоры в *FP16* и используют десятичный гигабайт (10^9 байт). Фактический размер репозитория зависит от формата сериализации, разбиения на файлы и служебных данных. Для *Mixtral 8×22B* приведены все 141 млрд параметров, хотя на одном токене активируется 39 млрд. Число параметров приведено по первичным публикациям и карточкам моделей [1; 11; 12; 8; 9; 13; 14; 10; 15; 2].

Таблица 1 демонстрирует, что за период с 2018 по 2026 год расчётный объём весов ведущих моделей вырос от сотен мегабайт до терабайт — почти на четыре порядка. Если для *BERT Base* загрузка занимала секунды, то для *DeepSeek V4 Pro* при той же пропускной способности сети она может занимать часы. Именно это изменение масштаба превращает загрузку весов из тривиальной операции в доминирующий компонент задержки холодного старта.

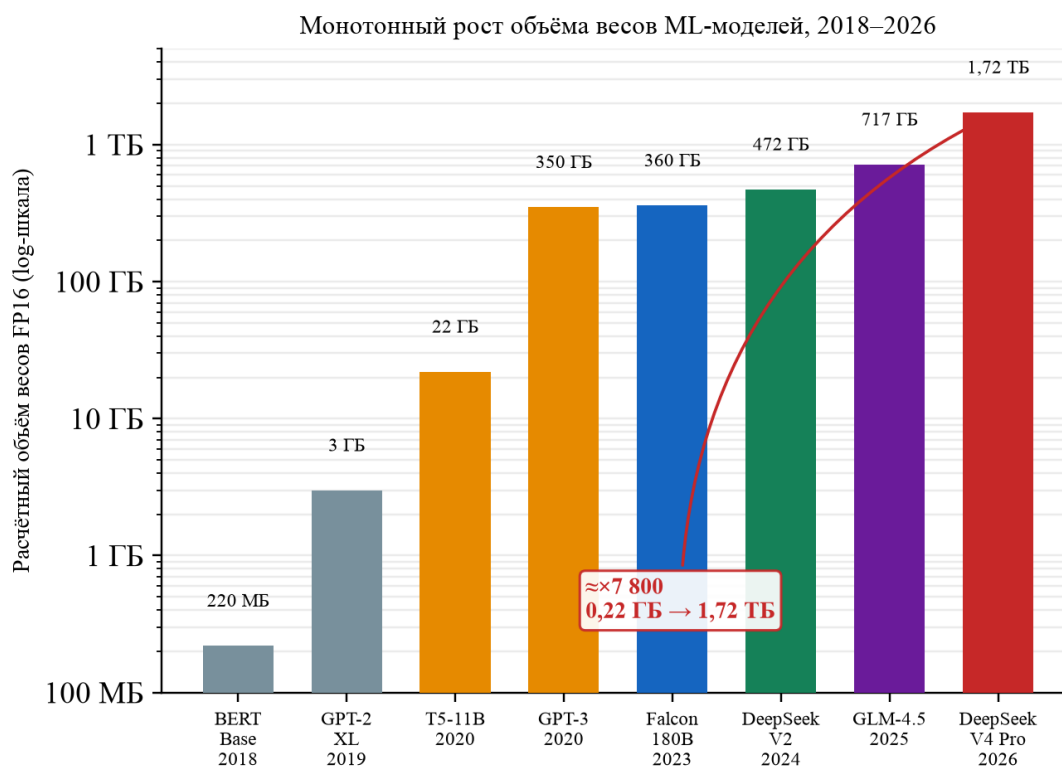


Рисунок 1 – Монотонная последовательность отобранных *ML*-моделей с возрастающим расчётным объёмом основных весов в *FP16*, 2018–2026 (логарифмическая шкала). График иллюстрирует рост выбранной последовательности, а не утверждает, что каждый выпуск соответствующего года крупнее всех других моделей.

Таблица 1 – Расчётный объём весов ключевых *ML*-моделей в формате *FP16*

Модель	Год	Архитек-тура	Пара-метры	Вес (<i>FP16</i>)	Влияние
<i>AlexNet</i>	2012	<i>CNN</i>	60 млн	≈0.12 ГБ	Незначительное
<i>BERT Base</i>	2018	<i>Transformer</i>	110 млн	≈0.22 ГБ	Малое
<i>GPT-2 XL</i>	2019	<i>Transformer</i>	1.5 млрд	≈3 ГБ	Умеренное
<i>T5-11B</i>	2020	<i>Transformer</i>	11 млрд	≈22 ГБ	Существенное
<i>GPT-3</i>	2020	<i>Transformer</i>	175 млрд	≈350 ГБ	Критическое
<i>DeepSeek-LLM 7B</i>	2024	<i>Transformer</i>	7 млрд	≈14 ГБ	Существенное
<i>DeepSeek-LLM 67B</i>	2024	<i>Transformer</i>	67 млрд	≈134 ГБ	Критическое
<i>Falcon 180B</i>	2023	<i>Transformer</i>	180 млрд	≈360 ГБ	Критическое
<i>DeepSeek V2</i>	2024	<i>MoE</i>	235.7 млрд	≈472 ГБ	Критическое
<i>GLM-4.5</i>	2025	<i>MoE</i>	358.3 млрд	≈717 ГБ	Критическое
<i>DeepSeek V4 Pro</i>	2026	<i>MoE</i>	861.6 млрд	≈1.72 ТБ	Критическое

1.1.3 Законы масштабирования как движущая сила роста

Рост числа параметров подчиняется так называемым законам масштабирования (*scaling laws*). Эмпирически установлено, что качество языковых моделей, измеряемое кросс-энтропийными потерями, улучшается в степенной зависимости от трёх факторов: объёма параметров модели, размера обучающего набора данных и доступных вычислительных ресурсов [8]. Из этих законов следует устойчивый технологический стимул к дальнейшему увеличению масштаба: большие модели достигают того же уровня качества с меньшим числом шагов обучения и могут демонстрировать способности, не наблюдаемые у меньших моделей.

Параллельно с ростом параметров развивается концепция «плотности возможностей» (*capability density*) [3]: компактные модели, такие как *Mistral 7B* и семейство *Phi-3*, достигают результатов, сопоставимых с более крупными аналогами, благодаря качественным обучающим данным и архитектурным оптимизациям. Однако даже модели в диапазоне 7–70 млрд параметров требуют примерно от 13 до 140 ГБ для хранения основных весов в *FP16* — объёмов, принципиально меняющих характер инфраструктурной

задачи.

Таким образом, рост размеров *ML*-моделей является устойчивой долгосрочной тенденцией, а значит, проблема загрузки весов при холодном старте будет лишь обостряться с течением времени.

1.2 Механизм холодного старта в *Kubernetes*

1.2.1 Декомпозиция задержки холодного старта

При развёртывании *ML*-инференса в *Kubernetes* каждый новый инстанс сервиса проходит ряд последовательных фаз. Суммарная задержка холодного старта формализуется следующим образом:

$$T_{\text{start}} = T_{\text{schedule}} + T_{\text{pull}} + T_{\text{unpack}} + T_{\text{init}} + T_{\text{download}} + T_{\text{load}}, \quad (2)$$

где T_{schedule} — время планировщика *Kubernetes* для назначения пода на конкретную ноду; T_{pull} — загрузка образа контейнера из реестра; T_{unpack} — декомпрессия и распаковка слоёв образа на диск; T_{init} — выполнение *init*-контейнера (в том числе *storage-initializer*); T_{download} — скачивание весов модели из внешнего репозитория; T_{load} — загрузка весов из файловой системы в оперативную память и *GPU*-видеопамять.

Каждое слагаемое вносит самостоятельный вклад в суммарную задержку. Рассмотрим их характеристики и причины, по которым традиционная схема не позволяет эффективно сократить наиболее затратные фазы.

1.2.2 Фаза планирования: T_{schedule}

Планировщик *Kubernetes* анализирует доступные ноды с учётом запрошенных ресурсов (*CPU*, память, *GPU*), ограничений через *nodeSelector* и *tolerations*, правил *affinity* и *topologySpreadConstraints*. При загруженном кластере или ожидании освобождения *GPU*-слота эта фаза может занимать от долей секунды до нескольких минут. Поскольку планировщик не учитывает наличие кэшированных весов модели на ноде, он может назначить под на ноду без нужных данных, вынуждая к их повторной загрузке.

1.2.3 Фазы загрузки и распаковки образа: $T_{\text{pull}} + T_{\text{unpack}}$

Образ контейнера для *ML*-инференса включает среду выполнения

языка, фреймворк (*PyTorch*, *TensorFlow*), *CUDA*-библиотеки и вспомогательные зависимости. Для типичного контейнера с *vLLM* или *Triton Inference Server* размер образа может составлять 15–25 ГБ. Загрузка по каналу 10 Гбит/с занимает ориентировочно 12–20 секунд для передачи данных, однако реальное время больше из-за последовательного характера протокола реестра контейнеров: слои образа обычно скачиваются без полноценного параллелизма [6].

После скачивания каждый слой образа декомпрессируется из формата *gzip* или *zstd* и записывается на диск. Эта операция *CPU*-интенсивна: декомпрессия 20 ГБ образа может занять от трёх до восьми минут. Механизм *Copy-on-Write*, используемый большинством драйверов хранения контейнеров (*overlay2*, *fuse-overlayfs*), добавляет накладные расходы при последующем доступе к файлам внутри образа.

1.2.4 Фаза *init*-контейнера: T_{init}

Классическая схема развёртывания *ML*-инференса в *Kubernetes* предполагает использование *init*-контейнера (*storage-initializer*), который запускается до основного *runtime*-контейнера. Его задача — скачать файлы весов модели из внешнего хранилища (*HuggingFace Hub*, *S3*, *GCS*) и записать их в том типа *emptyDir*, разделяемый с основным контейнером.

Принципиальная схема взаимодействия выглядит следующим образом: планировщик назначает под на ноду, после чего инициализируется том *emptyDir* на локальном диске этой ноды. Затем запускается *init*-контейнер *storage-initializer*, который последовательно скачивает файлы весов и записывает их в смонтированный *emptyDir*. После успешного завершения *init*-контейнера запускается *runtime*-контейнер (*vLLM*, *TGI*, *Triton*), который считывает файлы из *emptyDir* и загружает их в *GPU*-память.

1.2.5 Фаза загрузки весов из репозитория: T_{download}

Компонент T_{download} является наиболее критичным с точки зрения зависимости от размера модели. При линейном росте объёма весов это слагаемое растёт пропорционально. Для *DeepSeek-LLM 7B* (порядка 15 ГБ) при канале 1 Гбит/с без конкуренции загрузка занимает ориентировочно 120 секунд; для *DeepSeek-LLM 67B* (порядка 140 ГБ) — уже порядка

1120 секунд, то есть около 19 минут [6].

Ключевая особенность проблемы состоит в том, что при масштабировании до N реплик на разных нодах внешний репозиторий получает N независимых запросов на один и тот же набор файлов. Каждый *init*-контейнер выполняет полную загрузку, игнорируя тот факт, что те же данные могут уже присутствовать на соседних нодах кластера.

1.2.6 Ограничения схемы с *emptyDir*

Том *emptyDir* привязан к жизненному циклу пода и физически размещается на диске конкретной ноды. При завершении или перепланировании пода данные в *emptyDir* безвозвратно удаляются. Следствием является отсутствие механизма разделения данных между репликами на одной ноде: при масштабировании к двум репликам на одном узле каждая из них независимо скачивает полный набор весов. Отсутствует и возможность передачи уже загруженных данных между нодами кластера, что исключает локальное повторное использование при перепланировании подов.

Важно разграничить два типа повторной загрузки. Первый — внутринодовая избыточность: несколько подов на одной ноде, работающих с одной и той же моделью, независимо скачивают и хранят идентичные веса на одном диске. Второй — межнодовая избыточность: если модель уже загружена на соседней ноде, схема с *emptyDir* не предоставляет никакого механизма для получения этих данных без повторного обращения во внешний репозиторий. При масштабировании системы до N реплик, распределённых по M нодам, возникает до N независимых операций загрузки одного и того же содержимого, что создаёт кратную нагрузку на канал до внешнего репозитория и увеличивает операционные расходы.

1.2.7 Фаза загрузки в *GPU*-память: T_{load}

После того как файлы весов оказались в *emptyDir*, *runtime*-контейнер (*vLLM*, *TGI*, *Triton*) считывает их с диска и загружает в *GPU*-память. Для модели класса *DeepSeek-LLM 7B* эта операция занимает ориентировочно 60–120 секунд; для крупных моделей — несколько минут. Кроме того, некоторые *inference*-движки, в частности *vLLM*, выполняют компиляцию графа *CUDA* и профилирование *KV*-кэша, что дополнительно увеличивает

T_{load} [6].

1.2.8 Итоговый характер задержки

Для типичного контейнера с моделью семейства *DeepSeek* объёмом порядка 15 ГБ суммарная задержка холодного старта на чистом узле складывается ориентировочно следующим образом: $T_{\text{pull}} + T_{\text{unpack}} \approx 5\text{--}8$ мин; $T_{\text{download}} \approx 2\text{--}5$ мин; $T_{\text{load}} \approx 1\text{--}3$ мин. Итоговое время в зависимости от конфигурации сети и дисков может превышать 10–15 минут [6].

При этом задержка масштабируется нелинейно: при переходе от 8B к 70B параметров T_{download} возрастает пропорционально размеру весов, тогда как T_{load} дополнительно зависит от скорости шины памяти *GPU* и конкретного *inference*-движка.

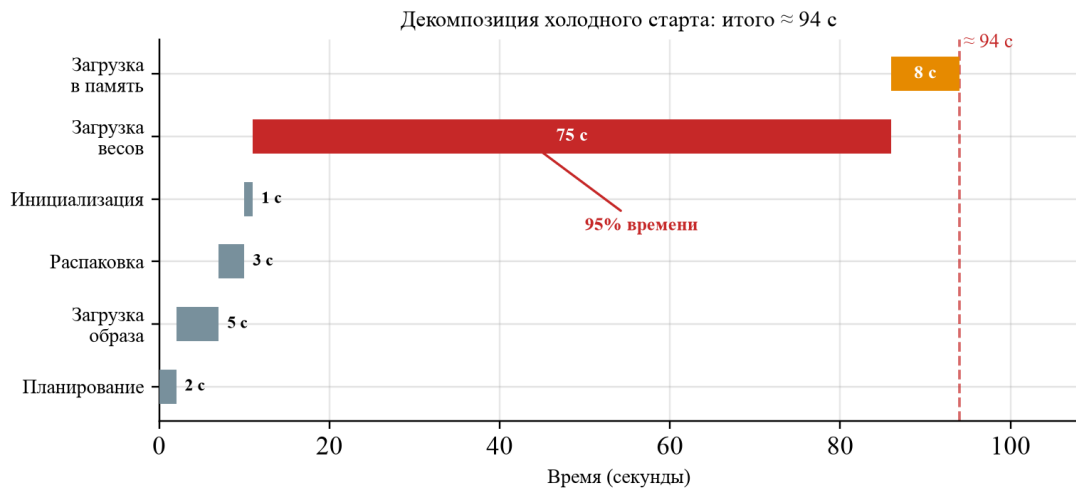


Рисунок 2 – Декомпозиция задержки холодного старта для *DeepSeek-LLM 7B* (15 ГБ, канал 200 МБ/с). Фаза загрузки весов ($T_{\text{download}} \approx 75$ с) составляет около 80% суммарного времени ≈ 94 с.

1.3 Влияние холодного старта на *SLA*, пользовательский опыт и стоимость

1.3.1 *SLA* и метрики задержки

Соглашение об уровне обслуживания (*SLA*) для интерактивных *AI*-сервисов, как правило, регламентирует задержку на перцентилях *P95* и *P99*, а также специфические для генеративных моделей метрики: *TTFT* (*Time to First Token* — время до первого токена) и *TPS* (*Tokens Per Second* —

скорость генерации). Для диалоговых ассистентов требования к $TTFT$ на $P95$ нередко составляют от нескольких сотен миллисекунд до нескольких секунд, а для голосовых систем порог ещё ниже.

Холодный старт, занимающий 10–15 минут, делает соблюдение таких требований при использовании стратегии *scale-to-zero* принципиально невозможным без специальных архитектурных мер. Запросы, поступившие в момент инициализации пода, вынуждены ожидать завершения всей цепочки фаз из формулы (2), что многократно нарушает целевые значения $P95$ и $P99$ *latency*.

1.3.2 Закон Литтла: накопление ожидающих запросов

Для количественной оценки влияния холодного старта применим закон Литтла [16]. Рассмотрим подсистему ожидания инициализации подов как стационарную систему массового обслуживания. Если L_{wait} — среднее число запросов, находящихся в состоянии ожидания завершения холодного старта, λ — интенсивность входящих запросов в период масштабирования, а W_{cold} — среднее время холодного старта, то:

$$L_{\text{wait}} = \lambda \cdot W_{\text{cold}}. \quad (3)$$

где L_{wait} — среднее число ожидающих запросов, λ — интенсивность входящего потока запросов, W_{cold} — среднее время холодного старта.

Из формулы (3) следует фундаментальный вывод: при $\lambda = 10$ запр./с и $W_{\text{cold}} = 600$ с (10 мин) в очереди единовременно накапливается $L_{\text{wait}} = 6000$ ожидающих запросов. Каждый из них занимает соединение и ресурсы *API*-шлюза, что при достижении порогов ограничения по соединениям приводит к отказу в обслуживании.

Закон Литтла носит универсальный характер: он не зависит от распределения времён поступления запросов и от внутренней структуры системы массового обслуживания [16]. Из уравнения (3) прямо следует, что уменьшение W_{cold} в k раз пропорционально уменьшает число ожидающих запросов L_{wait} . Это делает сокращение времени холодного старта первоочередной задачей при проектировании масштабируемого *ML*-инференса.

1.3.3 Вероятность холодного старта при стратегии *scale-to-zero*

В бессерверных системах с автомасштабированием до нуля применяется механизм *keep-alive*: реплика сервиса поддерживается в активном состоянии в течение времени T_{keep} после последнего запроса, после чего освобождается. Если поток входящих запросов к конкретной модели описывается пуассоновским процессом с интенсивностью λ , то вероятность холодного старта для произвольного запроса определяется формулой [16]:

$$P_{\text{cold}} = e^{-\lambda \cdot T_{\text{keep}}}, \quad (4)$$

где P_{cold} — вероятность холодного старта для произвольного запроса, λ — интенсивность входящего потока запросов, T_{keep} — время поддержания реплики в активном состоянии после последнего запроса.

При малой нагрузке $\lambda \rightarrow 0$, и $P_{\text{cold}} \rightarrow 1$: практически каждый запрос инициирует холодный старт. Это характерно для длинного «хвоста» редко используемых специализированных моделей. Обратно, при высокой нагрузке $\lambda \rightarrow \infty$, и $P_{\text{cold}} \rightarrow 0$: реплики постоянно активны и к ним поступают запросы до истечения *keep-alive*.

Из формулы (4) следует, что при фиксированном целевом *SLA* на максимально допустимую вероятность холодного старта $P_{\text{cold}} \leq \varepsilon$ требуемое время поддержания реплики определяется как:

$$T_{\text{keep}} \geq -\frac{\ln \varepsilon}{\lambda}. \quad (5)$$

где T_{keep} — время удержания реплики в активном состоянии, ε — максимально допустимая вероятность холодного старта, λ — интенсивность входящих запросов.

Для модели с $\lambda = 0.01$ запр./с и допустимым $\varepsilon = 0.05$ (5% вероятность холодного старта) получаем $T_{\text{keep}} \geq 299$ с, то есть около 5 минут. За это время простаивающая *GPU* потребляет энергию и занимает слот, за который взимается плата — при типичной стоимости *GPU*-инстанса это превращается в значительную статью операционных расходов.

1.3.4 Дилемма *scale-to-zero* и *over-provisioning*

Из приведённого анализа вытекает фундаментальная дилемма, с которой сталкиваются операторы *ML*-платформ. Стратегия *scale-to-zero* обеспечивает оптимальное использование ресурсов в периоды низкой нагрузки, однако при высоком W_{cold} делает невозможным соблюдение *SLA* на задержку первого токена.

Альтернатива — *over-provisioning*, постоянное поддержание пула «тёплых» реплик — устраняет задержку, но аннулирует экономическое преимущество бессерверной модели. Для модели, которая обрабатывает в среднем один запрос в час, постоянное удержание одной активной реплики с *GPU*-ускорителем создаёт крайне низкую утилизацию оборудования и высокую удельную стоимость обработки запроса.

Оптимальным является третий путь: снижение W_{cold} до уровня, при котором допустимо использовать небольшой *keep-alive* или даже полный *scale-to-zero* без критических нарушений *SLA*. Если холодный старт занимает 30 секунд вместо 10 минут, то допустимое время ожидания первого запроса переходит в категорию приемлемых задержек для многих производственных сценариев.

Таблица 2 иллюстрирует ориентировочный вклад различных компонентов в суммарную задержку холодного старта для модели с 8 и 70 миллиардами параметров при типичных конфигурациях. Следует подчеркнуть, что приведённые значения носят оценочный характер и существенно зависят от пропускной способности сети, производительности дискового ввода-вывода и конкретного *inference*-движка.

Из таблицы 2 видно, что при переходе от 8B к 70B параметров компоненты T_{download} и T_{load} вносят наибольший вклад в прирост суммарной задержки. Именно на эти компоненты должны быть направлены основные архитектурные оптимизации.

1.3.5 Влияние на пользовательский опыт

С точки зрения восприятия пользователем задержка свыше 3 секунд при обращении к веб-сервису существенно снижает вероятность продолжения взаимодействия. Для голосовых ассистентов и интерактивных чат-ботов практически любая задержка первого токена сверх 2–3 секунд

Таблица 2 – Ориентировочный вклад фаз холодного старта (оценочные данные)

Фаза	Типичная длительность		Зависимость от размера модели
	8B модель	70B модель	
T_{schedule} (планирование)	5–30 с	5–30 с	Нет
T_{pull} (загрузка образа)	2–5 мин	2–5 мин	Слабая
T_{unpack} (распаковка)	2–4 мин	2–4 мин	Слабая
T_{download} (загрузка весов)	2–4 мин	15–25 мин	Линейная
T_{load} (в GPU-память)	1–2 мин	5–10 мин	Линейная
T_{start} (итого)	7–15 мин	29–44 мин	Значимая

воспринимается негативно. Холодный старт *ML*-сервиса, длящийся несколько минут, делает продукт практически непригодным к использованию в периоды низкой нагрузки при стратегии *scale-to-zero*. Это вынуждает продуктовые команды выбирать между ухудшением *UX* и значительными дополнительными затратами на инфраструктуру.

1.4 Требования к системе оптимизации холодного старта

На основании проведённого анализа механизма холодного старта, его количественных характеристик и экономических последствий сформулированы следующие требования к системе оптимизации.

1.4.1 Функциональные требования

ФТ-1. Минимизация задержки первого чтения данных модели.

Система должна предоставлять первые байты данных модели в наиболее короткие сроки, не дожидаясь полной загрузки модели из внешнего источника. Для этого необходима поддержка потоковой или ленивой загрузки с возможностью начать инференс до полного завершения скачивания.

ФТ-2. Повторное использование уже загруженных весов.

Если файлы модели или их фрагменты уже присутствуют на дисках узлов кластера, новые реплики должны получать данные из этого локального или соседнего источника без обращения к внешнему репозиторию. Повторная загрузка идентичных байт из интернета при наличии их в кластере недопустима.

ФТ-3. Прозрачность для *ML-runtime*.

Сервисы *vLLM*, *TGI*, *Triton Inference Server* и аналогичные не должны требовать модификации: они должны видеть обычный файловый путь в файловой системе и работать со стандартными *POSIX*-операциями чтения. Переход на новую систему не должен нарушать совместимость с существующими *runtime*-окружениями.

ФТ-4. Поддержка больших файлов весов.

Система должна корректно работать с отдельными файлами весов размером от сотен мегабайт до нескольких сотен гигабайт, включая формат *safetensors* и бинарные форматы *PyTorch*. Обязательна поддержка произвольного доступа по смещению (*range reads*), характерного для ленивой загрузки.

ФТ-5. Горизонтальное масштабирование.

Производительность системы не должна деградировать при добавлении новых *worker*-нод и увеличении числа одновременно запущенных реплик инференса. Добавление узлов должно линейно увеличивать суммарный объём локального кэша, доступного всему кластеру.

ФТ-6. Локальность данных и приоритет источников.

При обслуживании запроса на данные должен соблюдаться следующий приоритет источников: первый — локальный диск той же ноды; второй — диски соседних нод кластера по внутренней сети; третий — внешний репозиторий модели. Такая иерархия минимизирует сетевой трафик и задержки.

1.4.2 Нефункциональные требования

НТ-1. Отказоустойчивость при падении узла.

Выход одной или нескольких *worker*-нод из строя не должен делать модель полностью недоступной, при условии что её данные присутствуют на других нодах или доступны через *fallback* на внешний репозиторий. Система должна корректно обнаруживать недоступность узла и переключаться на альтернативный источник.

НТ-2. Устранение единой точки отказа (*SPOF*).

Архитектура не должна содержать единственного центрального компонента, недоступность которого делает систему нефункциональной. Это требование непосредственно исключает использование одного *NFS*-сервера в качестве

единственного хранилища весов.

НТ-3. Контролируемые накладные расходы.

Объём метаданных и управляющего трафика (регистрация чанков, *heartbeat*-сигналы, запросы к реестру) не должен создавать значимой нагрузки на сеть кластера или перегружать реестр метаданных. Накладные расходы на один запрос данных должны составлять пренебрежимо малую долю по сравнению с полезной нагрузкой.

НТ-4. Совместимость с *Kubernetes*.

Все компоненты системы должны развёртываться как стандартные ресурсы *Kubernetes* — *DaemonSet* для агентов на каждой ноде, *Deployment* для управляющих компонентов, стандартные *PVC/PV* для постоянного хранения — без модификации ядра операционной системы, *kubelet* или компонентов *control plane*.

Совокупность сформулированных требований определяет целевые свойства архитектуры, которая должна обеспечивать приоритет локальных данных, прозрачность для *runtime*-сред и отказоустойчивость без центральной точки отказа.

1.5 Выводы по разделу

Проведённый анализ позволяет сформулировать следующие выводы.

Рост размеров *ML*-моделей за период с 2012 по 2024 год на три-четыре порядка превратил холодный старт из второстепенной инфраструктурной задержки в архитектурный приоритет. Компоненты T_{download} и T_{load} из формулы (2) доминируют в суммарном времени инициализации и напрямую определяются наличием локально кэшированных весов.

Применение закона Литтла (3) показывает, что линейное снижение W_{cold} пропорционально снижает число ожидающих запросов L_{wait} , что критически важно при *burst*-масштабировании. Формула (4) демонстрирует, что для редко запрашиваемых моделей вероятность холодного старта стремится к единице вне зависимости от политики *keep-alive*, что оправдывает применение архитектурных механизмов ускорения.

Классическая схема с *emptyDir* не переиспользует уже загруженные веса между репликами и нодами, что при масштабировании приводит к N -кратному увеличению нагрузки на внешний репозиторий.

Сформулированные функциональные и нефункциональные требования определяют необходимые свойства системы: локальность данных, прозрачность для *ML-runtime*, горизонтальная масштабируемость, отказоустойчивость без *SPOF*.

Центральный вывод раздела состоит в том, что ускорять необходимо не вычислительную фазу инференса, а подготовительную: доставку, кэширование и представление весов модели приложению. Это требует специализированной архитектуры хранения данных, ориентированной на локальность, *P2P*-передачу между узлами и прозрачный файловый интерфейс для *runtime*-окружений.

2 АНАЛИЗ СУЩЕСТВУЮЩИХ РЕШЕНИЙ ДЛЯ ХРАНЕНИЯ И КЭШИРОВАНИЯ МОДЕЛЕЙ

В данном разделе исследуются архитектуры и свойства наиболее распространённых систем распределённого хранения и кэширования, рассматриваемых в качестве кандидатных решений для задачи ускорения холодного старта *ML*-инференса в *Kubernetes*. Для каждого решения проводится анализ применимости к сценарию бессерверного инференса: определяется, в какой мере система устраняет повторные загрузки весов, обеспечивает локальность данных на *worker*-нодах и сохраняет прозрачный файловый интерфейс для *runtime*-контейнера. На основе анализа формулируется обоснование разработки специализированного решения.

2.1 Критерии сравнения решений

Чтобы сравнение систем было обоснованным и воспроизводимым, необходимо зафиксировать критерии, непосредственно вытекающие из требований, сформулированных в разделе 1. Каждый критерий отражает конкретный аспект задачи ускорения холодного старта.

Задержка при холодном старте определяет, насколько система способна уменьшить время до первого доступного байта весов модели. Ключевой характеристикой здесь является поддержка ленивой загрузки (*lazy loading*) или чтения по требованию: *runtime*-контейнер должен иметь возможность стартовать и обращаться к отдельным фрагментам файла ещё до его полной передачи.

Поддержка *Kubernetes* означает наличие официального механизма развёртывания в контейнерной среде: *CSI*-драйвера, *Helm*-чарта или *Kubernetes Operator*. Отсутствие этих инструментов существенно увеличивает операционную сложность интеграции.

Локальный кэш на нодe — возможность хранить горячие данные непосредственно на дисках *worker*-нод вблизи *compute*-ресурсов, что обеспечивает минимальную *latency* при повторном чтении. Решения, хранящие данные только в центральном хранилище, не позволяют избавиться от сетевого хопа при каждом обращении.

P2P-доставка отражает способность системы передавать данные между *worker*-нодами без обращения к центральному источнику, тем самым

распределяя нагрузку и ускоряя одновременное масштабирование большого числа реплик инференса.

FUSE/POSIX-интерфейс оценивает, предоставляет ли система *runtime*-контейнеру стандартный файловый путь, не требующий изменений в *ML*-фреймворке. Наличие *POSIX*-семантики позволяет *vLLM*, *Triton* и аналогичным инструментам работать без модификации кода.

Отказоустойчивость характеризует поведение системы при выходе из строя отдельных узлов: существуют ли механизм репликации данных и автоматический *fallback* на альтернативный источник.

Наличие единой точки отказа (SPOF) указывает, имеется ли в архитектуре компонент, отказ которого делает недоступными все данные. Это критически важно для сред с требованиями высокой доступности.

Зависимость от внешнего объектного хранилища определяет, является ли *object storage* обязательным звеном в пути чтения. Обязательное обращение к *S3/GCS* при каждом промахе кэша существенно увеличивает *latency* и создаёт зависимость от внешней инфраструктуры.

Прозрачность для runtime оценивает, насколько решение скрывает распределённую природу данных от приложения. Идеальная прозрачность означает, что *ML*-контейнер «видит» обычную директорию с файлами модели.

Сложность эксплуатации суммирует требования к компетенциям команды и инфраструктурным зависимостям. Решения с высокой сложностью неприемлемы для небольших платформ без выделенного *SRE*-отдела.

Применимость к *serverless cold start* является интегральным критерием: насколько хорошо решение адаптировано к сценарию эфемерных подов, работающих с крупными бинарными файлами весов, при жёстких ограничениях на время инициализации.

Перечисленные критерии используются в сравнительной таблице 3 и в итоговом обосновании разработки специализированного решения.

2.2 Alluxio

Alluxio (ранее *Tachyon*) — виртуальная распределённая файловая система (*Virtual Distributed File System*, *VD FS*), выступающая слоем оркестрации данных между вычислительными фреймворками и системами постоянного хранения (*Amazon S3*, *HDFS*, *NFS* и другими) [17]. Система разработана в Калифорнийском университете в Беркли и нашла применение в

крупномасштабных аналитических и *ML*-рабочих нагрузках.

2.2.1 Архитектура

Alluxio строится по модели *Master/Worker/Client*. Ведущий узел (*Leading Master*) управляет глобальными метаданными: деревом *inode*, распределением блоков и ёмкостью рабочих узлов. Все транзакции файловой системы записываются в распределённый журнал для обеспечения восстановления состояния при сбое. Для обеспечения высокой доступности (*HA*) рядом с ведущим мастером работают несколько резервных мастеров, непрерывно синхронизирующихся через журнал и готовых принять управление при отказе основного.

Worker-узлы управляют локальными ресурсами хранения (*RAM*, *SSD*, *HDD*), выделенными под кэш *Alluxio*. Данные хранятся в виде блоков. При заполнении ёмкости применяются политики вытеснения (*LRU* и аналоги). *Worker*-узлы также выполняют операции с базовым хранилищем (*Under File System*, *UFS*): при первом обращении к данным они кэшируют блоки локально, снижая нагрузку на *UFS* при последующих чтениях.

Client предоставляет приложению шлюз к кластеру *Alluxio*. Он взаимодействует с *Master* для операций с метаданными и с *Worker* для чтения и записи данных. Поддерживаются нативный *Java API*, *REST*, *HDFS API* и *S3-совместимый API*. Важно отметить, что клиент *Alluxio* никогда не обращается напрямую к базовому хранилищу: данные всегда передаются через *Worker*-узлы [17].

Интеграция с *Kubernetes* реализована через *Helm*-чарты и *Alluxio Operator*, который автоматизирует развёртывание, настройку и управление жизненным циклом кластера *Alluxio* поверх *Kubernetes* [18]. *Alluxio* поддерживает монтирование через *FUSE* (*Alluxio POSIX API*), что обеспечивает прозрачный доступ для приложений через стандартный файловый интерфейс.

В *Enterprise*-версии архитектура мастера эволюционировала в сторону децентрализации: владение метаданными распределяется между *Worker*-узлами через согласованное хеширование, что устраняет централизованный мастер как узкое место и позволяет обходить его при чтении данных.

2.2.2 Преимущества

Alluxio обеспечивает высокую производительность за счёт приоритетного кэширования в оперативной памяти (*memory-first tiered architecture*). Унифицированное пространство имён (*Unified Namespace*) позволяет монтировать разнородные хранилища в единую логическую файловую систему, что актуально для платформ с несколькими объектными хранилищами. Поддержка локальности данных выгодна для *ML*-нагрузок: когда модельные данные кэшируются на ноде, где запущен инференс, задержка чтения снижается до уровня локального диска.

2.2.3 Недостатки

Первый существенный недостаток — *сложность эксплуатации*. Даже при наличии *Kubernetes Operator* настройка *HA*-конфигурации мастеров, управление *eviction*-политиками, диагностика утечек памяти в *Worker*-узлах и оптимизация распределения блоков требуют глубоких знаний архитектуры системы. Операционная нагрузка несопоставима с задачей кэширования весов модели.

Второй недостаток — *высокое потребление памяти*. *Alluxio* является *memory-centric* системой: для достижения декларируемой производительности *Worker*-узлы должны располагать значительным объёмом оперативной памяти. При кэшировании нескольких больших моделей (десятки гигабайт каждая) потребности в памяти конкурируют с самим *GPU*-инференсом.

Третий недостаток — *модель лицензирования*. Ключевые функции для производственных *ML*-сред: децентрализованная архитектура, расширенные политики кэширования и продвинутое *AI/ML*-оптимизации — доступны только в *Enterprise Edition* [19]. Стоимость не публикуется и определяется индивидуально, что создаёт финансовую неопределённость.

Четвёртый недостаток — *несовместимость с serverless-подами*. Клиент *Alluxio* предполагает наличие состояния и устойчивого соединения с кластером. В эфемерных подах *Kubernetes* каждая инициализация клиента добавляет задержку к холодному старту, а данные, кэшированные по запросу короткоживущего пода, могут быть вытеснены прежде, чем повторно использованы.

2.2.4 Вывод по *Alluxio*

Alluxio технически близок к задаче: он обеспечивает кэширование данных с локальностью и поддерживает *FUSE*-монтирование. Однако он представляет собой тяжёлый *data orchestration layer* общего назначения, предназначенный для аналитики и распределённого *ML*-обучения. Для узкой задачи *chunk*-кэширования весов модели в *serverless*-инференсе это архитектурно и операционно избыточное решение.

2.3 *JuiceFS*

JuiceFS — высокопроизводительная распределённая файловая система с *POSIX*-совместимым интерфейсом, ориентированная на облачные среды [20]. *Community Edition* распространяется под лицензией *Apache 2.0*. Система нашла применение в *ML*-платформах благодаря нативной интеграции с *Kubernetes* и возможности использовать уже существующие объектные хранилища как *backend*.

2.3.1 Архитектура

Фундаментальной особенностью *JuiceFS* является *строгое разделение данных и метаданных*. Архитектура включает три компонента.

Metadata Engine хранит всю информацию о файловой системе: имена файлов, права доступа, временные метки, структуру директорий и, что критически важно, отображение файлов на физические блоки данных. *Community Edition* поддерживает несколько бэкендов: *Redis*, *TiKV*, *PostgreSQL*, *MySQL/MariaDB* [21]. Выбор *Redis* обеспечивает минимальные задержки (порядка миллисекунд) за счёт хранения в оперативной памяти, *TiKV* обеспечивает горизонтальную масштабируемость для платформ с миллиардами объектов.

Object Storage хранит содержимое файлов в виде блоков. При записи файл логически делится на чанки по 64 МБ, каждый из которых при сбросе (*flush*) разбивается на блоки по 4 МБ, параллельно загружаемые в объектное хранилище [21]. *JuiceFS* поддерживает *Amazon S3*, *MinIO*, *Google Cloud Storage* и другие *S3*-совместимые системы. В самом объектном хранилище исходные файлы не представлены: там хранятся пронумерованные блоки, а их связь с файловой иерархией поддерживается через *Metadata Engine*.

Клиент *JuiceFS* реализует файловый интерфейс (*FUSE*, *CSI*-драйвер, *S3 Gateway*) и взаимодействует одновременно с *Metadata Engine* (для получения структуры файла и местоположения блоков) и с объектным хранилищем (для чтения и записи данных).

В *Kubernetes JuiceFS* развёртывается через *CSI*-драйвер, который реализует архитектуру с выделенным *Mount Pod* на каждом узле. Этот под исполняет клиент *FUSE* и обслуживает все *application pods* на узле, использующие один и тот же *PVC*. Такое разделение позволяет перезапускать *CSI*-компоненты без прерывания доступа к данным для уже запущенных приложений [22].

Кэширование на локальных дисках является ключевым механизмом ускорения: при повторном чтении одних и тех же блоков *JuiceFS* обслуживает запросы с локального *NVMe*-диска вместо обращения к объектному хранилищу. По данным разработчиков, при последовательном чтении крупных файлов с включённым локальным кэшем пропускная способность может вырасти с 4,7 ГБ/с до 13,2 ГБ/с, а задержка случайного чтения — снизиться с 29 мс до 0,3 мс [22].

2.3.2 Ограничения для *serverless cold start*

Зависимость от объектного хранилища. *JuiceFS* не имеет собственного слоя хранения данных и обязательно требует внешнего *S3*-совместимого хранилища. При промахе кэша (*cache miss*) клиент обращается к объектному хранилищу, задержки которого на порядок превышают задержки локального диска.

Metadata Engine как потенциальная точка отказа. При использовании *Redis* в качестве бэкенда метаданных и отсутствии *HA*-конфигурации (*Redis Sentinel* или *Redis Cluster*) отказ экземпляра *Redis* делает недоступной всю файловую систему, даже если объектное хранилище функционирует нормально. Настройка кластерного *Redis* добавляет операционную сложность.

Проблема прогрева кэша в эфемерных средах. В *serverless*-сценарии поды часто запускаются на «чистых» узлах, где локальный кэш пуст. В отличие от долгоживущих приложений, эфемерный под не успевает накопить полезный кэш до своего завершения. При масштабировании на новые узлы все читаемые блоки будут приходить из объектного хранилища — именно в тот момент, когда критична *latency* первого байта.

Накладные расходы FUSE. Реализация файловой системы в пространстве пользователя через *FUSE* вносит дополнительные переключения контекста при каждом файловом вызове. При большом числе мелких операций чтения это снижает эффективную пропускную способность одного потока.

Шторм монтирований при scale-out. При одновременном масштабировании на большое число новых узлов создание *Mount Pod*’ов для каждого узла увеличивает нагрузку на *API*-сервер *Kubernetes* и может задерживать инициализацию новых подов.

2.3.3 Вывод по *JuiceFS*

JuiceFS представляет ценный архитектурный ориентир: разделение данных и метаданных, использование *Redis* как быстрого *metadata backend*, *FUSE*-интерфейс — все эти идеи применены в разрабатываемой системе. Однако *JuiceFS* является файловой системой общего назначения с обязательной зависимостью от объектного хранилища, тогда как для решения задачи требуется специализированный *cache/fetch* слой с *P2P*-передачей чанков между нодами кластера.

2.4 *Dragonfly*, *CubeFS* и *Rook/Ceph*

2.4.1 *Dragonfly*

Dragonfly — *P2P*-система распределения файлов и контейнерных образов, разработанная *Alibaba Cloud* и переданная в *Cloud Native Computing Foundation (CNCF)* [23]. Система ориентирована на сценарии эффективного распространения крупных файлов на большое число узлов одновременно, что делает её концептуально близкой к задаче *distributing* весов *ML*-моделей.

Архитектура.

Dragonfly сочетает элементы *CDN* и *P2P*. *Scheduler* является центральным компонентом: он отслеживает доступность узлов-пиров, их загрузку и местоположение данных, определяя оптимальные источники для каждого запроса. *Seed Peer* — специализированный пир, кэширующий данные и выступающий первичным источником для других пиров, особенно когда исходный сервер недоступен или перегружен. *Peer (dfdaemon)* работает на каждом узле *Kubernetes* как *DaemonSet*, перехватывает запросы на загрузку

образов и файлов, перенаправляя их в *P2P*-сеть. При загрузке каждый пир одновременно становится источником для других, распределяя нагрузку.

Применимость к *ML*-инференсу.

Dragonfly хорошо зарекомендовал себя для параллельной дистрибуции образов контейнеров на сотни нод. Та же модель применима к весам *ML*-моделей: при первой загрузке модели на один из узлов она кэшируется и становится доступной для скачивания другими узлами через *P2P*-сеть вместо обращения к центральному репозиторию. Это снижает нагрузку на *Hugging Face* или внутренний *model registry* и ускоряет масштабирование.

Ограничения.

Dragonfly не предоставляет стандартного *FUSE/POSIX*-интерфейса: система занимается доставкой файлов, но не монтированием файловой системы. Для предоставления *ML-runtime* доступа к весам модели через обычный файловый путь потребуется дополнительный уровень. *Scheduler* и *Seed Peer* требуют отдельного развёртывания и поддержки, увеличивая операционную нагрузку. Наличие централизованного *Scheduler* вводит компонент, отказ которого нарушает координацию *P2P*-сети, хотя фактическая доставка данных продолжается между пирами. Тем не менее *Dragonfly* является ближайшим по духу предшественником *P2P*-подхода, разработанного в данной работе: идея обмена фрагментами данных между узлами кластера вместо многократного скачивания из центрального источника лежит в основе обоих решений.

2.4.2 *CubeFS*

CubeFS — облачно-ориентированная распределённая файловая система уровня *CNCF Graduated* [24], разработанная *JD.com*. Система предоставляет унифицированное хранилище с поддержкой нескольких протоколов доступа и предназначена для высоконагруженных производственных сред.

Архитектура.

CubeFS включает несколько подсистем. *Master*-узлы управляют ресурсами кластера: отслеживают состояние узлов метаданных и узлов данных, управляют томами и обеспечивают согласованность конфигурации с использованием *Raft* и *RocksDB*. Узлы метаданных управляют шардами метаданных (*metadata partition*). Каждый шард содержит две *B-Tree* структуры в памяти: *inode B-Tree* и *dentry B-Tree*. Минимальная конфигурация требует

трёх экземпляров. *Data Nodes* управляют шардами данных (*Data Partition*) и образуют репликационные группы. *CubeFS* поддерживает как репликацию данных, так и *Erasure Coding* (через подсистему *Blobstore*) для более эффективного использования дискового пространства. Объектный шлюз (*Object Subsystem*) предоставляет *S3*-совместимый *API*.

Поддерживаемые протоколы.

CubeFS поддерживает *POSIX*, *S3* и *HDFS*-интерфейсы, а также *CSI*-драйвер для *Kubernetes*. Мультипротокольная поддержка делает систему гибкой для смешанных рабочих нагрузок, где различные компоненты *ML*-платформы используют разные интерфейсы доступа к данным.

Применимость к *ML*-инференсу.

CubeFS обеспечивает высокую пропускную способность для крупных файлов за счёт параллелизма *Data Nodes*. *POSIX*-совместимость позволяет использовать систему для хранения и одновременного доступа к файлам весов модели из нескольких реплик инференса. Репликация данных обеспечивает отказоустойчивость при выходе из строя отдельных узлов.

Ограничения.

Сложность развёртывания *CubeFS* сопоставима с *Ceph*: кластер требует управления несколькими типами узлов (*Master*, узел метаданных, *DataNode*), настройки репликации, мониторинга состояния и обеспечения кворума узлов метаданных. Для небольших и средних *ML*-платформ операционные требования несоразмерны задаче. Кроме того, *CubeFS* проектировалась как система хранения общего назначения, а не как специализированный кэш для бессерверного инференса: в ней отсутствуют *P2P*-доставка чанков и встроенный механизм *lazy loading* для *ML*-весов.

2.4.3 *Rook/Ceph*

Rook — *cloud-native* оркестратор для *Kubernetes*, автоматизирующий развёртывание и управление распределёнными системами хранения [25]. В наиболее распространённой конфигурации *Rook* используется для оркестрации *Ceph* — одной из наиболее функционально богатых *open-source*-систем хранения данных.

Архитектура *Ceph*.

Ceph предоставляет блочное (*RADOS Block Device, RBD*), файловое (*CephFS*)

и объектное (*RADOS Gateway*, *RGW*) хранилище в единой системе [26]. Архитектура строится на нескольких типах демонов. *Monitor* (*MON*) — поддерживает главную копию *cluster map* и обеспечивает консенсус между узлами через алгоритм *Paxos*; для *HA* требуется не менее трёх мониторов. *Manager* (*MGR*) — отвечает за мониторинг, управление и расширения (плагины). *OSD Daemon* — хранит данные на физических дисках, выполняет репликацию и проверку состояния. *Metadata Server* (*MDS*) — управляет метаданными *CephFS* и обеспечивает *POSIX*-совместимый доступ к файловой системе.

Децентрализованное размещение данных реализовано через алгоритм *CRUSH* (*Controlled Replication Under Scalable Hashing*), который позволяет клиентам самостоятельно вычислять местоположение данных без обращения к центральному индексу. Это устраняет масштабируемые ограничения централизованного метаданных-сервера.

***CephFS* в *Kubernetes*.**

Rook автоматизирует создание и управление пулами *CephFS* и предоставляет *CSI*-драйвер, поддерживающий режим доступа *ReadWriteMany* (*RWX*). Это позволяет нескольким подам одновременно монтировать один том для доступа к общим данным — актуальный сценарий для *ML*-инференса, где несколько реплик читают одну модель.

Ограничения.

Развёртывание *Ceph* требует специализированных знаний: настройка пулов *CRUSH*, управление *OSD*-журналами, диагностика медленных *OSD* и оптимизация производительности под конкретную нагрузку создают значительный барьер для команд без опыта работы с *Ceph*. Минимальный *production*-кластер требует нескольких выделенных узлов хранения с отдельными дисками.

Ceph является универсальной *production*-системой хранения, а не специализированным решением для кэширования весов *ML*-моделей. Его возможности существенно превосходят необходимые для данной задачи, а операционные требования несоразмерны её масштабу. Для сценария бессерверного инференса, где ключевая метрика — *latency* первого байта при холодном старте, — *Rook/Ceph* обеспечивает избыточную функциональность при высокой стоимости эксплуатации.

2.5 Сравнительная таблица

Для удобства сопоставления результаты анализа сведены в таблицу 3.

Таблица 3 – Сравнение решений для хранения и кэширования *ML*-моделей

Критерий	<i>Alluxio</i>	<i>JuiceFS</i>	<i>Dragonfly</i>	<i>CubeFS</i>	<i>Rook/Ceph</i>
Лок. кэш на ноде	Да (<i>RAM/SSD</i>)	Да (диск)	Да (диск)	Да (диск)	Да (<i>OSD</i>)
<i>P2P</i> -доставка	Нет	Нет	Да	Нет	Нет
<i>FUSE/POSIX</i>	Частично	Да	Нет	Да	Да (<i>CephFS</i>)
<i>K8s</i> -интеграция	<i>Helm/Operator</i>	<i>CSI</i>	<i>DaemonSet</i>	<i>CSI</i>	<i>Rook Operator</i>
<i>SPOF</i> в архитектуре	<i>Master (OSS)</i>	<i>Metadata Engine</i>	<i>Scheduler</i>	<i>Master</i>	<i>MON</i> (кворум)
Объектное хранилище	Опцио-нально	Обяза-тельно	Не нужно	Не нужно	Встроено (<i>RGW</i>)
Сложность эксплуатации	Высокая	Средняя	Средняя	Высокая	Очень высокая
Применимость к <i>serverless</i>	Средняя	Низкая	Высокая	Средняя	Низкая
Лицензия	<i>Community/Ent.</i>	<i>Apache 2.0</i>	<i>Apache 2.0</i>	<i>Apache 2.0</i>	<i>LGPL/ Apache</i>

Анализ таблицы 3 позволяет сформулировать следующие наблюдения. Ни одно из рассмотренных решений не удовлетворяет всем требованиям одновременно. *Dragonfly* наиболее близок по концепции *P2P*-доставки и показывает высокую применимость к *serverless*-сценариям, однако не предоставляет *FUSE*-интерфейса и требует отдельной инфраструктуры с собственным *Scheduler*. *JuiceFS* имеет *POSIX*-интерфейс и хорошую архитектуру метаданных, но обязательная зависимость от объектного хранилища и отсутствие *P2P* между нодами делают его неоптимальным для данной задачи. *Alluxio* и *Rook/Ceph* обладают богатой функциональностью, но их операционная сложность несоразмерна специализированной задаче кэширования весов. *CubeFS* занимает промежуточное положение: поддерживает *POSIX* и *K8s*-интеграцию, но лишена *P2P*-доставки и сложна в эксплуатации.

2.6 Обоснование разработки специализированного решения

Проведённый анализ демонстрирует системное несоответствие между возможностями существующих решений и требованиями задачи. Каждая из рассмотренных систем спроектирована как универсальная инфраструктура хранения данных общего назначения. Задача же данной работы принципиально уже: обеспечить быстрый локальный или межнодовый доступ к фрагментам (чанкам) файлов весов *ML*-моделей для контейнеров бессерверного инференса в *Kubernetes*. Ниже приведены конкретные аргументы в пользу разработки специализированного решения.

Специализация вместо универсальности.

Все рассмотренные системы обеспечивают функциональность, существенно выходящую за рамки необходимой: поддержку множественных протоколов доступа, управление большими кластерами хранения, операции записи для приложений. Для задачи *serverless*-инференса нужен узкий *cache/fetch* слой, ориентированный на оптимизацию чтения крупных бинарных файлов по фрагментам. Дополнительная функциональность увеличивает поверхность отказов и затраты на эксплуатацию без соответствующего увеличения полезности.

Приоритет локальности данных.

В идеальном случае чанк весов модели должен читаться с локального диска той же ноды, на которой запущен *inference*-контейнер. Это исключает сетевой хоп и обеспечивает минимальную задержку. Ни одна из рассмотренных систем не реализует явного приоритета «локальный диск -> соседняя нода -> внешний источник» в виде встроенного трёхуровневого *read path* для *serverless*-сценария.

***P2P* в сочетании с *FUSE*.**

Dragonfly реализует *P2P*-доставку, но не предоставляет *POSIX*-интерфейса. *JuiceFS* предоставляет *FUSE*, но не поддерживает *P2P* между нодами. Требуемая комбинация — *P2P*-доставка чанков между узлами кластера с представлением результата через стандартный файловый путь — отсутствует ни в одном из существующих решений в готовом виде.

Простая модель метаданных с *TTL*.

Redis в роли реестра метаданных чанков обеспечивает атомарные операции, низкую задержку *lookup* ($O(1)$ по ключу) и встроенный механизм *TTL* для

автоматического удаления устаревших записей о недоступных нодах. Это принципиально проще, чем поддержка полноценного *Metadata Engine* с журналами, репликой *Raft* и сложной схемой партиционирования, как в *JuiceFS* или *CubeFS*. Для кластеров малого и среднего размера (до тысяч нод) одиночный или сентинельный *Redis* является достаточным и значительно более управляемым решением.

***Fallback* без жёсткой зависимости.**

Специализированная система допускает ситуацию, когда нужный чанк отсутствует в кластере: в этом случае он скачивается из внешнего источника (*Hugging Face* или другого *registry*) и сохраняется в локальном кэше. Такой *fallback* требуется лишь при первом обращении к модели; все последующие поды получают чанк из локального или соседнего кэша. Это свойство устраняет принципиальную зависимость от объектного хранилища как обязательного звена в пути чтения, в отличие от *JuiceFS*.

Отказоустойчивость без центрального файлового сервера.

NFS как общее хранилище, рассмотренное в разделе 3, решает задачу устранения повторных загрузок из интернета, но создаёт центральное узкое место: все чтения крупных файлов проходят через один файловый сервер. Специализированная система использует локальные диски всех *worker*-нод как распределённый кэш: при отказе одной ноды чанки, хранившиеся на ней, могут быть получены от других нод или из внешнего источника. *Redis* с *TTL* автоматически удаляет устаревшие записи об отказавшем агенте. Такая модель принципиально более устойчива к частичным отказам, чем централизованный *NFS*.

Дополнительным аргументом является экономическая доступность: все описанные компоненты (*Redis*, *FUSE*-библиотека, *Go*-реализация агента) распространяются под открытыми лицензиями. *Alluxio Enterprise* с ключевыми функциями кэширования и *Rook/Ceph* с его операционными требованиями несут значительные финансовые и инфраструктурные затраты, неприемлемые для небольших *ML*-платформ.

2.7 Выводы по разделу

Рассмотренные решения — *Alluxio*, *JuiceFS*, *Dragonfly*, *CubeFS* и *Rook/Ceph* — предоставляют ценные архитектурные идеи, нашедшие

отражение в разрабатываемой системе: разделение данных и метаданных (*JuiceFS*), *P2P*-дистрибуция между узлами кластера (*Dragonfly*), предоставление *POSIX*-совместимого интерфейса через *FUSE* (*JuiceFS*, *CubeFS*, *CephFS*).

Вместе с тем ни одно из существующих решений не обеспечивает требуемую комбинацию свойств: трёхуровневый *read path* с приоритетом локального кэша, *P2P*-доставку чанков между нодами и прозрачный *FUSE*-интерфейс для *ML-runtime* без обязательной зависимости от внешнего объектного хранилища и тяжёлой операционной инфраструктуры.

Это обосновывает разработку специализированной системы: *lightweight chunk*-кэша на локальных дисках *worker*-нода, сопровождаемого *Redis*-реестром метаданных чанков и *FUSE*-клиентом, обеспечивающим *runtime*-контейнеру стандартный файловый интерфейс. Описание промежуточного этапа — подхода на основе *NFS* и координации загрузок — приведено в разделе 3. Основная архитектура разрабатываемой системы описана в разделе 4.

3 ПРОМЕЖУТОЧНОЕ РЕШЕНИЕ НА БАЗЕ *NFS* И КООРДИНАЦИИ ЗАГРУЗОК

В данном разделе описывается промежуточный этап оптимизации системы бессерверного инференса: переход от изолированной загрузки весов модели в том *emptyDir* к использованию сетевой файловой системы *NFS* в качестве общего хранилища. Раскрываются недостатки исходной схемы, механизм интеграции *NFS* в *Kubernetes* через ресурсы *PersistentVolume* и *PersistentVolumeClaim*, проблемы синхронизации параллельных загрузок, особенности файловых блокировок поверх сетевых файловых систем, применение механизма *Leader Election* через *Kubernetes Lease API*, а также системные ограничения, обусловившие необходимость разработки распределённой *P2P/FUSE*-архитектуры.

3.1 Исходная схема: *storage-initializer* и *emptyDir*

3.1.1 Паттерн *init*-контейнера и жизненный цикл тома *emptyDir*

Исходная схема развёртывания модели в *Kubernetes* использует паттерн *init*-контейнера. Под (*pod*) содержит два контейнера: *storage-initializer*, запускаемый первым в режиме *init*, и *runtime*-контейнер, являющийся основным. *Init*-контейнер монтирует том типа *emptyDir*, запускает процесс загрузки весов модели из внешнего репозитория (*HuggingFace*, *ModelScope*, объектное *S3*-совместимое хранилище) и записывает файлы в смонтированный том. После успешного завершения *init*-контейнера *Kubernetes* запускает *runtime*-контейнер (*vLLM*, *TGI*, *Triton*), который читает веса из того же тома *emptyDir* и загружает их в *GPU*-память для обслуживания запросов.

Том типа *emptyDir* создаётся *Kubernetes* в момент планирования пода на узел и физически размещается на локальном диске этого узла, либо, при явной конфигурации, в памяти. Ключевая характеристика данного типа тома состоит в том, что его жизненный цикл жёстко привязан к жизненному циклу пода: при удалении или перепланировании пода том уничтожается вместе со всем содержимым. Том *emptyDir* не является персистентным и принципиально не разделяется между подами: каждый под получает собственную изолированную директорию.

На рисунке 3 показано, что путь данных полностью замкнут внутри

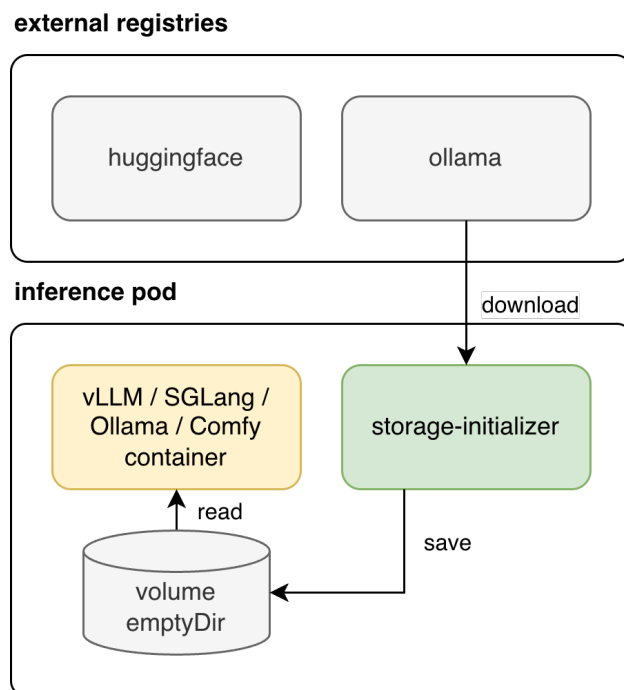


Рисунок 3 – Исходная схема развёртывания модели с *init*-контейнером *storage-initializer* и томом *emptyDir*

одного *pod*: *init*-контейнер загружает веса во временный локальный том, после чего *runtime*-контейнер читает только эту копию. Такое решение упрощает запуск отдельной реплики, но не создаёт механизма межподового переиспользования уже загруженной модели.

3.1.2 Фундаментальный недостаток: повторные загрузки

Отсутствие разделяемого хранилища порождает фундаментальный недостаток данной схемы. При создании новой реплики, даже на той же узловой машине, каждый под выполняет полную загрузку файлов модели независимо от всех ранее запущенных подов. Если уже работающий под уже завершил загрузку той же модели, новый под этого не знает и не может воспользоваться имеющейся копией данных.

При масштабировании до N реплик внешний репозиторий получает N независимых последовательных или параллельных запросов на один и тот же набор файлов. Для современных моделей класса *LLM* с объёмом весов от 7 до 70 ГБ это означает:

- суммарный исходящий трафик на уровне десятков и сотен гигабайт при каждом событии масштабирования;

- время *cold start* каждой реплики полностью определяется скоростью загрузки из внешнего источника, а не скоростью чтения с диска;
- нагрузка на внешний канал связи возрастает пропорционально числу реплик, что может приводить к деградации пропускной способности и к превышению *rate*-лимитов репозитория.

Таким образом, схема *storage-initializer* + *emptyDir* не решает задачу *cold start* при масштабировании: повторные загрузки по-прежнему присутствуют, хотя и перемещены с уровня повторных запусков того же пода на уровень каждой новой реплики.

3.2 Переход к *NFS* как общему хранилищу

3.2.1 *NFS* и режим *ReadWriteMany*

Network File System (NFS) — зрелый протокол сетевого доступа к файлам, разработанный компанией *Sun Microsystems* и стандартизированный в версиях *NFSv3 (RFC 1813)* и *NFSv4 (RFC 7530)*. В контексте *Kubernetes NFS* привлекателен прежде всего тем, что поддерживает режим доступа *ReadWriteMany (RWX)*, при котором несколько подов на разных узлах кластера могут одновременно монтировать один и тот же том как для чтения, так и для записи [27]. Большинство блочных хранилищ поддерживают лишь режим *ReadWriteOnce (RWO)*, допускающий монтирование только одним подом, что исключает их применение для общего кэша весов моделей.

3.2.2 Интеграция в *Kubernetes: PV* и *PVC*

Схема использования *NFS* в *Kubernetes* предполагает создание двух ресурсов: *PersistentVolume (PV)*, описывающего физическое хранилище, и *PersistentVolumeClaim (PVC)*, представляющего запрос пода на использование этого хранилища. Ниже приведены примеры манифестов.

Листинг 1: *PersistentVolume* на базе *NFS*

```

1 apiVersion: v1
2 kind: PersistentVolume
3 metadata:
4   name: models - nfs - pv
5 spec:
6   capacity:
7     storage: 500Gi
```

```

8  volumeMode: Filesystem
9  accessModes:
10   - ReadWriteMany
11  persistentVolumeReclaimPolicy: Retain
12  nfs:
13   path: /exports/models
14   server: 10.0.1.100

```

Листинг 2: *PersistentVolumeClaim* для NFS-тома

```

1  apiVersion: v1
2  kind: PersistentVolumeClaim
3  metadata:
4   name: models - nfs - pvc
5   namespace: inference
6  spec:
7   accessModes:
8    - ReadWriteMany
9   resources:
10    requests:
11     storage: 500Gi
12  volumeName: models - nfs - pv

```

Поды, смонтировавшие этот *PVC* через *spec.volumes*, получают доступ к единой файловой системе, физически расположенной на *NFS*-сервере. *Kubernetes* автоматически связывает *PVC* с подходящим *PV* при наличии совпадающих параметров ёмкости и режима доступа. Для автоматизации создания *PV* по запросу *PVC* без ручного администрирования применяется динамическое выделение ресурсов (*dynamic provisioning*) через внешние провизионеры, такие как *NFS Subdir External Provisioner*, которые автоматически создают поддиректории на *NFS*-сервере для каждого нового *PVC*.

3.2.3 Устранение повторных загрузок

Введение *NFS* принципиально изменяет характер загрузок: первый под, обнаруживший отсутствие модели в *NFS*-директории, загружает её из внешнего репозитория и записывает в общее хранилище. Все последующие поды, монтирующие тот же *PVC*, обнаруживают файлы уже готовыми и пропускают этап загрузки. Таким образом, при масштабировании с 1 до *N*

реплик загрузка из внешнего источника выполняется ровно один раз, а не N раз, что устраняет ключевую неэффективность схемы *emptyDir*.

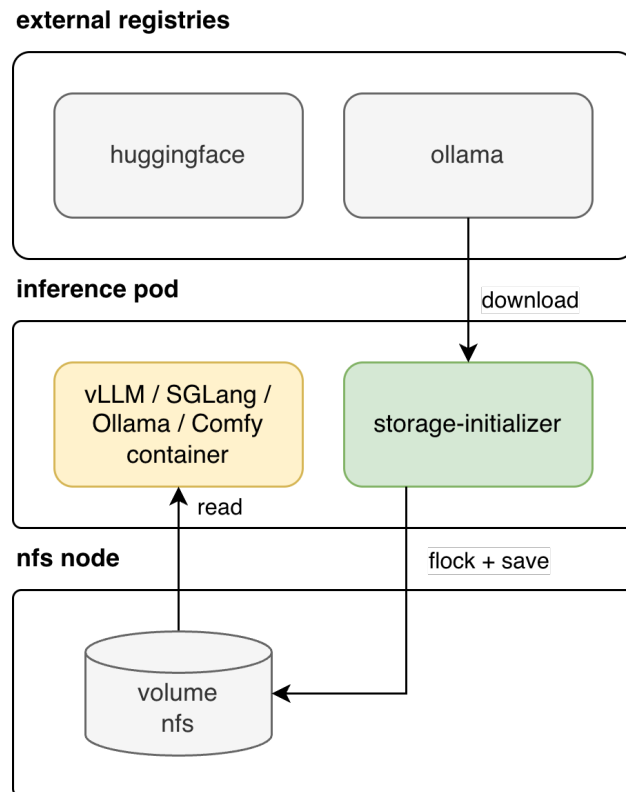


Рисунок 4 – Промежуточная архитектура на базе *NFS*: *init*-контейнер сохраняет модель в общее хранилище, а *runtime*-контейнеры повторно используют уже загруженные файлы

На рисунке 4 показан новый путь данных после перехода на общее *RWX*-хранилище. В отличие от схемы с *emptyDir*, загрузка выполняется не в локальный том *pod*, а в сетевой том *NFS*, доступный нескольким репликам одновременно. Это устраняет повторные скачивания одной и той же модели, но одновременно делает корректность инициализации зависимой от механизма взаимного исключения при первой записи в общую директорию.

3.3 Проблема параллельного скачивания и условие гонки

3.3.1 *Race condition* при первом развёртывании

Несмотря на устранение повторных загрузок при масштабировании уже существующей модели, переход к *NFS* порождает новую задачу: состояние гонки (*race condition*) при первом развёртывании или после принудительной инвалидации кэша. В этом сценарии несколько реплик запускаются

одновременно. Каждая из них на этапе инициализации проверяет директорию модели в *NFS* и обнаруживает её отсутствие. После этого несколько реплик независимо начинают параллельную загрузку весов по одному и тому же пути назначения.

Параллельная запись одних и тех же файлов из нескольких источников приводит к непредсказуемым последствиям: файлы могут оказаться повреждёнными вследствие чередования блоков от разных источников, директория модели может находиться в частично записанном состоянии, которое воспринимается другими подами как готовое. В результате *runtime*-контейнер может попытаться загрузить модель из повреждённого набора файлов, что приведёт к ошибке инициализации или некорректным ответам модели.

3.3.2 Алгоритм координированной загрузки

Для решения задачи консистентности при параллельном старте разработан алгоритм координированной загрузки, основанный на сочетании файловых блокировок и атомарных файловых операций. Алгоритм реализует паттерн «один загрузчик — множество ожидающих»:

- а) Под проверяет наличие финального маркер-файла *.ready* в директории модели. Если маркер присутствует, модель готова к использованию; загрузка не требуется, выполнение переходит к запуску *runtime*-контейнера.
- б) Если маркер отсутствует, под открывает специальный файл блокировки *.lock* в директории модели и выполняет попытку получения эксклюзивной блокировки (*LOCK_EX* | *LOCK_NB*).
- в) Под, успешно получивший блокировку, становится лидером загрузки. Перед началом загрузки лидер повторно проверяет наличие маркера *.ready* — другой лидер мог успеть завершить загрузку в промежутке между проверкой и получением блокировки. При обнаружении маркера лидер освобождает блокировку и переходит к запуску *runtime*-контейнера.
- г) Лидер начинает загрузку весов модели во временную директорию *<model>_tmp*, физически расположенную в том же *NFS*-разделе. Запись ведётся во временную директорию, что гарантирует отсутствие частично записанных данных по целевому пути.

- д) После завершения загрузки всех файлов лидер выполняет верификацию целостности: сравнивает суммарный объём файлов и, при наличии манифеста, проверяет контрольные суммы.
- е) Лидер выполняет атомарное переименование временной директории: $\langle model \rangle_tmp \rightarrow \langle model \rangle$. В *POSIX*-совместимых файловых системах операция *rename(2)* является атомарной в пределах одной файловой системы: наблюдатели видят либо отсутствие директории, либо полностью готовую директорию, но никогда не видят промежуточного состояния.
- ж) Лидер создаёт маркер-файл *.ready* в директории модели и освобождает файловую блокировку.

Поды, не получившие эксклюзивную блокировку на шаге 2, устанавливают разделяемую блокировку (*LOCK_SH*) на тот же файл *.lock*. Вызов *flock(LOCK_SH)* является блокирующим и возвращает управление только после того, как лидер освободит эксклюзивную блокировку. После возврата из блокирующего вызова под повторно проверяет маркер *.ready*. Если маркер присутствует, *pod* переходит к запуску *runtime*-контейнера. Если маркер отсутствует (например, лидер завершился аварийно до создания маркера), под повторяет попытку получить эксклюзивную блокировку и при успехе становится новым лидером.

Описанный алгоритм в псевдокоде:

Листинг 3: Псевдокод координированной загрузки модели

```
1 function EnsureModelReady(model_dir, lock_path, ready_marker):
2     if Exists(ready_marker):
3         return OK
4
5     lock_fd = Open(lock_path, CREATE | RDWR)
6
7     if TryLock(lock_fd, LOCK_EX | LOCK_NB) == SUCCESS:
8         # Повторная проверка после получения блокировки
9         if Exists(ready_marker):
10             Unlock(lock_fd)
11             return OK
12
13     tmp_dir = model_dir + "_tmp"
14     err = DownloadModel(source_url, tmp_dir)
15     if err != nil:
```

```

16         Unlock( lock_fd )
17         RemoveAll( tmp_dir )
18         return ERROR
19
20         Rename( tmp_dir , model_dir )      # атомарно
21         CreateFile( ready_marker )
22         Unlock( lock_fd )
23         return OK
24     else :
25         # Ожидаем завершения лидера
26         Lock( lock_fd , LOCK_SH )          # блокирующий вызов
27         Unlock( lock_fd )
28         # Рекурсивная проверка маркера
29         return EnsureModelReady( model_dir , lock_path , ready_marker )

```

3.4 Файловые блокировки *flock* и их особенности поверх *NFS*

3.4.1 *Advisory locking*: разделяемые и эксклюзивные блокировки

Системный вызов *flock(2)* реализует рекомендательные (*advisory*) блокировки на уровне открытого файлового дескриптора [28]. Рекомендательный характер означает, что ядро операционной системы не препятствует доступу к файлу процессам, которые не запрашивают блокировку. Консистентность обеспечивается только при условии, что все участники взаимодействия явно запрашивают и соблюдают соглашение о блокировках.

Поддерживаются два вида блокировок:

- **Разделяемая** (*shared, LOCK_SH*) — несколько процессов могут одновременно удерживать разделяемую блокировку на одном файле. Используется для операций чтения, не порождающих конфликтов при параллельном доступе.
- **Эксклюзивная** (*exclusive, LOCK_EX*) — только один процесс может удерживать эксклюзивную блокировку. Пока она активна, ни один другой процесс не может получить ни разделяемую, ни эксклюзивную блокировку на тот же файл.

Блокировки *flock* привязаны к описанию открытого файла (*open file description*), а не к иноду. Это означает, что дублирование файлового дескриптора через *fork(2)* или *dup(2)* приводит к тому, что оба дескриптора

ссылаются на одно описание и, следовательно, на одну и ту же блокировку. Блокировка автоматически снимается при закрытии последнего файлового дескриптора, ссылающегося на данное описание открытого файла, или при завершении процесса. Это свойство является встроенной защитой от «зависших» блокировок: аварийное завершение процесса-лидера автоматически освобождает блокировку.

3.4.2 NFSv3: блокировки через *Network Lock Manager*

Поведение *flock* поверх *NFS* существенно зависит от версии протокола. В *NFSv3* блокировки файлов не являются частью основного протокола. Они реализуются через вспомогательный сервис *Network Lock Manager (NLM)*, который взаимодействует с демонами *lockd* и *statd* на стороне клиента и сервера.

Данная архитектура имеет ряд ограничений. Демоны *lockd* и *statd* являются дополнительными точками отказа: при их недоступности или некорректной конфигурации операции блокировки возвращают ошибку *ENOLCK*. При сетевом разделе между клиентом и *NFS*-сервером блокировка может зависнуть. При перезагрузке *NFS*-сервера состояние блокировок восстанавливается не всегда корректно, что может привести к ситуации, когда двое клиентов считают себя держателями эксклюзивной блокировки. В ранних версиях ядра *Linux* (до 2.6.11) *flock* поверх *NFS* ограничивался локальной машиной и не распространял блокировку на другие клиенты.

3.4.3 NFSv4: встроенные блокировки и *lease*-механизм

В *NFSv4* файловые блокировки интегрированы непосредственно в основной протокол, что устраняет зависимость от внешних демонов *NLM*. Сервер *NFSv4* отслеживает состояние всех активных блокировок клиентов через механизм аренды (*lease*): каждый клиент обязан периодически подтверждать свою активность; при истечении аренды сервер освобождает все блокировки данного клиента. Это предотвращает зависшие блокировки даже при аварийном отключении клиента.

Тем не менее *NFSv4* не лишён ограничений. В *Linux*, начиная с версии 2.6.12, клиент *NFSv4* эмулирует *flock* через байт-диапазонные блокировки *fcntl(F_SETLK)*, что означает взаимодействие между двумя типами

блокировок. Для эксклюзивной блокировки через *flock* файл должен быть открыт с флагом записи, даже если запись не предполагается. При сетевом разделе (*network partition*) сервер продолжает удерживать информацию о блокировках до истечения аренды, а не немедленно освобождает их, что создаёт паузу в доступности ресурса.

3.4.4 Практическая модель надёжности: *flock* + *rename* + маркер

Анализ ограничений *flock* поверх *NFS* показывает, что в производственной среде нельзя полагаться исключительно на корректность блокировок при сетевых сбоях. Именно поэтому описанный алгоритм координированной загрузки использует комбинацию трёх механизмов, образующих эшелонированную защиту:

- а) **Файловая блокировка (*flock*)** — координирует конкурентный доступ в штатном режиме работы, когда сеть стабильна.
- б) **Атомарное переименование (*rename*)** — гарантирует, что читатели никогда не видят частично записанных данных. Операция *rename(2)* атомарна в пределах одной файловой системы согласно *POSIX*-стандарту.
- в) **Маркер готовности (*.ready*)** — является финальным критерием готовности модели. Даже если блокировка была выдана некорректно двум клиентам одновременно, только тот под завершит загрузку, который успеет выполнить *rename* и создать маркер первым. Второй под, завершив загрузку во временную директорию, обнаружит, что целевая директория уже существует, и примет её как готовую.

Такая комбинация значительно повышает устойчивость алгоритма к нештатным ситуациям, не требуя строгих гарантий от файловой системы.

3.5 *Kubernetes Lease* и *Leader Election*

3.5.1 Ресурс *coordination.k8s.io/v1/Lease*

Файловые блокировки поверх *NFS* предоставляют достаточную надёжность в большинстве сценариев, однако остаются уязвимыми к ситуациям, связанным с некорректной конфигурацией *NFS*-клиента или нестандартным поведением *NFS*-сервера. Для случаев, когда требуется более высокий уровень гарантий, *Kubernetes* предоставляет встроенный механизм

распределённой координации через ресурс *coordination.k8s.io/v1/Lease* [29].

Ресурс *Lease* представляет собой объект *Kubernetes API*, хранимый в *etcd* и обладающий строгими гарантиями согласованности за счёт оптимистичной блокировки через поле *resourceVersion*. Объект содержит следующие ключевые поля спецификации:

- *holderIdentity* — строковый идентификатор текущего держателя аренды (как правило, имя пода или уникальный идентификатор процесса). Пустое значение означает, что аренда никем не удерживается.
- *leaseDurationSeconds* — максимальная длительность аренды в секундах без обновления. Если текущий держатель не обновил аренду в течение этого интервала, другие кандидаты вправе захватить её.
- *renewTime* — временная метка последнего обновления аренды текущим держателем. Лидер обязан периодически обновлять это поле, чтобы подтвердить свою активность.
- *acquireTime* — временная метка первоначального захвата аренды текущим держателем.
- *leaderTransitions* — счётчик количества смен лидера, полезный для мониторинга стабильности координации.

Kubernetes API-сервер гарантирует, что при одновременной попытке обновить объект *Lease* несколькими клиентами только один запрос будет успешным: операция обновления проверяет *resourceVersion*, и если объект был изменён другим клиентом с момента последнего чтения, запрос отклоняется с ошибкой конфликта. Это свойство обеспечивает выбор ровно одного лидера даже при одновременном участии произвольного числа кандидатов.

3.5.2 Алгоритм *client-go leaderelection*

Стандартная библиотека *k8s.io/client-go/tools/leaderelection* реализует алгоритм выбора лидера поверх ресурса *Lease*. Алгоритм работает следующим образом:

- а) При запуске каждый кандидат пытается создать объект *Lease* с собственным *holderIdentity* или обновить существующий объект, если аренда истекла. Операция выполняется через *Kubernetes API*.
- б) Кандидат, чей запрос на обновление был принят первым (с

корректным *resourceVersion*), становится лидером. Остальные кандидаты получают ответ о конфликте версии и переходят в режим ожидания.

- в) Лидер периодически (с интервалом *RetryPeriod*) обновляет поле *renewTime* объекта *Lease*, не давая сроку аренды истечь. Функция *OnStartedLeading(ctx)* вызывается при получении статуса лидера и выполняет целевую операцию — загрузку модели.
- г) Если лидер завершается аварийно или теряет связь с *API*-сервером, обновление *renewTime* прекращается. По истечении *leaseDurationSeconds* аренда помечается как просроченная.
- д) Один из ожидающих кандидатов обнаруживает просроченную аренду, захватывает *Lease* и становится новым лидером. Функция *OnStoppedLeading* вызывается на бывшем лидере при обнаружении потери статуса.

Структура *LeaderElectionConfig* задаёт ключевые параметры: *LeaseDuration* (типовое значение — 15 секунд), *RenewDeadline* (типовое значение — 10 секунд), *RetryPeriod* (типовое значение — 2 секунды), а также набор *callback*-функций (*OnStartedLeading*, *OnStoppedLeading*, *OnNewLeader*).

3.5.3 Координация *per-model* через пространство имён *Lease*

В контексте загрузки моделей ресурс *Lease* именуется по схеме *model-<name>-<revision>* в пространстве имён соответствующего сервиса инференса. Это позволяет независимо координировать загрузку разных моделей: одновременно могут присутствовать несколько объектов *Lease*, каждый из которых управляет загрузкой конкретной версии конкретной модели.

Лидер, захвативший *Lease* для данной пары (модель, ревизия), выполняет загрузку в *NFS*, создаёт маркер *.ready* и освобождает *Lease*. Остальные поды, работающие с той же моделью, отслеживают либо состояние *Lease* через механизм *Watch*, либо появление маркера *.ready* в *NFS*-директории, либо используют оба сигнала в *conjunction*. Данный подход исключает условие гонки на уровне *Kubernetes* без каких-либо требований к семантике *NFS*-блокировок.

Преимущество *Kubernetes Lease* перед *flock* состоит в том, что гарантии согласованности обеспечиваются *etcd* — распределённым хранилищем с

алгоритмом *Raft*, а не файловой системой. Таким образом, *Leader Election* через *Kubernetes Lease* является предпочтительным механизмом координации для случаев, когда *init*-контейнер имеет доступ к *Kubernetes API*. В случаях, когда доступ к *API* недоступен или нежелателен (например, в минималистичных окружениях), *flock* с атомарным переименованием остаётся резервным вариантом.

3.6 Ограничения *NFS*-этапа

3.6.1 Централизованный сервер как единая точка отказа

Несмотря на существенное улучшение по сравнению со схемой *emptyDir*, *NFS*-подход обнаружил ряд системных ограничений, которые были выявлены в процессе промышленной эксплуатации и ставшие основанием для разработки следующего этапа архитектуры.

***NFS*-сервер как *SPOF*.**

Весь ввод-вывод чтения весов моделей проходит через один *NFS*-сервер. При его недоступности — перезагрузке, сетевой проблеме, исчерпании дискового пространства или переполнении очереди запросов — вся система инференса лишается доступа к моделям. *Kubernetes* не содержит встроенных механизмов высокой доступности для внешнего *NFS*-сервера: сбой сервера воспринимается как продолжительный таймаут сетевой операции, что приводит к зависанию подов на этапе монтирования или чтения.

***Bottleneck* пропускной способности.**

При одновременном масштабировании большого числа реплик все они читают файлы модели через сетевую карту *NFS*-сервера. Пропускная способность его сетевого интерфейса физически ограничена и не масштабируется горизонтально. При *burst*-масштабировании с 0 до N реплик суммарный входящий трафик на *NFS*-сервер достигает $N \times$ скорость чтения, что при типичных размерах моделей (7–70 ГБ) быстро приводит к насыщению канала.

Отсутствие локальности данных.

Даже если узел уже обслуживал данную модель ранее, при создании нового пода *runtime*-контейнер читает данные из *NFS* по сети заново. Локальный диск узла не используется для кэширования: информация, однажды прочитанная с *NFS*, не сохраняется на узле и при следующем запросе снова загружается по

сети. Это приводит к тому, что задержка *cold start* определяется не скоростью локального чтения (которая может достигать 3–5 ГБ/с для *NVMe*), а пропускной способностью сети между нодой и *NFS*-сервером.

Сложность построения отказоустойчивого *NFS*.

Создание высокодоступного *NFS*-сервера требует значительных дополнительных усилий: кластеризации с использованием *Pacemaker/Corosync*, синхронной репликации данных через *DRBD* или аналогичные средства, настройки *shared-storage* для кластерного *NFS*. Такое решение существенно увеличивает операционную сложность и стоимость инфраструктуры, при этом не устраняя принципиальных проблем с масштабируемостью полосы пропускания.

Нагрузка на сеть при *burst*-масштабировании.

В отличие от схемы *emptyDir*, где нагрузка распределялась между нодами (каждый узел загружал из внешнего источника независимо), в *NFS*-схеме весь трафик концентрируется на одном сервере. При резком росте числа реплик весь кластер конкурирует за пропускную способность одного сетевого пути.

3.6.2 CAP-теорема и выбор *NFS*

CAP-теорема [30] утверждает, что распределённая система не может одновременно обеспечивать согласованность (*Consistency*), доступность (*Availability*) и устойчивость к разделению сети (*Partition tolerance*). При разрыве сети (*Partition*) между нодами *Kubernetes* и *NFS*-сервером система вынуждена выбирать между двумя оставшимися свойствами.

Классический *NFS* с режимом монтирования *hard* выбирает согласованность, жертвуя доступностью: при недоступности сервера все операции ввода-вывода блокируются и ожидают восстановления соединения. *Runtime*-контейнер, ожидающий чтения весов из *NFS*, зависает до тех пор, пока сетевой сбой не будет устранён. Режим монтирования *soft* позволяет операциям завершаться с ошибкой по таймауту, однако приводит к потенциальной несогласованности данных при восстановлении.

В итоге *NFS*-этап предлагает приемлемые характеристики для небольших кластеров с умеренной нагрузкой, но системно неприменим в условиях масштабирования, требующего высокой параллельности и устойчивости к сбоям сети.

3.7 Выводы по разделу

NFS-этап значительно сократил количество обращений к внешнему репозиторию: вместо N независимых загрузок на N реплик однократная загрузка в общее хранилище обеспечивает повторное использование данных всеми подами. Алгоритм координации через файловые блокировки *flock* и маркер готовности обеспечивает консистентность при параллельном старте. *Kubernetes Lease API* предоставляет более надёжную альтернативу для координации в случаях, когда гарантии *NFS*-блокировок недостаточны.

Вместе с тем *NFS* как централизованное сетевое хранилище переносит проблему с внешнего репозитория на один внутренний сервер, создавая единую точку отказа и ограниченный по пропускной способности ресурс. При масштабировании до десятков параллельных реплик, каждая из которых читает модель размером 7–70 ГБ, *NFS*-сервер становится узким местом архитектуры. Отсутствие локального кэширования на дисках нод означает, что одни и те же данные многократно передаются по сети при каждом событии масштабирования.

Для устранения этих ограничений необходим переход к архитектуре, в которой данные хранятся на локальных дисках *worker*-нод и доставляются напрямую между агентами, минуя централизованный сервер. Такая архитектура, реализованная через распределённый кэш на уровне нод с *P2P*-взаимодействием и *FUSE*-клиентом, обеспечивает локальность данных, горизонтальную масштабируемость пропускной способности и устойчивость к отказу отдельных компонентов. Именно она составляет содержание следующего раздела.

4 РАЗРАБОТКА *P2P/FUSE*-СИСТЕМЫ КЭШИРОВАНИЯ *ML*-МОДЕЛЕЙ

Данный раздел описывает основную разработку работы — специализированную систему распределённого кэширования *ML*-моделей, устраняющую ограничения *NFS*-этапа. Система построена на трёх ключевых компонентах: *Redis*-реестре метаданных (*Control Plane*), агенте *storage-agent* на каждой *worker*-ноде (*Data Plane*) и *FUSE*-клиенте *storage-mounter*, предоставляющем *ML-runtime* стандартный файловый интерфейс. Вместе они реализуют трёхуровневую иерархию доступа к данным: локальный диск ноды, *P2P*-сеть кластера и внешний репозиторий как резервный источник.

4.1 Общая идея и требования к архитектуре

Центральная идея архитектуры состоит в том, что локальные диски *worker*-нод образуют распределённый кэш, доступный всем подам в кластере. Данные модели хранятся не в одном месте, а распределены между нодами в виде чанков — блоков фиксированного размера. При запуске нового пода *ML-runtime* обращается к ближайшей копии данных, а не к централизованному *NFS* или внешнему репозиторию. Этот подход переносит хранилище данных из единого централизованного узла в локальность вычислений, что соответствует принципу *data locality*, давно утвердившемуся в распределённых системах обработки данных.

Переход от *NFS*-этапа к *P2P/FUSE*-архитектуре мотивирован фундаментальным ограничением сетевых файловых систем: единый *NFS*-сервер становится узким местом при одновременном запуске нескольких подов с тяжёлыми *ML*-моделями. Если 10 подов одновременно читают модель весом 70 ГБ с одного *NFS*-сервера, суммарный поток данных составляет сотни гигабайт, что превышает пропускную способность большинства сетевых интерфейсов. *P2P*-архитектура устраняет это ограничение: каждый под обращается к локальному диску своей ноды или к нескольким соседним нодам, распределяя нагрузку на чтение горизонтально.

Требования, которым должна удовлетворять архитектура, сформулированы исходя из ограничений предшествующих подходов:

- **Минимальная задержка первого байта:** *ML-runtime* должен получить первые данные модели в течение миллисекунд, не

дожидаясь полной загрузки весов. Это обеспечивается механизмом *lazy loading* через *FUSE*.

- **Отсутствие единой точки отказа:** падение одной ноды не должно делать модель недоступной. Репликация чанков на несколько нод и *fallback* на внешний репозиторий обеспечивают это свойство.
- **Прозрачность для *ML-runtime*:** фреймворки *vLLM* и *TGI* не должны требовать модификации кода для работы с системой. *FUSE* предоставляет стандартный *POSIX*-интерфейс.
- **Масштабируемость:** добавление новых нод должно автоматически увеличивать совокупный кэш без перенастройки.
- **Устойчивость к *stale metadata*:** устаревшие записи об ушедших нодах должны автоматически удаляться через *TTL* механизм *Redis*.
- **Совместимость с *Kubernetes*:** компоненты должны разворачиваться стандартными *Kubernetes*-примитивами (*DaemonSet*, *ConfigMap*, *Secret*) без изменения прав хостовой системы, за исключением доступа к диску.

Разработанная архитектура логически делится на три уровня.

Уровень 1. *Control Plane* (Реестр метаданных).

Роль центрального источника информации выполняет *Redis*-кластер. В нём хранятся метаданные о каждом чанке каждой модели: на каких нодах он расположен, каков его размер и хеш. Кроме того, каждый активный *storage-agent* периодически обновляет запись о себе (*heartbeat*) с *TTL*. *Redis* выступает «телефонной книгой» кластера: любой компонент может узнать, у каких агентов есть нужный чанк, за одну операцию чтения. *Control Plane* намеренно отделён от *Data Plane*: метаданные не смешиваются с данными, что упрощает масштабирование и отказоустойчивость каждого уровня независимо.

Уровень 2. *Data Plane* — локальный (Инференс и виртуальная файловая система).

На каждой *compute*-ноде работает контейнер с *ML*-движком и компонент *storage-mounter*. *FUSE*-клиент монтирует виртуальную файловую систему в локальный путь пода. Для *ML*-движка создаётся иллюзия, что файлы модели уже целиком лежат на диске. При чтении определённого диапазона байтов *storage-mounter* перехватывает системный вызов и по *Unix Domain Socket* запрашивает данные у локального *storage-agent*. *UDS* выбран намеренно:

коммуникация остаётся в пределах одного хоста и не нагружает сеть датацентра.

Уровень 3. *Data Plane* — горизонтальный (*P2P*-сеть).

При промахе локального кэша *storage-agent* обращается к *Redis* для поиска узла, имеющего нужный чанк. Данные передаются напрямую по *TCP* между агентами на разных нодах. Если чанк отсутствует во всём кластере — выполняется *fallback*-загрузка из внешнего репозитория (*HuggingFace*, *Ollama*, *ModelScope*). Горизонтальный *Data Plane* реализует свойство *self-healing* кэша: каждая загрузка из внешнего источника наполняет локальный кэш, уменьшая вероятность повторного обращения к внешнему репозиторию.

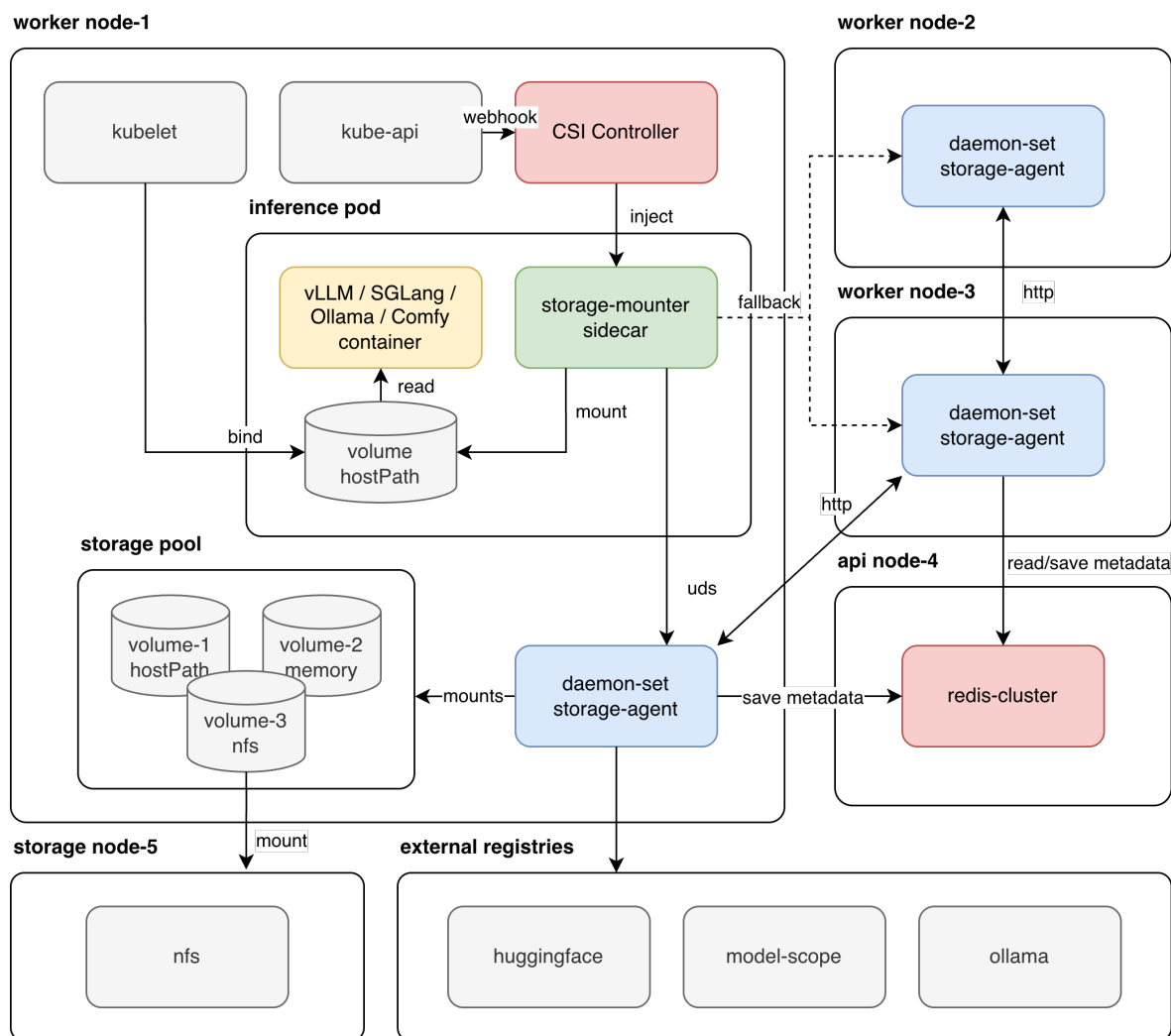


Рисунок 5 – Целевая *P2P/FUSE*-архитектура распределённого кэширования *ML*-моделей

Рисунок 5 визуализирует полный маршрут чтения данных в

предложенной системе. Локальные обращения *storage-mounter* к *storage-agent* изолированы в пределах ноды через *UDS*, межузловой обмен чанками выполняется напрямую между агентами, а *Redis* используется только как реестр метаданных и не включается в тракт передачи самих весов. Это принципиально отличает предложенный подход от *NFS*: центральный компонент сохраняется лишь для координации, тогда как основной поток чтения распределяется между рабочими узлами.

4.2 Модель данных: репозиторий, модель, ревизия, файл, чанк

Для корректного кэширования и адресации данных принята иерархическая модель, отражающая структуру реестров *ML*-моделей. Иерархия адресации включает следующие уровни:

- **registry** — источник модели: *huggingface*, *ollama*, *modelscope*;
- **repository** — пространство имён владельца (например, *deepseek-ai*);
- **model** — имя модели (например, *DeepSeek-LLM-67B-Chat*);
- **revision** — версия модели (*commit hash* или *tag*); обеспечивает иммутабельность кэша;
- **filepath** — относительный путь к файлу внутри репозитория (например, *model-00001-of-00004.safetensors*);
- **chunk_id** — порядковый номер чанка в файле (начиная с 0);
- **chunk_size** — фиксированный размер чанка (настраивается, типично 8–64 МБ).

Иммутабельность через *revision* принципиально важна для корректности кэша. Файлы конкретной ревизии модели никогда не изменяются: если *revision* зафиксирован как конкретный *commit hash*, то содержимое файлов при данном *hash* всегда одинаково. Поэтому кэшированный чанк всегда корректен, пока *revision* в ключе совпадает с запрошенной версией. Это устраняет необходимость в механизмах инвалидации кэша при обновлении модели: новая версия получает новый *revision*, и её чанки хранятся независимо от чанков предыдущей версии. Старые чанки вытесняются по *LRU*-политике по мере заполнения диска.

Физический путь к чанку на диске агента строится следующим образом:

```
<cache_dir>/  
  <registry>/<repo>/<model>/<revision>/  
    <hash(filepath)[0:2]>/
```

```
<hash(filepath)[2:4]>/  
<hash(filepath)>/  
chunk_<chunk_id>
```

Хеширование пути к файлу решает две задачи. Первая — устранение проблемы специальных символов и длинных имён: имена файлов в *HuggingFace*-репозиториях могут содержать слешы, пробелы и длинные пути, несовместимые с прямым использованием в качестве имени файла на диске. Хеш *SHA-256* от *filepath* даёт детерминированное, фиксированной длины имя директории. Вторая задача — равномерное распределение файлов: файловые системы *Linux* имеют ограничения на число записей в одной директории (*ext4* — до нескольких десятков тысяч записей без *dir_index*, что вызывает деградацию производительности); двухуровневое шардирование по первым четырём символам хеша обеспечивает $256 \times 256 = 65536$ возможных поддиректорий, практически исключая перегрузку отдельных директорий.

Ключ в *Redis* для записи о чанке строится аналогично иерархии на диске:

```
<registry>:<repo>:<model>:<revision>:<hash(filepath)>:<  
chunk_id>
```

Такая схема позволяет находить все чанки конкретного файла по префиксу ключа через *SCAN*, а также атомарно получать список нод, хранящих конкретный чанк, одной командой *SMEMBERS*. Единообразие схемы адресации между диском и *Redis* упрощает отладку: по ключу в *Redis* можно немедленно определить физический путь на диске агента.

Размер чанка влияет на компромисс между накладными расходами и гранулярностью. Мелкие чанки (4–8 МБ) допускают более точное управление тем, какие части модели кэшируются, но увеличивают число метаданных в *Redis* и число операций *FUSE*. Крупные чанки (64–128 МБ) снижают накладные расходы, но могут приводить к загрузке данных, которые *ML-runtime* не запросит. Диапазон 16–32 МБ является практически обоснованным для последовательного чтения больших *.safetensors*-файлов в *ML*-инференсе.

4.3 *Redis* как реестр метаданных чанков

4.3.1 Обоснование выбора *Redis*

Выбор *Redis* в роли *Control Plane* обусловлен рядом характеристик,

критичных для задачи реестра метаданных в динамической *P2P*-среде.

Суб-миллисекундная задержка.

Redis хранит данные в оперативной памяти и обеспечивает время отклика на операции чтения и записи, измеряемое микросекундами [31]. При обработке запроса чанка *storage-agent* выполняет один *SMEMBERS*-запрос к *Redis* перед обращением к соседнему узлу: накладные расходы на этот *lookup* должны быть пренебрежимо малы по сравнению со временем сетевой передачи чанка (десятки–сотни миллисекунд). Суб-миллисекундная задержка *Redis* делает его не узким местом системы.

Атомарность операций.

Команды *SADD*, *HSET*, *EXPIRE* выполняются атомарно. Это позволяет агенту регистрировать чанк без состояний гонки: добавить свой идентификатор в множество пиров (*SADD chunk:...:peers <node_id>*) и установить метаданные (*HSET chunk:...:meta size ... checksum ...*) атомарно. Другой агент, запрашивающий *SMEMBERS chunk:...:peers*, получит либо запись целиком, либо вообще не получит идентификатор данного узла — промежуточного состояния не существует.

Встроенный *TTL*.

Каждый ключ в *Redis* может иметь ограниченное время жизни. Если агент прекратил публиковать *heartbeat* (нода упала или перезагрузилась), его записи исчезнут автоматически через *TTL*. Это устраняет необходимость в явном механизме *garbage collection*: реестр самоочищается без дополнительного координатора. *Redis* использует комбинацию ленивого удаления (при обращении к истёкшему ключу) и периодической активной очистки, обеспечивая эффективное управление памятью при наличии *TTL*.

Горизонтальное масштабирование.

Redis Cluster шардирует ключи по 16384 хеш-слотам, распределяя нагрузку между мастер-узлами. При отказе мастера кластер автоматически повышает реплику [31]. *Redis Sentinel* решает аналогичную задачу для меньших кластеров: мониторит мастер, выполняет автоматическое переключение на реплику при его недоступности [32]. Оба решения устраняют *Redis* как единую точку отказа, что было ключевым недостатком *NFS*-этапа.

Сравнение с альтернативами.

etcd обеспечивает строгую линейную консистентность через алгоритм *Raft*,

но вносит задержку в единицы миллисекунд и оптимизирован для небольшого объёма редко меняющихся данных конфигурации — не для высокочастотного реестра тысяч чанков. *ZooKeeper* не имеет встроенного *TTL* и требует сложного клиентского кода для имитации его функциональности; кроме того, *ZooKeeper* написан на *Java* и сложен в операционном обслуживании. *Consul* избыточен: большинство его возможностей (*service mesh*, *ACL*, *health checks*) не используются при решении задачи реестра чанков. *Redis* обеспечивает точно необходимый набор: быстрое чтение/запись эфемерных метаданных, автоматический *TTL*, богатые структуры данных (*Sets*, *Hashes*), горизонтальное масштабирование.

4.3.2 Схема ключей

В реестре применяется следующая трёхуровневая схема ключей:

- *node:<node_id>* — *heartbeat* агента. Значение (*JSON* или строка) содержит *IP*-адрес, *TCP*-порт, *load score*, метку времени последнего обновления. *TTL* = 30–60 с. Если агент не обновляет *heartbeat*, запись автоматически исчезает, и следующий запрос к этому агенту не выполняется.
- *chunk:<logical_id>:peers* — *Set* идентификаторов нод, хранящих данный чанк. Операция *SADD <node_id>* регистрирует ноду; *SMEMBERS* возвращает всех известных пиров за $O(1)$ (одна хеш-таблица в памяти *Redis*). Сложность *SMEMBERS* для *Set* из k пиров — $O(k)$.
- *Hash* с метаданными чанка строится по следующей схеме:

```
model:<registry>:<repo>:<model>:<revision>:  
file:<hash>:chunk:<id>:meta
```

В *Hash* хранятся размер в байтах (*size*) и контрольная сумма *SHA-256* (*checksum*); ключ используется для верификации целостности после загрузки.

Разделение информации о пирах и метаданных чанка на два ключа (*:peers* и *:meta*) позволяет обновлять их независимо. Метаданные чанка (размер, хеш) не изменяются после первой записи — ключ *:meta* фактически *write-once*. Множество пиров (*:peers*) динамически растёт по мере того, как новые ноды кэшируют чанк, и может уменьшаться при явном *SREM* или истечении *TTL*.

После успешного сохранения чанка на диске агент выполняет следующую атомарную последовательность: *SADD chunk:...:peers <node_id>* и *HSET chunk:...:meta size <n> checksum <sha256>*. Это обеспечивает согласованность реестра с фактическим состоянием дисков.

4.3.3 *Heartbeat* и управление жизненным циклом

Каждый *storage-agent* публикует *heartbeat* каждые 10–30 с командой:

SET node:<node_id> <metadata_json> EX <ttd>

Команда *SET ... EX* атомарно записывает значение и устанавливает *TTL*. Это исключает ситуацию, когда значение записано, но *TTL* ещё не установлен — состояние, которое возникло бы при раздельном вызове *SET* и *EXPIRE*.

TTL выбирается в диапазоне 2–3 интервалов *heartbeat*: при *TTL* = 60 с и интервале *heartbeat* = 20 с нода выдерживает 3 пропущенных *heartbeat* прежде чем её запись устареет. Это обеспечивает устойчивость к кратковременным сбоям сети (задержке *heartbeat*) без риска оставить в реестре записи реально отказавших нод.

Любой агент, запрашивающий список пиров для чанка, проверяет свежесть каждого кандидата: *EXISTS node:<peer_id>*. Если *heartbeat* истёк, агент пропускает данный пир и переходит к следующему. Это двухуровневая проверка: *TTL heartbeat* — первый уровень защиты, явная проверка *EXISTS* — второй.

4.4 Обнаружение узлов: *Gossip*, *mDNS* и *Redis heartbeat*

Для построения реестра активных агентов рассматривались три подхода.

4.4.1 *Gossip Protocol*

Gossip (эпидемический протокол) — децентрализованный механизм, при котором каждый узел периодически обменивается состоянием с k случайно выбранными соседями [33]. Выбор соседей случаен и независим в каждом раунде, что обеспечивает экспоненциальное распространение информации: после r раундов информация достигает примерно $(k+1)^r$ узлов (при отсутствии пересечений). Для кластера из N нод информация распространяется за:

$$T_{\text{gossip}} = O(\log N). \quad (6)$$

где N — число нод в кластере, а T_{gossip} — число раундов, требуемых для распространения информации по всему кластеру.

Для кластера из 1000 нод это порядка 10 раундов; для 25 000 нод — около 15 раундов. Каждый раунд занимает время, равное интервалу *gossip* (обычно 1–5 с), что означает задержку сходимости от 10 до 75 с для крупных кластеров. Именно этот временной разрыв делает *Gossip* неприемлемым для задачи реестра чанков.

Преимущества *Gossip*.

Полная децентрализация без центрального координатора, высокая отказоустойчивость — протокол продолжает работать при отказе произвольного числа узлов, масштабируемость до сотен тысяч нод. *Gossip* применяется в *Redis Cluster* для обмена информацией о конфигурации слотов и состоянии узлов, а также в *Apache Cassandra* и *HashiCorp Consul* для поддержания актуальной топологии. Для этих применений *eventual consistency* приемлема, поскольку топология меняется редко.

Недостатки для задачи реестра чанков.

Задержка сходимости $O(\log N)$ раундов неприемлема при *cold start*, где каждая миллисекунда критична. *Gossip* создаёт постоянный фоновый трафик (каждый узел периодически инициирует обмен с несколькими соседями), что нагружает сеть. Отладка *eventual-consistent* системы существенно сложнее: нет гарантии, что в момент запроса реестр отражает актуальное состояние. Наконец, *Gossip* не даёт детерминированного ответа на вопрос «где сейчас находится чанк X » — только вероятностный.

4.4.2 *mDNS/UDP Broadcast*

Multicast DNS (RFC 6762) позволяет устройствам в локальной сети анонсировать себя через *UDP*-мультикаст на адрес 224.0.0.251, порт 5353 [34]. Каждое устройство периодически отправляет *mDNS Announcement*, сообщая своё имя хоста и *IP*-адрес. Другие устройства могут найти сервис через *mDNS Query*.

В качестве механизма *zero-configuration networking mDNS* прост в реализации и не требует дополнительной инфраструктуры — достаточно

стандартной поддержки в ОС. Именно это делает *mDNS* популярным в домашних сетях (*Apple Bonjour*, *DNS-SD*).

Однако *mDNS* имеет серьёзные ограничения в среде *Kubernetes* [34]:

- **Overlay-сети блокируют мультикаст**: большинство *CNI*-плагинов (*Calico*, *Flannel*, *Cilium*) используют *VXLAN* или *IPIP*, которые инкапсулируют *L2*-трафик в *L3*-пакеты и не распространяют мультикаст-фреймы между хостами.
- **Отсутствие *IGMP/PIM* в облаке**: облачные провайдеры (*AWS*, *GCP*, *Azure*) не поддерживают *multicast routing* на уровне *underlay*-сети.
- **Нарушение сетевой изоляции**: обходной путь через *hostNetwork: true* даёт пододу доступ к мультикасту хостовой сети, но нарушает принципы сетевой изоляции *Kubernetes* и создаёт риски безопасности.
- **Ограниченная масштабируемость**: мультикаст масштабируется только в пределах одного *broadcast*-домена (*VLAN*), что несовместимо с *multi-zone* кластерами, охватывающими несколько зон доступности.

Таким образом, *mDNS* неприменим для обнаружения узлов в типичном *production Kubernetes*-кластере.

4.4.3 *Redis heartbeat*: выбранный подход

Каждый *storage-agent* периодически обновляет запись о себе в *Redis* с *TTL*. Поиск активных агентов сводится к одному *SCAN* по префиксу *node:** или *SMEMBERS* для конкретного чанка. Это обеспечивает время поиска $O(1)$, что принципиально отличает *Redis heartbeat* от *Gossip* ($O(\log N)$ раундов).

Таблица 4 обобщает ключевые различия трёх подходов. Из анализа следует, что для кластеров *Kubernetes* умеренного масштаба (до нескольких тысяч нод) *Redis heartbeat* обеспечивает оптимальный баланс: минимальная задержка поиска, детерминированный ответ, прозрачная интеграция с *Kubernetes*-инфраструктурой.

Единственный риск — зависимость от доступности *Redis* — устраняется развёртыванием *Redis Sentinel* (для кластеров малого и среднего масштаба) или *Redis Cluster* с несколькими мастерами и репликами (для больших кластеров). При временной недоступности *Redis storage-agent* переключается на локальный кэш и *fallback*, сохраняя работоспособность.

Таблица 4 – Сравнение алгоритмов обнаружения узлов

Критерий	<i>Gossip</i>	<i>mDNS/Broadcast</i>	<i>Redis heartbeat</i>
Скорость <i>lookup</i>	$O(\log N)$ раундов	Быстро (локальная сеть)	$O(1)$
Консистентность	<i>eventual</i>	Нет	<i>TTL-based</i>
Поддержка в <i>K8s</i>	Да	Плохая	Да
<i>SPOF</i>	Нет	Нет	<i>Redis</i> (без <i>HA</i>)
Сложность реализации	Средняя	Низкая	Низкая
Масштабируемость	Высокая	Ограниченная	Высокая (<i>Cluster</i>)
Гарантия актуальности	Нет (<i>eventual</i>)	Только локальная сеть	<i>TTL</i> -ограниченная

4.5 Компонент *storage-agent*: *DaemonSet*-агент на каждой ноде

storage-agent — агент, развёртываемый как *DaemonSet* на каждой *worker*-ноде кластера. *Kubernetes DaemonSet* гарантирует, что на каждой подходящей ноде запущен ровно один экземпляр, который автоматически создаётся при добавлении новой ноды и удаляется при её выводе из эксплуатации. Агент монтирует локальный диск ноды (через *hostPath volume*) и использует его как кэш-хранилище чанков.

Использование *DaemonSet* вместо *Deployment* критически важно: именно *DaemonSet* обеспечивает ковенант «один агент на ноду», без которого невозможно гарантировать, что локальный кэш ноды управляется ровно одним процессом. Конкурентный доступ нескольких агентов к одному кэш-директории потребовал бы механизма блокировок, что усложнило бы архитектуру.

Агент является основным компонентом *Data Plane* и выполняет следующие функции:

- управление локальным диском и *chunk*-кэшем ноды (запись, чтение, вытеснение по *LRU*);
- обслуживание запросов от локального *storage-mounter* через *Unix Domain Socket* (в целевой архитектуре);
- обслуживание *TCP*-запросов от агентов на соседних нодах (*P2P*-поставщик чанков);

- периодическая публикация *heartbeat* и метаданных чанков в *Redis*;
- *fallback*-загрузка чанков из внешнего репозитория при их отсутствии в кластере.

4.5.1 HTTP API и текущая реализация

В текущем прототипе реализован *HTTP-API* с двумя основными эндпоинтами:

- получение дерева файлов модели (имена файлов, размеры, хеши):

```
GET /api/v1/models/{registry}/{repo}/{model}/{revision}/tree
```

- чтение диапазона байтов файла модели:

```
GET /api/v1/models/{registry}/{repo}/{model}/{revision}/raw/{filepath}
```

Диапазон байтов в запросе чтения задаётся *HTTP*-заголовком *Range: bytes=X-Y*.

Выбор *HTTP* как протокола в текущем прототипе обусловлен простотой реализации и отладки. В целевой архитектуре для локальной связи *storage-mounter* → *storage-agent* планируется переход на бинарный протокол поверх *UDS* для снижения накладных расходов.

Компонент *model-gateway* реализует приоритетный порядок источников при обработке запроса чанка:

- а) проверяется наличие чанка в локальном хранилище (*localstorage*) через *os.Stat* по физическому пути;
- б) при промахе инициируется загрузка из *HuggingFace* через *HTTP Range Request*;
- в) загруженный чанк записывается на локальный диск атомарно: сначала во временный файл в той же файловой системе, затем переименовывается через *os.Rename*.

Атомарная запись через *os.Rename* критически важна: системный вызов *rename(2)* на *Linux* атомарен при условии, что исходный и целевой пути находятся в одной файловой системе. Это гарантирует, что конкурентный читатель никогда не увидит частично записанный чанк: файл либо полностью существует, либо отсутствует. Этот инвариант устраняет необходимость в явных блокировках при параллельных запросах к одному чанку.

Компонент *localstorage* организует файлы на диске по иерархии,

описанной в разделе 4.2. Доступ к метаданным файловой системы (размер чанка, наличие файла) осуществляется через *os.Stat*, что исключает необходимость отдельной локальной базы данных: файловая система сама является индексом локального кэша.

4.5.2 Поддержка *singleflight* для предотвращения *thundering herd*

При одновременном поступлении нескольких запросов одного и того же чанка (например, когда несколько горутин в *storage-mounter* запрашивают один чанк параллельно) без специальных мер каждый запрос инициирует отдельную загрузку из внешнего источника. Это явление называется *thundering herd*. Для его предотвращения планируется использование механизма golang.org/x/sync/singleflight: если загрузка чанка уже выполняется, повторные запросы ожидают её завершения и получают тот же результат. Это сокращает нагрузку на внешний репозиторий и ускоряет ответ повторным запросам.

4.5.3 Целевое *P2P*-расширение

В целевой архитектуре *storage-agent* дополняется логикой *P2P*-обмена. При промахе локального кэша агент выполняет следующую последовательность:

- а) запрашивает *Redis: SMEMBERS chunk:<id>:peers* для получения списка нод с нужным чанком;
- б) фильтрует список: проверяет *EXISTS node:<peer_id>* для каждого кандидата;
- в) ранжирует кандидатов по *load score* и задержке;
- г) скачивает чанк по *TCP* от выбранного пира;
- д) сохраняет на локальный диск атомарно через *os.Rename*;
- е) регистрирует себя в *Redis: SADD chunk:<id>:peers <local_node_id>*.

Этот процесс замыкает *P2P*-цикл: каждый новый потребитель чанка становится потенциальным поставщиком для следующего. После нескольких таких циклов чанки популярных моделей распространяются по кластеру органически, без центрального координатора.

4.5.4 Многоуровневый кэш *storage-agent*

Эффективность *storage-agent* как прокси-шлюза между *storage-mounter* и внешними реестрами определяется организацией кэша. В прототипе реализована трёхуровневая иерархия, каждый уровень которой обслуживает свой класс запросов.

***L0* — горячий уровень в оперативной памяти.**

Компонент *inMemoryChunkCache* реализует *SLRU*-кэш (*Segmented Least Recently Used*) с ёмкостью, задаваемой в байтах. Внутри кэша поддерживается двусвязный список активных чанков и два хеш-словаря: первый сопоставляет ключ (идентификатор модели и номер чанка) с узлом списка, второй — с самими данными. При обращении к чанку его узел перемещается в начало списка. При переполнении ёмкости вытеснение выполняется с хвоста списка, пока не освободится достаточно места. Время операций *Get* и *Set* составляет $O(1)$. Уровень *L0* обслуживает повторные обращения к одним и тем же чанкам без дискового ввода-вывода; при перезапуске агента его содержимое теряется, однако это допустимо, поскольку данные восстановимы с диска или из внешнего источника.

***L1* — персистентный кэш на локальном диске.**

Каждый чанк хранится как отдельный файл в иерархии директорий, описанной в разделе 4.2. Запись на диск выполняется по схеме *write-behind*: метод *Write()* помещает чанк в буферизованный канал и немедленно возвращает управление, не дожидаясь физической записи. Отдельная горутина-воркер читает из канала и записывает файл атомарно — сначала во временный файл в той же файловой системе, затем вызывает *os.Rename()*. Атомарность *rename(2)* гарантирует, что конкурентный читатель никогда не увидит частично записанного чанка. Разделение пути записи и пути чтения устраняет задержку дисковой операции из критического пути ответа клиенту.

Выбор файлового кэша.

Альтернативы на основе встроенных хранилищ — *BadgerDB*, *BoltDB*, *LevelDB* — не подходят для чанков размером 16–128 МБ. *LSM*-деревья (*BadgerDB*, *Pebble*) оптимизированы для записи небольших значений: крупные значения уходят в *Value Log*, что порождает *write amplification* и фрагментацию. *BoltDB* хранит все данные в едином *B-tree* файле с эксклюзивной блокировкой при записи, что создаёт конкуренцию под

нагрузкой. Файловый кэш лишён этих ограничений: операционная система обеспечивает параллельный доступ к независимым файлам, а её *page cache* автоматически удерживает горячие чанки в *RAM* без дополнительного уровня буферизации.

Расширение: *SQLite*-индекс для структурированного вытеснения.

В текущей реализации удаление устаревших чанков требует обхода файловой системы по времени последнего доступа (*atime*). Запланированное расширение — таблица *SQLite* в *WAL*-режиме с записями (*model_ref*, *chunk_id*, *size_bytes*, *last_access_unix*) — позволит выполнять *LRU*-вытеснение запросом вида *ORDER BY last_access_unix ASC LIMIT N*, не читая файловую систему. Пара *SQLite WAL* + файловый кэш сохраняет преимущества прямого *I/O* и при этом добавляет структурированный учёт использования.

Улучшение: *ARC* вместо *LRU* для горячего уровня.

При смешанной нагрузке — повторные обращения к популярным моделям параллельно с разовыми сканированиями при первой загрузке — чистый *LRU* вытесняет горячие чанки разовыми промахами. Алгоритм *ARC* (*Adaptive Replacement Cache*) поддерживает два рабочих списка (*T1* — недавно добавленные, *T2* — повторно запрошенные) и два *ghost*-списка ключей (*B1* и *B2*). Попадания в *B1* сдвигают внутреннюю границу *p* в пользу рецентности, в *B2* — в пользу частотности. Это делает *ARC* самонастраивающимся без ручного подбора параметров.

4.6 Компонент *storage-mounter*: *FUSE*-клиент

storage-mounter — *FUSE*-клиент, предоставляющий *ML-runtime* стандартный файловый интерфейс для доступа к весам модели. Он монтирует виртуальную файловую систему в путь внутри пода (например, */mnt/models*).

4.6.1 Механизм *FUSE*

FUSE (*Filesystem in Userspace*) позволяет реализовать файловую систему в виде обычного пользовательского процесса без привилегий *root* и без изменения кода ядра [35]. Архитектура включает три взаимодействующих компонента:

- **Ядерный модуль *fuse.ko***: перехватывает обращения *VFS* к *FUSE*-точке монтирования и передаёт запросы пользовательскому

демону через символическое устройство */dev/fuse*.

- **VFS (Virtual Filesystem Switch)**: единый интерфейс ядра *Linux* для всех файловых систем; именно он определяет, что файл принадлежит *FUSE*-ФС, и направляет запрос соответствующему обработчику.
- **Пользовательский демон**: реализует логику файловой системы; в данном случае — *storage-mounter*, написанный на *Go* с использованием библиотеки github.com/hanwen/go-fuse/v2 [36].

Жизненный цикл системного вызова *read()* в *FUSE* выглядит следующим образом:

- а) Приложение *ML-runtime* (*vLLM*) вызывает *read(fd, buf, count)*.
- б) Ядро перехватывает вызов через *VFS*, определяет принадлежность файла к *FUSE*-ФС.
- в) Ядерный модуль *fuse.ko* упаковывает запрос в пакет протокола *FUSE* и помещает в очередь.
- г) Пользовательский демон *storage-mounter* читает пакет из */dev/fuse*.
- д) Демон обрабатывает запрос: вычисляет *chunk_id*, обращается к *storage-agent*.
- е) *storage-agent* возвращает данные чанка.
- ж) Демон записывает ответ обратно в */dev/fuse*.
- и) Ядро передаёт данные приложению.

Для *ML-runtime* этот процесс полностью прозрачен: он воспринимает *FUSE*-ФС как обычную локальную файловую систему. Это ключевое свойство позволяет интегрировать систему без изменения кода *vLLM* или *TGI*.

Преимущество *FUSE* перед альтернативами — *CSI volume* с полным копированием весов в *emptyDir* — в поддержке *lazy loading*: данные загружаются только при обращении к конкретному диапазону байтов. *ML-runtime* может начать инициализацию первых слоёв модели ещё до того, как остальные веса получены с диска или из сети. Это перекрывает фазы вычисления и ввода-вывода, сокращая время *cold start*.

Накладные расходы *FUSE*.

Каждый системный вызов, направленный *FUSE*-ФС, требует переключения контекста *kernel* $\leftrightarrow$ *userspace*. Для рабочей нагрузки *ML*-инференса, при которой читаются большие последовательные блоки (*.safetensors*-файлы по несколько гигабайт), накладные расходы минимизируются за счёт крупных чанков: каждое переключение контекста амортизируется на несколько

мегабайт полезных данных. Для мелкогранулярного случайного доступа *FUSE* может быть неоптимален, однако паттерн чтения *ML*-весов — преимущественно последовательный.

4.6.2 Ключевые операции

storage-mounter реализует следующие *FUSE*-операции [36]:

- *getattr* — возвращает метаданные файла: размер, права, тип. Позволяет *ML-runtime* видеть полный размер файла модели без его загрузки. Вызывается при каждом обращении к пути и при выполнении *stat(2)*.
- *readdir* — возвращает список файлов директории модели. Получается из *HuggingFace API* (*/api/models/<repo>/tree/<revision>*) и кэшируется локально на стороне клиента.
- *lookup* — разрешает имя файла в *inode*. Вызывается при открытии файла по имени; возвращает атрибуты файла или *ENOENT*.
- *open* — открывает файл для чтения. Может проверять права доступа и инициализировать внутренние структуры для отслеживания паттерна чтения.
- *read(offset, size)* — читает диапазон байтов файла. Здесь происходит ключевая трансляция в *chunk*-запросы к *storage-agent*.

4.6.3 Трансляция *read(offset, size)* в *chunk*-запросы

Ключевая операция *read(offset, size)* транслируется в запросы чанков следующим образом. По значениям *offset* и *chunk_size* вычисляется диапазон затронутых чанков:

$$chunk_id_{\text{start}} = \left\lfloor \frac{offset}{chunk_size} \right\rfloor, \quad chunk_id_{\text{end}} = \left\lfloor \frac{offset + size - 1}{chunk_size} \right\rfloor. \quad (7)$$

где *offset* — начальное смещение чтения в файле, *size* — объём запрошенного диапазона, *chunk_size* — размер одного чанка.

Для каждого чанка в диапазоне $[chunk_id_{\text{start}}, chunk_id_{\text{end}}]$ отправляется отдельный запрос к *storage-agent*. При *chunk_size* = 16 МБ и запросе на чтение 64 МБ, начиная с произвольного смещения, затронуто не более 5 чанков (4 полных + 1 граничный). Полученные данные объединяются в

буфере, из которого вырезается точный запрошенный диапазон. Граничные чанки используются частично: первый — начиная с $offset \% chunk_size$, последний — до $(offset + size - 1) \% chunk_size + 1$.

4.6.4 Текущая реализация и целевое развитие

В текущем прототипе *storage-mounter* реализован с использованием библиотеки github.com/hanwen/go-fuse/v2 [36]. Поддержаны операции *Lookup*, *Getattr*, *Readdir*. Список файлов модели получается из *HuggingFace API*. Операция *read* в текущей реализации делегируется непосредственно к *API HuggingFace* через *HTTP Range Request*.

В целевой архитектуре операция *read* транслируется в запросы к локальному *storage-agent* через *UDS*: вместо прямого обращения к внешнему репозиторию *storage-mounter* запрашивает чанк у агента, который обслуживает три сценария — *local cache hit*, *peer hit*, *remote fallback*.

Дополнительно планируется реализация *prefetching*: при обнаружении последовательного паттерна чтения *storage-mounter* асинхронно запрашивает следующие N чанков до их явного запроса *ML-runtime*. Это позволяет скрыть задержку сетевой или дисковой передачи за временем обработки текущего чанка *GPU*-ядром.

4.6.5 Кэш и буферизация чтения в *storage-mounter*

storage-mounter обрабатывает поток системных вызовов *read()* от *ML-runtime* и транслирует их в запросы чанков к *storage-agent*. Для минимизации задержки и нагрузки на агента в стороне клиента реализовано несколько механизмов буферизации.

Кольцевой буфер (*streaming ring buffer*).

Компонент *streaming.Cache* хранит массив фиксированной длины *capacity*, в котором чанк с номером k размещается по индексу $k \bmod capacity$. Запись нового чанка всегда успешна и выполняется за $O(1)$: если в слоте уже находится другой чанк, он вытесняется без дополнительных сравнений. Чтение проверяет соответствие $chunk.Id == chunkId$: при коллизии двух чанков с одинаковым индексом результат возвращается как промах. Такая схема оптимальна для последовательного чтения одного файла: при линейном обходе каждый слот занимает следующий чанком ровно тогда, когда

предыдущий уже не нужен.

Предзагрузчик (*prefetcher*).

При каждом вызове *Read(chunkId)* предзагрузчик запускает до *prefetchAhead-1* горутин, каждая из которых загружает следующий чанк, если его нет в кольцевом буфере. Количество одновременно выполняемых предзагрузок ограничивается параметром *prefetchParallelism*, что предотвращает насыщение полосы пропускания *UDS*-соединения. Для исключения дублирующихся загрузок одного чанка несколькими горутинками применяется механизм *singleflight.Group*: если загрузка данного *chunkId* уже выполняется, повторный запрос блокируется и получает тот же результат без отдельного обращения к агенту. Эффект предзагрузки — перекрытие фаз вычисления *GPU* и сетевого ввода-вывода: пока *GPU* обрабатывает текущий слой модели, следующие чанки уже загружаются в буфер.

Пул байтовых буферов.

Каждый чанк требует выделения байтового буфера объёмом *chunkSize*. При параллельных запросах нескольких *FUSE*-горутин без механизма переиспользования каждый вызов *make([]byte, chunkSize)* создаёт новую аллокацию: при *chunkSize* = 32 МБ и 64 параллельных горутинках это даёт 2 ГБ аллокаций в секунду, что насыщает сборщик мусора. Компонент *pool.Pool* реализует мультиразмерный пул через *sync.Pool*: для каждого уникального значения *chunkSize* хранится отдельный пул. Метод *Get(size)* возвращает переиспользованный буфер, *Put(buf)* возвращает его в пул после обработки чанка.

Ring buffer vs per-file LRU.

Рабочая нагрузка *ML*-инференса преимущественно последовательна: слои *.safetensors*-файла читаются от начала к концу, поэтому кольцевой буфер покрывает этот сценарий без коллизий. Однако шардированные модели (несколько *.safetensors*-файлов, загружаемых одновременно) порождают коллизии: чанки разных файлов с одинаковым номером вытесняют друг друга по модулю ёмкости буфера. В этом сценарии предпочтительна *per-file LRU*-структура: отдельный *LRU*-кэш для каждого открытого файла изолирует их чанки. Реализация *per-file LRU* хранит словарь *filepath* → *lruCache* с очисткой записи при закрытии файла. Кольцевой буфер остаётся предпочтительным для одиночных больших файлов как более простой и эффективный; переход к *per-file LRU* оправдан при наличии двух и более

одновременно открытых файлов с перекрывающимися диапазонами *chunkId*.

4.7 UDS и TCP: разделение локальной и междоузовой коммуникации

Архитектура применяет два различных транспортных протокола в зависимости от физической топологии взаимодействия. Разделение транспортных протоколов по границе «внутри ноды / между нодами» отражает фундаментальные различия в задержке и модели безопасности.

4.7.1 Unix Domain Socket: внутриузловая связь

Unix Domain Socket (UDS) используется для связи *storage-mounter* → *local storage-agent* на одной ноде. *UDS* работает через файловую систему (*/var/run/storage-agent.sock*), минуя *IP*-стек полностью. Ключевые технические свойства *UDS*:

- **Отсутствие сетевого стека:** данные не инкапсулируются в *IP/TCP*-пакеты; нет расчёта контрольных сумм, маршрутизации, обработки заголовков, трёхстороннего рукопожатия.
- **Копирование через память ядра:** передача данных осуществляется прямым копированием между адресными пространствами процессов в ядре. Типичная задержка — 1–5 мкс, пропускная способность — до 10 ГБ/с.
- **Изоляция через права доступа:** права на *socket*-файл (*chmod/chown*) определяют, какие процессы могут устанавливать соединение. Механизм *SO_PEERCREDS* позволяет серверу получить *PID*, *UID* и *GID* клиента непосредственно от ядра без дополнительной аутентификации.
- **Нет *NetworkPolicy*:** *UDS* не пересекает сетевые границы *Kubernetes*, поэтому политики *NetworkPolicy* к нему неприменимы.

4.7.2 TCP: междоузовая связь

TCP используется для связи *storage-agent* → *storage-agent* между нодами. *TCP* обеспечивает надёжную упорядоченную доставку данных, управление потоком и контроль перегрузки — свойства, необходимые при передаче чанков объёмом 8–64 МБ по реальной сети. *TCP*-соединения

совместимы со всеми *CNI*-плагинами и политиками *NetworkPolicy*.

Контроль перегрузки *TCP* (алгоритмы *CUBIC*, *BBR*) автоматически адаптирует скорость передачи к доступной пропускной способности канала и реагирует на потери пакетов. Это особенно важно при одновременной *P2P*-передаче нескольких чанков между несколькими парами нод: *TCP* предотвращает перегрузку коммутаторов датацентра.

Таблица 5 – Сравнение *UDS* и *TCP* в контексте системы

Критерий	<i>UDS</i>	<i>TCP</i>
Область применения	Один узел	Разные узлы
<i>IP</i> -стек	Отсутствует	Используется
Задержка	1–5 мкс	0,1–1 мс (<i>RTT</i> в ДЦ)
Накладные расходы	Минимальные	Заголовки, <i>handshake</i> , <i>CRC</i>
Надёжность	Да (ядро ОС)	Да (<i>ACK</i> , <i>retransmit</i>)
<i>Flow control</i>	Нет	Да (<i>TCP window</i>)
<i>Congestion control</i>	Нет	Да (<i>CUBIC</i> , <i>BBR</i>)
Безопасность	<i>FS perms</i> , <i>SO_PEERCRE</i>	<i>TLS/mTLS</i> (целевое)
<i>K8s NetworkPolicy</i>	Не применима	Применима

Таблица 5 демонстрирует, что выбор протокола строго определяется физическим расположением компонентов: *UDS* — когда оба процесса на одном хосте, *TCP* — при межнодовом взаимодействии. Комбинация двух протоколов даёт оптимум: внутри ноды минимальная задержка *UDS*, между нодами надёжность и совместимость *TCP*.

4.7.3 Оценка пропускной способности через *BDP*

Произведение полосы пропускания и задержки (*BDP*, *Bandwidth-Delay Product*) для *TCP*-канала:

$$BDP = B \cdot RTT, \quad (8)$$

где *B* — пропускная способность канала связи, *RTT* — *round-trip time* между двумя узлами.

Формула определяет объём данных, одновременно находящихся «в полёте» при полной утилизации канала. Для внутренней сети датацентра с

$$B = 10 \text{ Гбит/с и } RTT = 0.5 \text{ мс:}$$

$$BDP = \frac{10 \cdot 10^9}{8} \text{ Б/с} \times 0.5 \cdot 10^{-3} \text{ с} \approx 625 \text{ КБ.} \quad (9)$$

где $10 \cdot 10^9/8$ — перевод полосы пропускания из бит/с в байт/с, а $0.5 \cdot 10^{-3} \text{ с}$ — задержка RTT , выраженная в секундах.

Это означает, что для полной утилизации 10G-канала размер TCP -окна должен быть не менее 625 КБ. Типичный размер чанка 8–64 МБ существенно превышает BDP , что позволяет одному TCP -соединению утилизировать канал без специальной настройки размера окна [37]. При переходе на 25G и 100G сети BDP возрастает пропорционально, и может потребоваться явная настройка `net.core.rmem_max` и `net.ipv4.tcp_rmem`.

4.8 Алгоритм чтения чанка

4.8.1 Полная тринадцатиступенчатая последовательность

Полный сценарий чтения чанка включает следующую последовательность шагов, охватывающую все три уровня архитектуры:

- а) *ML-runtime* (*vLLM*) вызывает `read(filepath, offset, size)`.
- б) Ядро *Linux* перехватывает вызов через *VFS* и передаёт *storage-mounter* через `/dev/fuse`.
- в) *storage-mounter* вычисляет диапазон чанков по формуле (7).
- г) *storage-mounter* отправляет запрос (`chunk_id`) локальному *storage-agent* через *UDS*.
- д) *storage-agent* проверяет локальное хранилище операцией `os.Stat` по физическому пути.
- е) **Сценарий А (*local cache hit*)**: чанк найден на диске — данные читаются из файла и немедленно возвращаются через *UDS*.
- ж) **Сценарий В (*local miss*)**: агент запрашивает *Redis*: `SMEMBERS chunk:<id>:peers`.
- и) Агент проверяет свежесть каждого кандидата: `EXISTS node:<peer_id>`.
- к) **Сценарий В.1 (*peer cache hit*)**: выбирается пир с минимальным *score*; устанавливается TCP -соединение; чанк загружается; сохраняется атомарно на диск через `os.Rename`; агент регистрирует себя: `SADD chunk:<id>:peers <local_node>`.
- л) **Сценарий В.2 (*peer miss, remote fallback*)**: список пиров пуст или все

- пиры недоступны; агент выполняет *HTTP Range Request* к *HuggingFace*; полученный чанк сохраняется и регистрируется в *Redis*.
- м) Чанк возвращается *storage-mounter* через *UDS*.
- н) *storage-mounter* собирает данные всех запрошенных чанков и вырезает нужный диапазон.
- п) Данные возвращаются через *FUSE* в *ML-runtime*.

4.8.2 Модель среднего времени доступа (*AMAT*)

Среднее время доступа к чанку описывается трёхуровневой моделью *AMAT* (*Average Memory Access Time*) [37]:

$$AMAT = T_{local} + (1 - H_l) [T_{p2p} + (1 - H_p) \cdot T_{remote}], \quad (10)$$

где:

- T_{local} — время чтения чанка из локального диска (сценарий *A*);
- H_l — вероятность попадания в локальный кэш (*local hit rate*);
- T_{p2p} — время получения чанка от соседнего агента по *TCP* (сценарий *B.1*);
- H_p — вероятность попадания в кэш соседних нод (*peer hit rate*);
- T_{remote} — время загрузки чанка из внешнего репозитория (сценарий *B.2*).

Вывод формулы (10) основан на формуле полного математического ожидания. С вероятностью H_l запрос обслуживается локально за время T_{local} . С вероятностью $(1 - H_l)$ происходит промах: система переходит на второй уровень, где с вероятностью H_p чанк находится у соседнего агента (T_{p2p}), и с вероятностью $(1 - H_p)$ требуется загрузка из внешнего источника (T_{remote}) [38]. Перемножение условных вероятностей даёт коэффициенты перед каждым слагаемым.

После прогрева кластера (первая загрузка популярной модели завершена) можно ожидать $H_l \approx 0.85$ и $H_p \approx 0.90$. Тогда доля запросов, достигающих внешнего источника:

$$(1 - H_l)(1 - H_p) = 0.15 \times 0.10 = 0.015. \quad (11)$$

где H_l — вероятность попадания в локальный кэш, а H_p — вероятность попадания в кэш соседних нод при локальном промахе. Около 1.5% запросов

уходят во внешний репозиторий, тогда как 98.5% обслуживаются из локального или соседнего кэша. Это кардинально отличается от схемы *storage-initializer*, при которой каждый новый под загружает модель целиком из внешнего источника. Поскольку T_{remote} может составлять десятки секунд для модели весом 70 ГБ, снижение его коэффициента с 1.0 до 0.015 является определяющим фактором ускорения *cold start*.

4.9 Политика кэширования и вытеснения

4.9.1 Распределение популярности моделей: закон Ципфа

Популярность моделей в реальных производственных системах распределена неравномерно: небольшое число наиболее востребованных моделей обеспечивает большую часть запросов. Аналогичная закономерность наблюдается в *CDN (Content Delivery Networks)*, веб-кэшировании и системах хранения медиаконтента. Это распределение соответствует закону Ципфа [39]:

$$P(k) = \frac{k^{-\alpha}}{\sum_{i=1}^M i^{-\alpha}}, \quad (12)$$

где k — ранг модели по популярности (чем ниже ранг, тем популярнее), α — параметр наклона (для *ML*-моделей оценивается в диапазоне $\alpha \approx 0.8\text{--}1.2$ по публичным оценкам исследований веб-трафика), M — общее число моделей.

Закон Ципфа означает, что модель с рангом 1 запрашивается примерно в 2^α раз чаще модели с рангом 2. При $\alpha = 1$ и $M = 100$ моделях: топ-10 моделей (10%) генерируют более 60% запросов, топ-20 моделей — около 75%. Следствие для кэширования: даже кэш, вмещающий 20% самых популярных моделей, обеспечивает *hit rate* выше 75%. Нет необходимости хранить все существующие модели — достаточно приоритизировать наиболее запрашиваемые.

4.9.2 LRU как политика вытеснения

При превышении лимита дискового пространства применяется алгоритм *LRU (Least Recently Used)*: вытесняются чанки, к которым дольше всего не было обращений. *LRU* хорошо согласуется с законом Ципфа: часто используемые модели (низкий ранг) регулярно обращаются к своим чанкам,

обновляя метку «последнего использования», тогда как редко используемые (высокий ранг) постепенно вытесняются.

Практическая реализация *LRU* для файлового кэша: при обращении к чанку обновляется атрибут *atime* файла на диске (*O_NOATIME* отключён намеренно для отслеживания времени доступа). При необходимости вытеснения перебираются файлы в директории кэша, сортируются по *atime* по возрастанию, и удаляются до восстановления свободного места.

4.9.3 Аппроксимация Чи для оценки *hit rate*

Для приближённой оценки ожидаемого *hit rate* кэша размером C при распределении Ципфа применяется аппроксимация Чи (*Che's approximation*) [40]:

$$h(n) \approx 1 - e^{-\lambda_n t_C(C)}, \quad (13)$$

где λ_n — интенсивность запросов к n -й модели (определяется из (12) умножением на общую интенсивность запросов), $t_C(C)$ — характеристическое время кэша: при данном размере кэша C объект с интенсивностью запросов $\lambda_n > 1/t_C$ будет храниться с высокой вероятностью.

Аппроксимация Чи позволяет оценить совокупный *hit rate* для заданного размера кэша без полного моделирования трасс запросов. Это полезно при планировании дискового ресурса нод: можно заранее оценить, какой размер кэша на ноде (например, 200 ГБ или 1 ТБ *NVMe*) даст достаточный *hit rate* для ожидаемого распределения запросов.

4.10 Параллельная загрузка, выбор пира и *prefetching*

4.10.1 Алгоритм выбора пира

При наличии нескольких пиров, хранящих нужный чанк, система выбирает оптимальный узел для скачивания. Алгоритм выбора пира основан на следующих критериях, ранжированных по приоритету:

- а) **Свежесть *heartbeat***: пир с истёкшим или отсутствующим *heartbeat* немедленно исключается (*EXISTS node:<peer_id> = 0*). Это защита от направления трафика к недоступным нодам.

б) **Load score**: агент с меньшей вычислительной и сетевой нагрузкой предпочтительнее. *Load score* публикуется в *heartbeat*-записи *Redis* и обновляется каждые 10–30 с.

в) **Локальность**: пир на той же стойке (*rack*) или в той же зоне доступности предпочтительнее за счёт меньшего *RTT* и более высокой пропускной способности внутрирячной сети.

г) **Измеренный *RTT***: периодические зондирующие запросы позволяют уточнить актуальную задержку до известных пиров.

Оценочная функция пира: $\text{score}(p) = \alpha \cdot \text{latency_ms}(p) + \beta \cdot \text{load}(p)$, где α и β — весовые коэффициенты, настраиваемые в конфигурации агента. Выбирается пир с минимальным *score*.

4.10.2 Параллельная загрузка и закон Амдала

Если несколько чанков модели отсутствуют в локальном кэше, и доступны несколько пиров, параллельная загрузка с разных пиров существенно сокращает общее время. Время параллельной загрузки [39]:

$$T_{\text{parallel}} = \frac{S}{B_{\text{total}}} + \text{RTT}, \quad (14)$$

где S — суммарный объём загружаемых данных, $B_{\text{total}} = \sum_{i=1}^k B_i$ — суммарная полоса пропускания k параллельных соединений, *RTT* — задержка установления соединений (пренебрежимо мала в датацентре).

Ускорение параллельной загрузки ограничено последовательными частями процесса — законом Амдала [41]:

$$S(n) = \frac{1}{(1-p) + p/n}, \quad (15)$$

где p — доля работы, поддающейся параллелизации (загрузка чанков), $(1-p)$ — последовательная часть (координация через *Redis*, сборка результата, инициализация *FUSE*). При $p = 0.9$ и $n = 4$ пирах:

$$S(4) = \frac{1}{0.1 + 0.225} = \frac{1}{0.325} \approx 3.1. \quad (16)$$

где $S(4)$ — ускорение при использовании четырёх параллельных источников, а значения 0.1 и 0.225 соответствуют последовательной и распараллеливаемой частям формулы (15). При $n = 16$ пирах: $S(16) \approx 5.8$. Предел при $n \rightarrow \infty$:

$S_{\max} = 1/(1 - p) = 10$. Таким образом, увеличение числа параллельных источников сверх 8–10 даёт убывающую отдачу, и оптимальное число пиров для параллельной загрузки — в диапазоне 4–8 для типичного датацентра с 10G-сетью.

4.10.3 Оптимальный размер чанка

Выбор размера чанка представляет собой задачу оптимизации с двумя конкурирующими составляющими. Функция суммарных накладных расходов на единицу данных:

$$f(C) = \frac{T_{\text{overhead}}}{C} + \frac{C}{B_{\text{net}}}, \quad (17)$$

где T_{overhead} — фиксированные накладные расходы на один чанк (переключение контекста *FUSE*, установка *TCP*-соединения, операции *Redis*), B_{net} — пропускная способность сети. Минимум функции (17) достигается при значении, найденном из условия нулевой производной $df/dC = 0$:

$$C_{\text{opt}} = \sqrt{T_{\text{overhead}} \cdot B_{\text{net}}}. \quad (18)$$

При $T_{\text{overhead}} = 1$ мс и $B_{\text{net}} = 10$ Гбит/с $= 1.25 \times 10^9$ Б/с:

$$C_{\text{opt}} = \sqrt{10^{-3} \times 1.25 \times 10^9} = \sqrt{1.25 \times 10^6} \approx 1118 \text{ КБ} \approx 1 \text{ МБ}. \quad (19)$$

где 10^{-3} с — накладные расходы на один чанк, 1.25×10^9 Б/с — пропускная способность сети в байтах в секунду, C_{opt} — оптимальный размер чанка. Реальные накладные расходы T_{overhead} включают несколько компонент и в зависимости от конфигурации могут составлять от единиц до десятков миллисекунд; при $T_{\text{overhead}} = 10$ мс оптимальный размер чанка возрастает до ≈ 3.5 МБ. Диапазон 8–64 МБ, применяемый на практике, соответствует консервативной оценке накладных расходов с запасом для обеспечения приемлемого *time-to-first-byte*.

4.10.4 Алгоритм *prefetching*

При последовательном паттерне чтения (характерном для загрузки весов модели слой за слоем в *vLLM*) *storage-mounter* применяет алгоритм упреждающей загрузки. После обслуживания запроса чанка k асинхронно

запрашиваются чанки $k+1, k+2, \dots, k+N$, где N — размер окна *prefetching*. К моменту, когда *ML-runtime* запрашивает чанк $k+1$, он уже находится в локальном буфере *storage-mounter*. Сетевая задержка скрыта за временем обработки предыдущего чанка *GPU*-ядром.

Алгоритм обнаруживает последовательный паттерн за $O(1)$: достаточно сравнить текущее смещение с ожидаемым (предыдущее смещение + размер чанка). При обнаружении паттерна запускаются N параллельных горутин, каждая загружает один следующий чанк через *UDS* к *storage-agent*. Пространственная сложность буфера *prefetching*: $O(N \times C)$. Значение $N = 10$ при $C = 16$ МБ даёт буфер 160 МБ — приемлемо для типичной ноды.

4.11 Надёжность и высокая доступность

4.11.1 Формула доступности при репликации

Чанк считается доступным, если хотя бы одна нода, хранящая его, работоспособна. При факторе репликации r и вероятности доступности каждой ноды a :

$$A = 1 - (1 - a)^r. \quad (20)$$

где A — вероятность доступности чанка, a — вероятность доступности одной ноды, r — коэффициент репликации.

При $a = 0.99$ и $r = 3$ репликах:

$$A = 1 - (0.01)^3 = 1 - 10^{-6} = 0.999999. \quad (21)$$

где $0.01 = 1 - a$ — вероятность отказа одной ноды, а $r = 3$ — число независимых реплик чанка.

Вероятность одновременного отказа всех трёх реплик составляет 10^{-6} — один случай на миллион «нодо-дней» [42]. Для сравнения: *NFS*-сервер с доступностью $a = 0.999$ (три девятки) при $r = 1$ (нет репликации) обеспечивает ту же $A = 0.999$. Добавление репликации в *P2P*-архитектуре даёт шесть девяток без изменения доступности отдельной ноды.

Репликация в предложенной архитектуре возникает органически: как только один агент загружает чанк из внешнего источника и регистрирует его в *Redis*, другие агенты могут найти его через *SMEMBERS* и при необходимости

скачать копию. Администратор может задать минимальный фактор репликации для «горячих» моделей: агент при регистрации чанка проверяет, есть ли уже r пиров в `:peers`, и если нет — реплицирует самостоятельно.

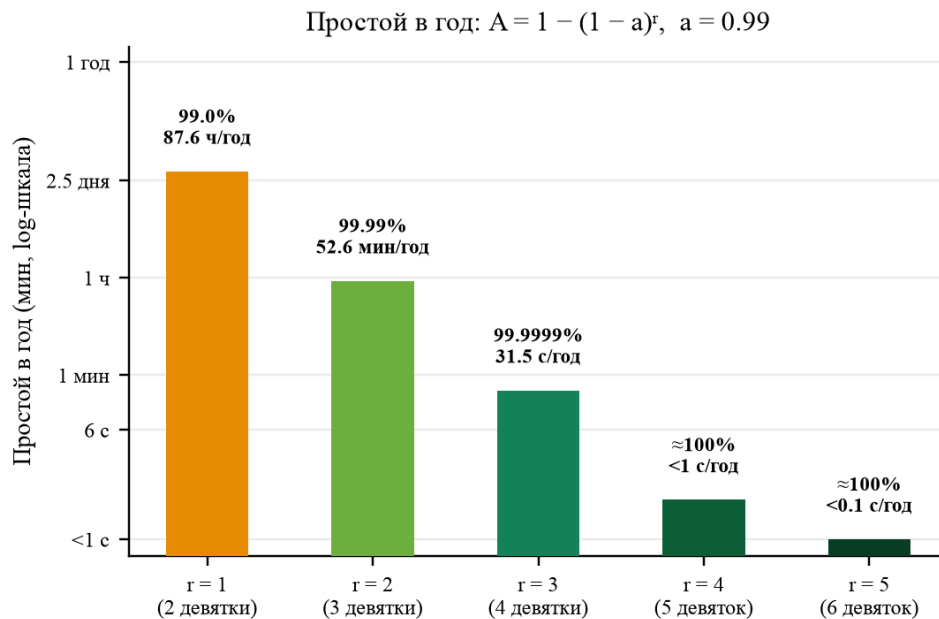


Рисунок 6 — Простой в год в зависимости от коэффициента репликации чанков ($a = 0.99$). При $r = 3$ простой сокращается с 87.6 ч/год до 31.5 с/год — переход от двух к четырём девяткам доступности.

4.11.2 *TTL* как защита от *stale metadata*

TTL на *Redis*-записях автоматически устраняет *stale metadata*: если агент прекратил обновлять *heartbeat*, его записи исчезнут через *TTL*. Следующий запрос не будет направлен к этому агенту. Агент, обнаружив ошибку соединения с потенциальным пиром (*ECONNREFUSED*, *ETIMEDOUT*), немедленно исключает его из текущей операции и опционально выполняет *SREM chunk:<id>:peers <dead_node>* для ускорения очистки реестра.

4.11.3 *Fallback* и *degraded mode*

External fallback — загрузка из *HuggingFace* или другого внешнего репозитория — является конечной гарантией доступности. Пока внешний репозиторий доступен, система способна получить любой чанк, даже если кластерный кэш полностью пуст. *Fallback* не является оптимальным по времени (десятки секунд для больших чанков), но обеспечивает корректность

операции.

При временной недоступности *Redis* агент переключается в деградированный режим: обслуживает запросы только из локального кэша или через *fallback*, без *P2P*-координации. Это предотвращает полный сбой системы при проблемах с *Redis* и соответствует принципу *graceful degradation*.

4.11.4 *Eventual consistency* для метаданных

Redis-реестр обеспечивает *eventual consistency* для метаданных чанков. При добавлении новой ноды с чанками её записи становятся видимы другим агентам немедленно после успешного *SADD*. При удалении ноды её *heartbeat*-запись исчезнет через *TTL*. В промежутке между отказом ноды и истечением *TTL* её идентификатор присутствует в реестре, но попытка соединения вернёт ошибку, и агент корректно обработает её, выбрав следующий пир. Таким образом, *eventual consistency* в части метаданных не нарушает корректность алгоритма чтения — лишь добавляет задержку в крайне редких пограничных случаях.

4.12 Выводы по разделу

Разработанная архитектура *P2P/FUSE*-системы кэширования устраняет все ключевые ограничения *NFS*-этапа и формирует многоуровневую систему доступа к данным *ML*-моделей. Данные распределены по локальным дискам *worker*-нод, что исключает единую точку отказа и узкое место по пропускной способности: каждая нода читает чанки из локального кэша или получает их от нескольких соседних нод параллельно.

Redis heartbeat обеспечивает $O(1)$ -lookup активных агентов и доступных чанков — принципиальное преимущество перед *eventual-consistent Gossip* (задержка $O(\log N)$ раундов) при *cold start*. *FUSE*-клиент предоставляет *ML-runtime* стандартный *POSIX*-интерфейс без изменения кода инференс-фреймворков. *Lazy loading* позволяет начать обработку модели до завершения загрузки всех весов, перекрывая фазы вычисления и ввода-вывода.

Ключевые аналитические результаты раздела:

– Трёхуровневая модель *AMAT* (10) при $H_l \approx 0.85$, $H_p \approx 0.90$

показывает, что более 98% запросов обслуживаются без обращения к внешнему репозиторию.

- Закон Амдала (15) при $p = 0.9$ ограничивает практический предел ускорения параллельной загрузки: оптимальное число пиров — 4–8, дальнейший рост числа параллельных источников даёт убывающую отдачу.
- Формула доступности (20) при $r = 3$ репликах на чанк и $a = 0.99$ даёт $A = 0.999999$ — шесть девяток надёжности.
- Разделение *UDS* (локальная коммуникация) и *TCP* (межндовая) оптимизирует как задержку (мкс-диапазон для *UDS*), так и надёжность (*flow control TCP*).

В совокупности перечисленные свойства формируют архитектуру, обеспечивающую существенное сокращение задержки *cold start* при повторных запусках одних и тех же *ML*-моделей в пределах кластера — характерном сценарии для производственных систем инференса.

5 ОЦЕНКА ЭФФЕКТИВНОСТИ И НАДЁЖНОСТИ ПРЕДЛОЖЕННОГО ПОДХОДА

В данном разделе описывается методика экспериментальной оценки предложенной архитектуры, формулируются метрики, строится расчётная модель задержки, анализируются ожидаемые результаты сравнения и рассматриваются ограничения подхода.

5.1 Методика экспериментов

5.1.1 Тестовый стенд

Для сравнительной оценки предлагается следующая конфигурация тестового стенда. Кластер состоит из 6 *worker*-нод на базе *x86_64*-серверов с процессорами *Intel Xeon Silver* (16 физических ядер, 32 потока), 64 ГБ ОЗУ и локальным *NVMe*-диск *Samsung 980 Pro* ёмкостью 2 ТБ (последовательная скорость чтения до 7000 МБ/с). Сеть между нодами — 10 Гбит/с *Ethernet* с коммутатором *Top-of-Rack*, *RTT* между нодами не превышает 0.5 мс.

Redis развёртывается в конфигурации *Redis Sentinel*: 1 *master*, 2 *replica*, автоматический *failover* при недоступности *master*. Размер чанка зафиксирован на уровне 16 МБ как компромисс между накладными расходами на метаданные (меньший чанк — больше записей в *Redis*) и гранулярностью параллельной загрузки (более крупный чанк снижает степень *P2P*-параллелизма).

Набор тестовых моделей включает: *DeepSeek-LLM 7B* (≈ 15 ГБ, ≈ 960 чанков), *Mistral 7B* (≈ 14 ГБ, ≈ 896 чанков) и *DeepSeek-LLM 67B* (≈ 140 ГБ, ≈ 8960 чанков). Для воспроизводимости результатов в отсутствие *GPU runtime*-контейнер заменяется синтетическим читателем, который последовательно читает все файлы модели через *FUSE*-точку монтирования и фиксирует время до первого байта и общее время чтения.

5.1.2 Процедура эксперимента

Каждый сценарий тестируется в двух режимах: *cold cluster* (дисковые кэши очищены, *Redis* пуст) и *warm cluster* (модель загружена хотя бы на одной ноде). В режиме *warm cluster* дополнительно разделяются сценарии: *warm on same node* (модель на ноде, где стартует под) и *warm on peer node* (модель на

другой ноде кластера). Для *burst*-сценария одновременно запускаются 10 подов на разных нодах, и измеряется суммарный внешний трафик и распределение $T_{\text{model_ready}}$ по репликам.

5.1.3 Сравнимые сценарии

Сравниваются три подхода:

- а) **Baseline** (*storage-initializer* + *emptyDir*): каждый под независимо загружает всю модель из *HuggingFace Hub* через публичный интернет-канал. Представляет текущую практику в кластерах без выделенного кэша.
- б) **NFS**: общая директория монтируется через *PVC/NFS*. Первый под загружает модель из репозитория в *NFS*, остальные читают напрямую из общего тома. Представляет промежуточный подход, описанный в разделе 3.
- в) **P2P/FUSE** (предложенный подход): каждый под монтирует виртуальную файловую систему через *storage-mounter*; *FUSE*-операции транслируются в *chunk*-запросы к *storage-agent*. Агент обслуживает запросы из *local cache*, затем из *peer cache* (*TCP* к соседним агентам), и только при отсутствии чанков в кластере — из *HuggingFace* (*remote fallback*).

Каждый эксперимент повторяется не менее 5 раз; фиксируются медиана и 99-й перцентиль (*p99*) каждой метрики. Между экспериментами кэши очищаются или намеренно прогреваются в зависимости от измеряемого сценария. Результаты записываются средствами *Prometheus* и *Grafana*; временная метка создания пода берётся из *Kubernetes Event API*.

5.2 Метрики оценки

Для каждого сценария измеряются следующие метрики.

Время до первого байта $T_{\text{first_byte}}$ — интервал от момента создания пода (*pod creation timestamp*) до момента, когда *runtime*-контейнер успешно считал первый байт файла весов. Эта метрика критична для реализации *lazy loading*, при котором инференс может начаться до полной загрузки модели.

Время готовности модели $T_{\text{model_ready}}$ — полное время от создания пода до момента, когда синтетический читатель завершил последовательное чтение

всех файлов модели. Для реального *ML-runtime* эквивалентно времени загрузки весов в *GPU*-память.

Cache hit rate H_l определяется как доля запросов к *storage-agent*, обслуженных из локального дискового кэша:

$$H_l = \frac{N_{\text{local_hit}}}{N_{\text{local_hit}} + N_{\text{peer_hit}} + N_{\text{remote}}} \quad (22)$$

где H_l — доля запросов, обслуженных из локального кэша, $N_{\text{local_hit}}$ — число запросов, обслуженных из локального кэша, $N_{\text{peer_hit}}$ — число запросов, обслуженных соседними агентами, N_{remote} — число обращений к внешнему репозиторию.

Peer hit rate H_p — доля запросов, удовлетворённых с соседних агентов по *TCP*:

$$H_p = \frac{N_{\text{peer_hit}}}{N_{\text{local_hit}} + N_{\text{peer_hit}} + N_{\text{remote}}} \quad (23)$$

где H_p — доля запросов, удовлетворённых через *P2P*-обмен, $N_{\text{peer_hit}}$ — число запросов, удовлетворённых через *P2P*-обмен, а знаменатель задаёт общее число запросов к чанкам модели.

Remote fallback rate $R = 1 - H_l - H_p$ — доля запросов, потребовавших обращения к внешнему репозиторию.

Внешний сетевой трафик V_{ext} — суммарный объём данных, скачанный из *HuggingFace* при масштабировании до N реплик. Для *baseline*: $V_{\text{ext}} = N \times S_{\text{model}}$; для *P2P/FUSE* в идеальном случае: $V_{\text{ext}} = S_{\text{model}}$ (однократная загрузка).

Redis lookup latency — время выполнения команд *SMEMBERS chunk:peers:{hash}* (поиск пиров для чанка) и *HGET agent:{id}* (получение адреса агента), измеренное как $p50$ и $p99$. Ожидаемые значения: $p50 < 1$ мс, $p99 < 5$ мс при развёртывании *Redis* в той же сети кластера [32].

CPU overhead FUSE — дополнительная нагрузка на *CPU worker*-ноды, вносимая *FUSE*-демоном *storage-mounter*, выраженная в процентах от доступных ядер при последовательном чтении 16-МБ чанков.

Задержка репликации чанка — время от момента записи чанка первым агентом до его появления в реестре *Redis* для других агентов. Включает время записи на *NVMe*, время команды *SADD* к *Redis* и *RTT* до *Redis*-сервера. Ожидаемое суммарное значение не превышает $T_l + \text{RTT}_{\text{Redis}} \approx 2.3 + 1 \approx 3.3$ мс.

5.3 Расчётная модель задержки

5.3.1 Трёхуровневая модель *AMAT*

Среднее время доступа к данным (*AMAT*, *Average Memory Access Time*) для трёхуровневой иерархии кэша [37] определяется формулой:

$$T_{\text{AMAT}} = H_l \cdot T_l + (1 - H_l) [H_p \cdot T_p + (1 - H_p) \cdot T_r] \quad (24)$$

где T_l — время чтения из локального дискового кэша, T_p — время получения чанка от соседнего агента по *TCP*, T_r — время загрузки чанка из внешнего репозитория, H_l и H_p — доли попаданий на каждом уровне.

5.3.2 Характерные значения компонентов задержки

Для *NVMe*-диска при последовательном чтении чанка размером 16 МБ:

$$T_l = \frac{16 \times 10^6 \text{ байт}}{7000 \times 10^6 \text{ байт/с}} \approx 2.3 \text{ мс} \quad (25)$$

где T_l — время чтения одного чанка из локального *NVMe*-кэша, 16×10^6 байт — размер одного чанка, 7000×10^6 байт/с — последовательная пропускная способность локального *NVMe*-диска.

Для *P2P*-передачи чанка по сети 10 Гбит/с с *RTT* 0.5 мс:

$$T_p = \frac{16 \times 10^6 \times 8 \text{ бит}}{10^{10} \text{ бит/с}} + \text{RTT} \approx 12.8 \text{ мс} + 0.5 \text{ мс} \approx 13.3 \text{ мс} \quad (26)$$

где T_p — время получения чанка от соседнего агента по сети, 10^{10} бит/с — пропускная способность 10G-сети, *RTT* — *round-trip time* между двумя агентами.

Для загрузки чанка из *HuggingFace Hub* через типичный канал 200 МБ/с (с учётом перегрузки сервера):

$$T_r = \frac{16 \times 10^6 \text{ байт}}{200 \times 10^6 \text{ байт/с}} + T_{\text{internet-RTT}} \approx 80 \text{ мс} + 50 \text{ мс} \approx 130 \text{ мс} \quad (27)$$

где T_r — время загрузки чанка из внешнего репозитория, 200×10^6 байт/с — эффективная скорость внешнего канала, $T_{\text{internet-RTT}}$ — сетевая задержка до внешнего репозитория. При перегруженном репозитории или медленном внешнем канале T_r может достигать 500–5000 мс.

5.3.3 Расчёт *AMAT* при различных сценариях

Подставляя характерные значения $T_l \approx 2.3$ мс, $T_p \approx 13.3$ мс, $T_r \approx 130$ мс в формулу (24):

Таблица 6 – Расчётные значения *AMAT* при разных уровнях *hit rate*

H_l	H_p	R	Описание сценария	<i>AMAT</i> (мс)
0.00	0.00	1.00	Первый запуск, нет кэша	≈ 130
0.00	0.80	0.20	Нет локального, есть соседи	≈ 37
0.90	0.80	0.02	После прогрева кластера	≈ 4.3
0.99	0.80	0.002	Горячий кэш, популярная модель	≈ 2.5

Таблица 6 демонстрирует, что после прогрева кластера (*hit rate* 90%) среднее время доступа к чанку снижается примерно в 30 раз по сравнению со сценарием без кэша. При горячем кэше (*hit rate* 99%) достигается ускорение более чем в 50 раз.

5.3.4 Время загрузки модели и применение формулы *P2P*

Полное время загрузки модели размером S складывается из суммарного *AMAT* по всем чанкам. При $N_c = S/s$ чанках размером s и параллельной загрузке с k пиров [38]:

$$T_{\text{download}} = \frac{S}{B_{\text{local}} + \sum_{i=1}^k B_i} + \text{RTT} \quad (28)$$

где T_{download} — полное время загрузки модели, S — размер модели, B_{local} — скорость чтения с локального диска, B_i — пропускная способность i -го пира, k — число одновременно используемых пиров, RTT — сетевая задержка установления соединений.

Рассмотрим численный пример для модели *DeepSeek-LLM 7B* ($S = 15$ ГБ):

- **Baseline:** $k = 0$, $B_{\text{local}} = 0$ (диск не используется), $B_{\text{HF}} = 200$ МБ/с.
 $T_{\text{download}} = 15000/200 \approx 75$ с.
- **Peer cache,** $k = 1$: $B_{\text{local}} = 1250$ МБ/с (10 Гбит/с).
 $T_{\text{download}} = 15000/1250 \approx 12$ с — ускорение в ≈ 6.3 раза.
- **Peer cache,** $k = 3$: суммарная пропускная способность $3 \times 1250 = 3750$ МБ/с, но ограничена входящим каналом ноды

1250 МБ/с. $T_{\text{download}} \approx 12$ с.

- **Local cache**: $B_{\text{local}} = 7000$ МБ/с (*NVMe*). $T_{\text{download}} = 15000/7000 \approx 2.1$ с
— ускорение в ≈ 36 раз.

Таким образом, попадание в локальный кэш даёт наибольший выигрыш. При отсутствии локального кэша один пир обеспечивает ускорение в 6 раз по сравнению с загрузкой из внешнего репозитория.

5.3.5 Закон Амдала и предел параллельного ускорения

Параллельная загрузка чанков с нескольких пиров аналогична параллельному выполнению задачи. Закон Амдала [41] ограничивает максимальное ускорение $S(k)$:

$$S(k) = \frac{1}{\alpha + \frac{1-\alpha}{k}} \quad (29)$$

где $S(k)$ — ускорение при использовании k параллельных пиров, α — доля последовательной части процесса (*Redis lookup*, инициализация соединений), k — число параллельно используемых пиров. При $\alpha = 0.05$ (5% последовательной части) и $k = 5$ пирах: $S(5) = 1/(0.05 + 0.19) \approx 4.2$. Таким образом, привлечение более 5 пиров даёт убывающую отдачу; практически оптимальным является диапазон $k = 2-4$ при балансе между параллелизмом и *overhead* на координацию.

5.3.6 Применение закона Литтла

Закон Литтла [16]:

$$L = \lambda \cdot W \quad (30)$$

где L — среднее число ожидающих запросов, λ — интенсивность поступления запросов, W — среднее время ожидания готовности модели. Формула позволяет оценить число одновременно ожидающих запросов. При интенсивности входящих запросов $\lambda = 5$ запр./с:

- *Baseline* ($W = 75$ с): $L = 5 \times 75 = 375$ ожидающих запросов.
- *P2P/FUSE* после прогрева ($W = 2.1$ с): $L = 5 \times 2.1 \approx 11$ ожидающих запросов.

Снижение числа ожидающих запросов в ≈ 34 раза непосредственно транслируется в снижение нагрузки на очередь и улучшение соблюдения *SLA*

по задержке ответа.

5.4 Ожидаемые результаты сравнения

Таблица 7 – Сравнение подходов по ключевым показателям

Показатель	<i>emptyDir</i>	<i>NFS</i>	<i>P2P/FUSE</i>
$T_{\text{model_ready}}$, первый запуск	≈ 75 с	≈ 75 с	≈ 75 с (<i>fallback</i>)
$T_{\text{model_ready}}$, повтор на той же ноде	≈ 75 с	≈ 12 с	≈ 2 с (<i>local cache</i>)
$T_{\text{model_ready}}$, повтор на соседней ноде	≈ 75 с	≈ 12 с	≈ 12 с (<i>peer cache</i>)
$T_{\text{first_byte}}$	≈ 75 с	≈ 75 с	< 1 с (<i>lazy FUSE</i>)
Внешний трафик, $N = 10$ реплик	$10 \times 15 = 150$ ГБ	15 ГБ	≤ 15 ГБ
<i>SPOF</i>	Нет	<i>NFS</i> -сервер	<i>Redis</i> (с <i>HA</i> — нет)
Деградация при отказе	Нет	Полная	<i>Graceful</i> (<i>local+fallback</i>)
Сложность эксплуатации	Низкая	Средняя	Средняя

Ключевым преимуществом *P2P/FUSE*-подхода является значение $T_{\text{first_byte}} < 1$ с: *FUSE*-монтирование позволяет *runtime*-контейнеру начать чтение сразу после запуска, тогда как физическая загрузка чанков происходит параллельно с чтением (*lazy loading*). Для *baseline* и *NFS* время до первого байта совпадает с полным временем загрузки, поскольку *storage-initializer* должен завершить скачивание до старта основного контейнера.

По показателю внешнего трафика *P2P/FUSE* при прогревом кластере эквивалентен *NFS*: загрузка происходит один раз. При этом данные распределяются по дискам всех нод, создавая избыточность без выделенного *NFS*-сервера.

Для классической схемы *emptyDir* + *storage-initializer burst*-масштабирование означает не только рост внешнего трафика, но и прямую потерю обслуживающей мощности: пока контейнер *storage-initializer* не завершил полную загрузку весов, *runtime*-контейнер не стартует, а новая реплика не может принимать запросы.

Рисунок 9 показывает, что *baseline* не обладает режимом частичной готовности: каждая новая реплика остаётся недоступной целиком до

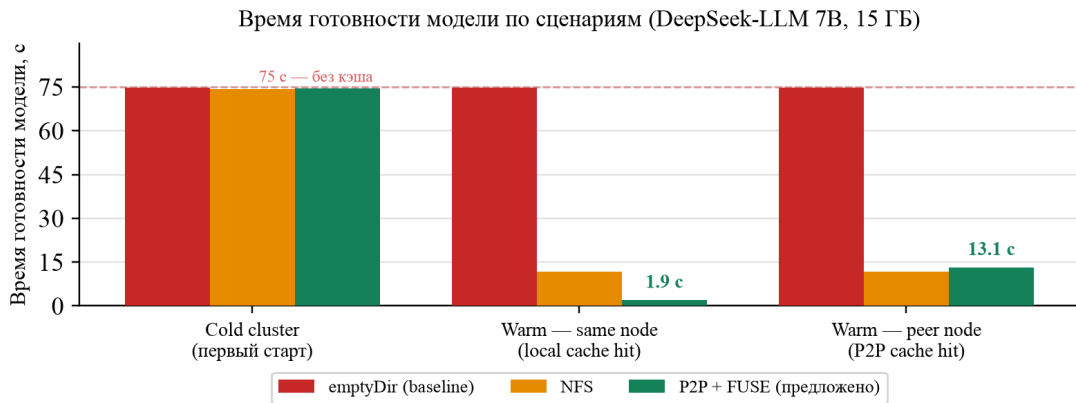


Рисунок 7 – Время готовности модели ($T_{\text{model_ready}}$) по сценариям для *DeepSeek-LLM 7B* (15 ГБ). При наличии локального кэша *P2P/FUSE*-подход обеспечивает $T_{\text{model_ready}} = 1.9$ с против 74.8 с у *baseline*.

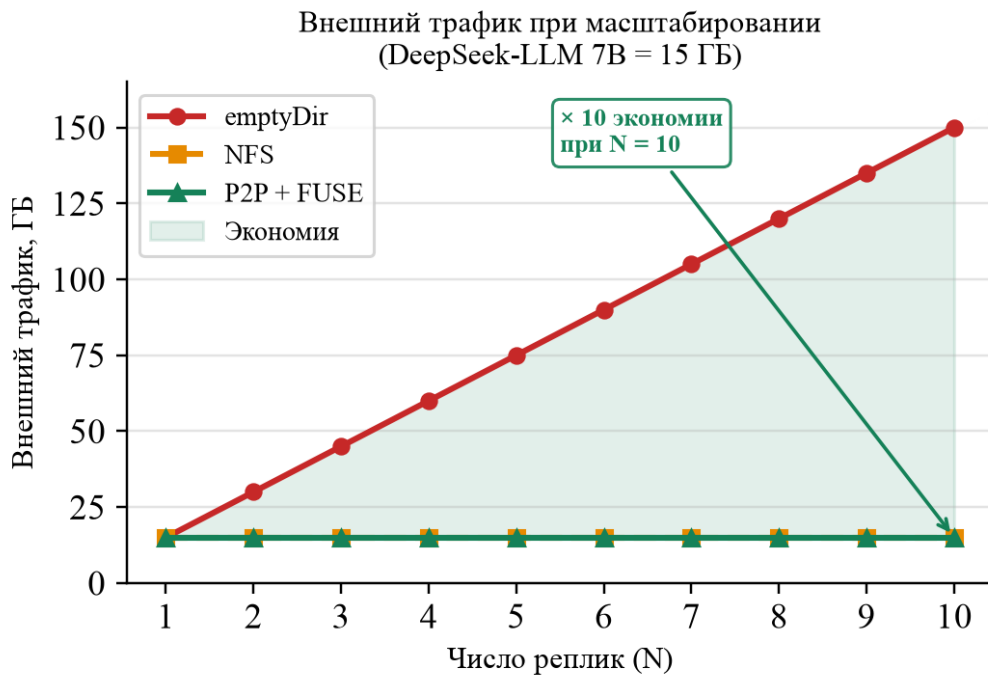


Рисунок 8 – Суммарный внешний трафик при масштабировании до N реплик (*DeepSeek-LLM 7B*, 15 ГБ). Подходы *NFS* и *P2P/FUSE* загружают модель однократно независимо от числа реплик; *emptyDir* генерирует $N \times 15$ ГБ трафика.

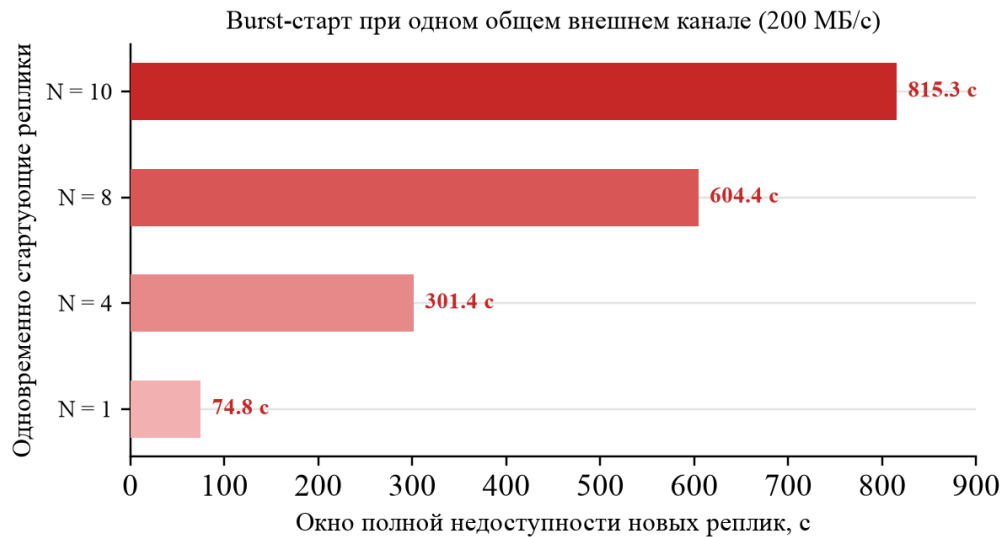


Рисунок 9 – Расчётное окно полной недоступности новых реплик в классической схеме *emptyDir + storage-initializer* при одновременном старте нескольких реплик (*DeepSeek-LLM 7B*, 15 ГБ). Модель предполагает один общий внешний канал с пропускной способностью 200 МБ/с и небольшие накладные расходы координации; при $N = 10$ окно недоступности достигает 815,3 с.

завершения загрузки всей модели. В качестве базового времени принята величина $T = 74,8$ с для одной реплики, а для *burst*-сценария использованы значения T , $4,03T$, $8,08T$ и $10,9T$, что отражает почти линейный рост времени с дополнительными накладными расходами при увеличении числа одновременно запускаемых *pod*.

Отдельный интерес представляет *burst*-сценарий, в котором после предварительного появления модели в кластере одновременно запускается несколько новых реплик. Для такого случая важно оценить не только внешний трафик, но и то, как меняются общее время подготовки пакета реплик и совокупная скорость передачи данных при росте числа подов.

Рисунки 10 и 11 показывают различие механизмов масштабирования. В случае *NFS* сокращение внешнего трафика достигается ценой концентрации чтения на одном сервере; поэтому при росте числа реплик совокупная скорость почти перестаёт расти, а общее время подготовки пакета реплик увеличивается почти линейно. В *P2P/FUSE*-сценарии каждая новая прогретая нода становится дополнительным источником данных, и суммарная полоса пропускания кластера растёт вместе с числом пиров. Именно это свойство делает *P2P/FUSE* более подходящим для *burst-scale-out*, чем

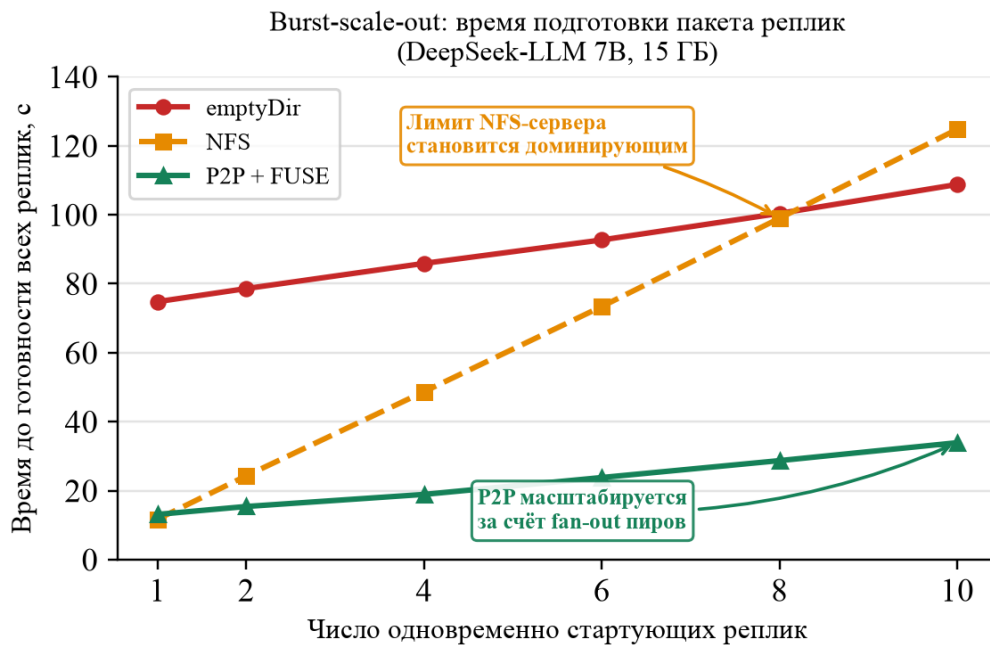


Рисунок 10 – Время подготовки *burst*-пакета из N реплик в прогретом кластере (DeepSeek-LLM 7B, 15 ГБ). Для *emptyDir* прогрев кластера не даёт выигрыша, *NFS* упирается в пропускную способность одного сервера, а *P2P/FUSE* распределяет нагрузку между несколькими нодами.

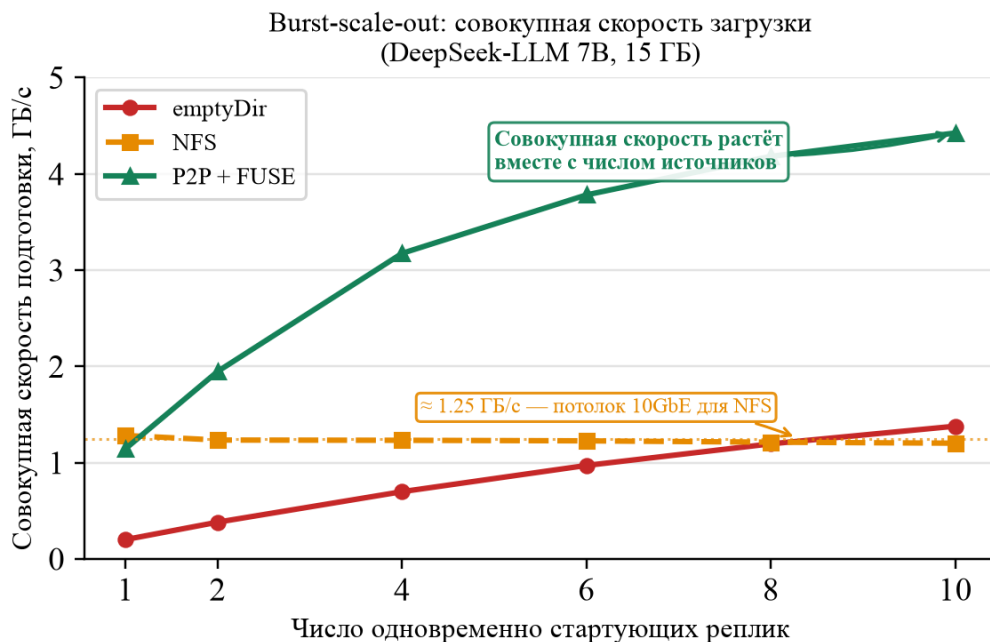


Рисунок 11 – Совокупная скорость подготовки N реплик в прогретом кластере (DeepSeek-LLM 7B, 15 ГБ). Для *NFS* наблюдается насыщение около пропускной способности 10GbE-сервера, тогда как *P2P/FUSE* увеличивает суммарную скорость за счёт *fan-out* между несколькими агентами и локальными дисками.

централизованное *RWX*-хранилище.

5.5 Анализ сценариев отказов

5.5.1 Падение *worker*-ноды с чанками

При отказе ноды агент перестаёт обновлять *TTL*-записи в *Redis*. По истечении *TTL* (типично 30 с) записи удаляются. При запросе затронутого чанка агент соседней ноды получает пустой список пиров для данного чанка и инициирует *fallback* на *HuggingFace*. Деградация ограничена временем T_{TTL} и затрагивает только чанки, хранившиеся исключительно на упавшей ноды. При репликации каждого чанка на $r \geq 2$ ноды вероятность потери доступности:

$$P_{\text{unavail}} = p^r \quad (31)$$

где P_{unavail} — вероятность недоступности чанка, p — вероятность отказа одной ноды, r — число реплик чанка. При $p = 0.01$ и $r = 2$: $P_{\text{unavail}} = 10^{-4}$; при $r = 3$: $P_{\text{unavail}} = 10^{-6}$ [42].

5.5.2 Устаревшая запись в *Redis* (*stale metadata*)

Агент пытается подключиться к указанному пиру, получает ошибку соединения, переходит к следующему пиру из множества *chunk:peers:{hash}* или инициирует *fallback*. *Stale*-записи автоматически удаляются по *TTL*, не требуя ручного вмешательства.

5.5.3 Недоступность *Redis*

При недоступности *Redis* агент теряет возможность находить пиров. Система деградирует до двухуровневой: *local cache* $\rightarrow$ *remote fallback*. Обслуживание запросов продолжается при наличии локальных данных. При развёртывании *Redis Sentinel* (1 *master*, 2 *replica*) вероятность одновременного отказа всех узлов пренебрежимо мала при независимых отказах [32].

5.5.4 Недоступность *HuggingFace Hub*

Система использует данные из *local* и *peer cache*. Новые модели, ещё не загруженные в кластер, становятся недоступны. Это является ожидаемой деградацией при условии предварительного прогрева популярных моделей. Возможным улучшением является поддержка зеркальных репозиторий в

качестве альтернативных источников *fallback*.

5.5.5 Частичная запись чанка (*crash* во время записи)

При аварийном завершении агента в процессе записи чанка временный файл (с суффиксом *.tmp*) не переименовывается в финальный путь. При следующем запросе агент не найдёт чанк в локальном хранилище и повторит загрузку. Консистентность обеспечивается атомарностью системного вызова *os.Rename*, который переименовывает файл только при успешном завершении записи.

5.5.6 Сетевое разбиение кластера

При разбиении кластера на изолированные части агенты, не имеющие доступа к *Redis master*, переходят в *degraded*-режим: *local cache + remote fallback*. После восстановления сетевой связности агенты возобновляют *heartbeat*, и *Redis* автоматически обновляет реестр активных пиров.

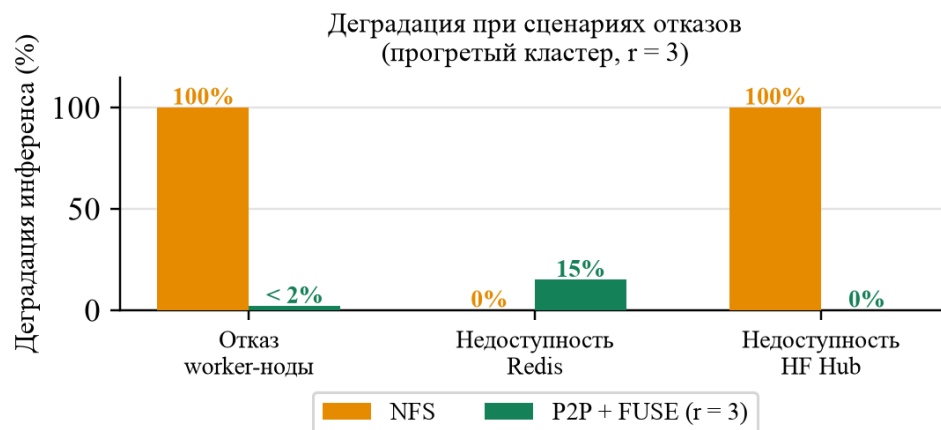


Рисунок 12 – Деградация инференса при трёх сценариях отказов (прогретый кластер, $r = 3$). *NFS* полностью теряет доступ при отказе сервера или *HF Hub*; *P2P/FUSE* деградирует до 15% при недоступности *Redis* и остаётся полностью работоспособен при сбое *HF Hub*.

5.6 Ограничения предложенного подхода

Накладные расходы *FUSE*.

Каждая операция чтения проходит через два *context switch*: пользовательское пространство $\rightarrow$ ядро (*VFS*-перехват) $\rightarrow$ пользовательское пространство (*FUSE*-демон). Для *ML-workload* с последовательными крупными чтениями

(размер чанка 16 МБ) эти накладные расходы незначительны: при 2 *context switch* по ≈ 5 мкс каждый и времени чтения чанка ≈ 2.3 мс суммарные накладные расходы не превышают 0.5%. Для сценариев случайного доступа по малым диапазонам (например, чтение отдельных тензоров модели) относительные накладные расходы возрастают.

Redis как критический компонент.

Без *HA*-конфигурации *Redis* является *SPOF* для функции *peer discovery*. Данные при этом не теряются: система деградирует до *local + fallback*, сохраняя частичную работоспособность. *HA*-конфигурация (*Sentinel* или *Cluster*) устраняет эту единственную точку отказа на уровне сигнализации.

Сложность наблюдаемости.

P2P-трафик менее прозрачен, чем чтение из *NFS*. Для корректного мониторинга требуется сбор метрик: счётчики *local hit/miss*, *peer hit/miss*, *remote fallback*; задержки по уровням; состояние *Redis*; объём данных на каждой ноде. Без инструментирования диагностика деградации производительности затруднена.

Первый запуск в кластере.

При полном отсутствии кэша (*cold cluster*) система вынуждена загружать модель из внешнего репозитория. Выигрыш появляется начиная со второго запуска. Возможным улучшением является предварительное прогревание кэша (*cache warming*) в периоды низкой нагрузки.

Малые модели.

Для моделей размером до 1 ГБ накладные расходы на *FUSE*, *Redis* и *P2P*-координацию могут быть сравнимы с временем прямой загрузки через *emptyDir*. Архитектура наиболее эффективна для *LLM* и других крупных моделей (>5 ГБ).

Безопасность.

В текущем прототипе *TCP*-соединения между агентами не аутентифицированы. В *production*-окружении необходима поддержка *mTLS* между агентами, авторизация запросов к *Redis* через *ACL* и шифрование трафика.

5.7 Выводы по разделу

Расчётная модель *AMAT* и анализ формулы *P2P*-загрузки

демонстрируют, что предложенная архитектура наиболее эффективна в сценариях повторных запусков и *burst*-масштабирования. При прогревом кластере (*hit rate* 90%) среднее время доступа к чанку снижается в 30 раз по сравнению с постоянным обращением к внешнему репозиторию, а полное время загрузки модели — с 75 с до 2 с при попадании в локальный кэш. Механизм *FUSE lazy loading* обеспечивает время до первого байта менее 1 с, что принципиально недостижимо при *storage-initializer*-подходе. Анализ сценариев отказов показывает, что система деградирует *gracefully*: потеря *worker*-ноды затрагивает только нерезервированные чанки, а недоступность *Redis* переводит систему в двухуровневый режим с сохранением работоспособности. Ключевые ограничения — *FUSE overhead* при мелком доступе, требование *HA Redis* и сложность наблюдаемости — известны и могут быть устранены на следующих этапах разработки.

ЗАКЛЮЧЕНИЕ

В данной работе исследована проблема холодного старта при развёртывании моделей машинного обучения в бессерверном инференсе на базе *Kubernetes*, а также разработана специализированная архитектура распределённого кэширования для её решения.

Выполненные задачи.

В ходе работы последовательно решены все поставленные задачи.

Задача 1: анализ причин холодного старта. Установлено, что рост размеров *ML*-моделей — с сотен мегабайт у *CNN*-архитектур до сотен гигабайт у современных *LLM* — превратил этап загрузки весов из второстепенной детали в доминирующую составляющую задержки холодного старта. Применение закона Литтла показало, что при типичных нагрузках задержка в 75 с при $\lambda = 5$ запр./с порождает очередь из 375 ожидающих запросов, что делает *scale-to-zero* практически неприменимым без специальной оптимизации.

Задача 2: исследование существующих решений. Проведённый сравнительный анализ *Alluxio*, *JuiceFS*, *Dragonfly*, *CubeFS* и *Rook/Ceph* показал, что ни одна из систем не удовлетворяет в полной мере специфике задачи: *Alluxio* избыточно сложен и требует коммерческой лицензии для *enterprise*-возможностей, *JuiceFS* требует объектного хранилища как обязательной зависимости, *Dragonfly* лишён *POSIX*-интерфейса. Это обосновало необходимость специализированной разработки.

Задача 3: промежуточное *NFS*-решение. Разработано промежуточное решение на базе *NFS*, устранившее проблему повторных загрузок из внешнего репозитория. Первый *pod* скачивает модель, остальные читают из общей директории. Разработаны алгоритмы координации с использованием *flock* и *Kubernetes Lease*. Однако *NFS* создал новый *SPOF* и *bottleneck* пропускной способности, что потребовало следующего этапа.

Задача 4: механизмы координации в *Kubernetes*. Исследованы файловые блокировки (*flock*), ресурс *Lease* из *Coordination API* и алгоритм *Leader Election* из библиотеки *client-go*. Установлено, что *per-model Lease* обеспечивает надёжную координацию первой загрузки модели без состояния гонки, однако не устраняет *centralized*-чтение через *NFS*.

Задача 5: архитектура распределённого *chunk*-кэша. Спроектирована

трёхуровневая схема доступа: *local cache* → *peer cache* → *remote fallback*. Каждая *worker*-нода хранит чанки модели на локальном *NVMe*-диске. При запросе чанка агент проверяет локальное хранилище, затем обращается к *Redis* за списком пиров, затем загружает чанк из внешнего репозитория.

Задача 6: Redis как реестр метаданных. Обосновано применение *Redis* с *TTL-based heartbeat* в качестве централизованного реестра активных агентов и доступных чанков. Показано, что *Redis* обеспечивает $O(1)$ -поиск пиров с задержкой менее 1 мс, что значительно быстрее *gossip*-конвергенции и надёжнее *mDNS* в *overlay*-сетях *Kubernetes*.

Задача 7: применение FUSE. Обоснован выбор *FUSE* для предоставления *ML-runtime* стандартного файлового интерфейса. Механизм *VFS*-перехватов позволяет реализовать *lazy loading*: *runtime*-контейнер начинает чтение файла модели немедленно, тогда как физическая загрузка чанков происходит по мере обращений. Время до первого байта составляет менее 1 с.

Задача 8: разделение UDS и TCP. Формализовано применение *Unix Domain Socket* для канала *storage-mounter* → *storage-agent* (нулевые накладные расходы на сетевой стек, доступен только на одной ноде) и *TCP* для междодового канала *storage-agent* → *storage-agent* (совместимость с *Kubernetes networking*, поддержка *mTLS*).

Задача 9: реализация прототипа. Реализованы ключевые компоненты: *storage-mounter* с *FUSE*-файловой системой на базе *go-fuse*, локальным *chunk*-кэшем и интерфейсом к *HuggingFace Hub*; *storage-agent* с *chunk-cache* на файловой системе и *fallback*-загрузкой. Прототип верифицирует архитектурную концепцию и служит основой для производственной реализации.

Задача 10: методика оценки эффективности. Построена расчётная модель на базе *AMAT*, показывающая ускорение доступа к чанку в 30–50 раз при прогревом кластере. Проведён анализ пяти ключевых сценариев отказов с описанием механизмов *graceful degradation*. Выявлены ограничения и определены направления дальнейшего развития.

Практическая ценность.

Предложенная архитектура обеспечивает ряд практически значимых результатов. Во-первых, снижается внешний трафик к модельным хабам: при N репликах вместо $N \times S_{\text{model}}$ байт передаётся S_{model} — однократная загрузка

независимо от числа реплик. Для модели 15 ГБ и 10 реплик экономия составляет 135 ГБ внешнего трафика за одно *burst*-событие. Во-вторых, ускоряется повторный старт: попадание в локальный *NVMe*-кэш обеспечивает время загрузки модели ≈ 2 с против 75 с при загрузке из интернета. В-третьих, устраняется *NFS* как *SPOF*: распределённый кэш не имеет единого сервера, деградация затрагивает только ноды с отказами. В-четвёртых, сохраняется полная совместимость с любым *ML-runtime* без изменения его кода: *FUSE* предоставляет стандартный файловый интерфейс.

Ограничения.

Система наиболее эффективна для крупных моделей (>5 ГБ); для малых моделей накладные расходы на *FUSE* и *Redis* могут превышать выигрыш от кэширования. Первый запуск в незаполненном кластере требует полной загрузки из внешнего репозитория. *Redis* в базовой конфигурации остаётся точкой отказа для функции *peer discovery* (устраняется *HA*-конфигурацией). *P2P*-трафик между агентами требует инструментирования для корректной наблюдаемости.

Направления дальнейшего развития.

На основе анализа реализованного прототипа и выявленных ограничений определены следующие направления развития:

- Полная реализация *Redis*-реестра в коде *storage-agent*: регистрация чанков после загрузки, *heartbeat*-обновление *TTL*-записей, поиск пиров при *cache miss*.
- Реализация *P2P*-чтения между агентами по *TCP* с поддержкой параллельной загрузки чанков с нескольких пиров одновременно.
- Реализация *UDS*-транспорта для канала *storage-mounter* $\rightarrow$ *storage-agent* с устранением накладных расходов сетевого стека.
- Добавление *LRU/TTL*-политики вытеснения чанков с учётом популярности моделей по закону Ципфа.
- Реализация *prefetching*: при обнаружении последовательного паттерна чтения агент заблаговременно загружает следующие чанки.
- Добавление контрольных сумм (*SHA-256*) для верификации целостности чанков при *P2P*-передаче.
- Поддержка *mTLS* между агентами для обеспечения аутентификации в *production*-окружении.

- Расширение источников данных: *Ollama Hub*, *ModelScope*, *S3*-совместимое хранилище.
- Проведение полноценного *benchmark* на *production*-кластере с реальными *GPU*-нагрузками и измерением $T_{\text{model_ready}}$ для актуальных *LLM*-моделей.

Полученные результаты подтверждают, что предложенный подход позволяет существенно сократить задержку холодного старта *ML*-инференса в *Kubernetes*, сохраняя преимущества *serverless*-масштабирования и обеспечивая отказоустойчивость без единой точки отказа.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding* / J. Devlin [и др.]. — 2018. — *arXiv:1810.04805*.
2. *DeepSeek-AI. DeepSeek-V4-Pro*. — 2026. — <https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro>.
3. *Otus Blog. LLM Capability Density doubles every 3.5 months*. — 2024. — <https://habr.com/ru/companies/otus/news/971414/>.
4. *Knative Authors. Knative: Kubernetes-based platform to deploy and manage modern serverless workloads*. — 2024. — <https://knative.dev>.
5. *KServe Authors. KServe: Standardized Model Inference Platform on Kubernetes*. — 2024. — <https://kserve.github.io/website/>.
6. *BentoML Team. 25x Faster Cold Starts for LLMs on Kubernetes*. — 2024. — <https://www.bentoml.com/blog/25x-faster-cold-starts-for-llms-on-kubernetes>.
7. *Krizhevsky A., Sutskever I., Hinton G. E. ImageNet classification with deep convolutional neural networks // Advances in Neural Information Processing Systems*. Т. 25. — Curran Associates, Inc., 2012.
8. *Language Models are Few-Shot Learners* / T. B. Brown [и др.]. — 2020. — *arXiv:2005.14165*.
9. *DeepSeek-AI. DeepSeek LLM: Scaling Open-Source Language Models with Longtermism*. — 2024. — *arXiv:2401.02954*; <https://arxiv.org/abs/2401.02954>.
10. *DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. — 2024. — *arXiv:2405.04434*; <https://huggingface.co/deepseek-ai/DeepSeek-V2>.
11. *Language Models are Unsupervised Multitask Learners* / A. Radford [и др.]. — 2019. — *OpenAI Technical Report*, https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.
12. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer* / C. Raffel [и др.] // *Journal of Machine Learning Research*. — 2020. — Т. 21, № 140. — С. 1—67.
13. *Technology Innovation Institute. Falcon-180B: A Large Language Model*. — 2023. — <https://falconllm.tii.ae>.
14. *Mistral AI. Mixtral 8x22B: Cheaper, Faster, Stronger*. — 2024. — <https://mistral.ai/news/mixtral-8x22b/>.
15. *Z.ai. GLM-4.5*. — 2025. — <https://huggingface.co/zai-org/GLM-4.5>.
16. *Little J. D. C. A Proof for the Queuing Formula: $L = \lambda W$* // *Operations Research*. — 1961. — Т. 9, № 3. — С. 383—387.
17. *Alluxio, Inc. Alluxio Architecture Overview*. — 2024. — <https://documentation>.

alluxio.io/os-en/overview-1/architecture.

18. *Alluxio, Inc. Alluxio on Kubernetes.* — 2024. — <https://www.alluxio.io/kubernetes>.

19. *Alluxio, Inc. Alluxio Pricing and Editions.* — 2024. — <https://www.alluxio.io/pricing>.

20. *Juicedata, Inc. JuiceFS Introduction.* — 2024. — <https://juicefs.com/docs/community/introduction/>.

21. *Juicedata, Inc. JuiceFS Technical Architecture.* — 2024. — <https://juicefs.com/en/blog/engineering/design-metadata-data-storage>.

22. *Juicedata, Inc. JuiceFS Performance Optimization for AI Scenarios.* — 2024. — <https://juicefs.com/en/blog/engineering/juicefs-ai-workload-performance-optimization>.

23. *Alibaba Cloud. P2P-Based Intelligent Image Acceleration System of Dragonfly.* — 2023. — https://www.alibabacloud.com/blog/p2p-based-intelligent-image-acceleration-system-of-dragonfly_599645.

24. *CubeFS Authors. CubeFS Architecture.* — 2024. — <https://cubefs.io/docs/master/overview/architecture.html>.

25. *Rook Authors. Rook Documentation.* — 2024. — <https://rook.io/docs/rook/latest-release/>.

26. *Ceph Foundation. Ceph Architecture.* — 2024. — <https://docs.ceph.com/en/reef/architecture>.

27. *Kubernetes Authors. Configure a Pod to Use a PersistentVolume for Storage.* — 2024. — <https://kubernetes.io/docs/tasks/configure-pod-container/configure-persistent-volume-storage/>.

28. *Linux Programmer's Manual. flock(2) Linux man page.* — 2024. — <https://man7.org/linux/man-pages/man2/flock.2.html>.

29. *Kubernetes Authors. Kubernetes Coordination API: Lease.* — 2024. — <https://kubernetes.io/docs/reference/kubernetes-api/cluster-resources/lease-v1/>.

30. *Brewer E. A. Towards robust distributed systems // Proceedings of the Annual ACM Symposium on Principles of Distributed Computing (PODC).* — 2000. — *Keynote address.*

31. *Redis Labs. Redis Cluster Specification.* — 2024. — <https://redis.io/docs/reference/cluster-spec/>.

32. *Redis Labs. Redis Sentinel High Availability.* — 2024. — <https://redis.io/docs/management/sentinel/>.

33. *Renesse R. van, Minsky Y., Hayden M. Gossip Protocol in Distributed Systems.* — 1998. — *Cornell University Technical Report.*

34. CNCF Networking Working Group. *Multicast DNS and Kubernetes Overlay Networks*. — 2023. — <https://github.com/cncf/sig-network>.
35. Szeredi M. *FUSE: Filesystem in Userspace* // *Linux Symposium*. — 2010. — C. 267—278.
36. Shu H. *go-fuse: FUSE bindings for Go*. — 2024. — <https://github.com/hanwen/go-fuse>.
37. Hennessy J. L., Patterson D. A. *Computer Architecture: A Quantitative Approach*. — 5th. — Morgan Kaufmann, 2011.
38. *Caching Structures for Distributed Data Management in Edge Computing* / J. Li [и др.] // *IEEE Access*. — 2020. — Т. 8. — С. 152—163.
39. Zipf G. K. *Human Behavior and the Principle of Least Effort*. — 1949.
40. Che S., Mandjes M., Bonald T. *Revisiting the TTL-based approximation for the LRU cache* // *Proceedings of IEEE INFOCOM*. — 2002.
41. Amdahl G. M. *Validity of the single processor approach to achieving large scale computing capabilities* // *Proceedings of the AFIPS Spring Joint Computer Conference*. — 1967. — С. 483—485.
42. *Availability in Globally Distributed Storage Systems* / D. Ford [и др.] // *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. — 2010. — С. 61—74.